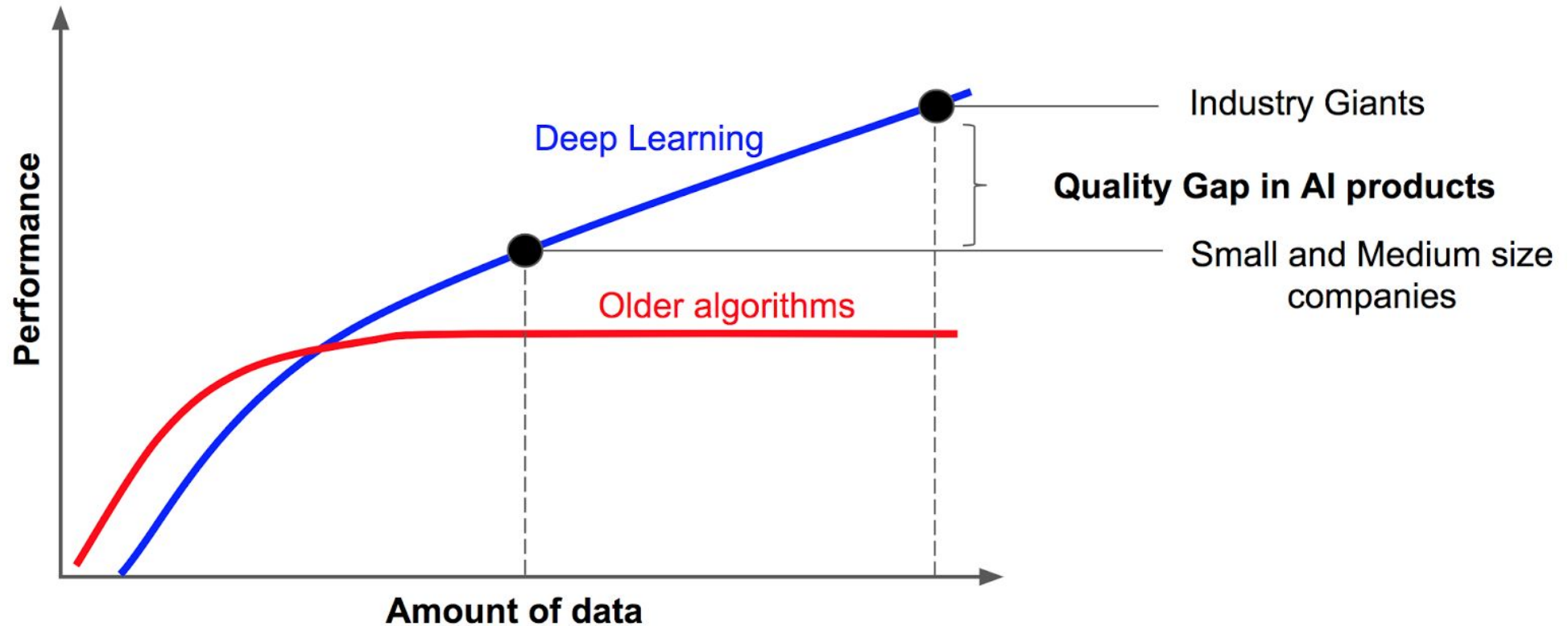


Neural Networks

Máster en Innovación Tecnológica (MINT)

imanolgranada@gmail.com

Why Neural Networks and Deep Learning?



Applications:

- Virtual Assistants
- Chatbots
- News Aggregation and Fake News Detection
- Composing Music
- Image Coloring
- Robotics
- Image Captioning
- Self Driving Cars
- Natural Language Processing
- Visual Recognition
- Automatic Machine Translation

Inglés

↔

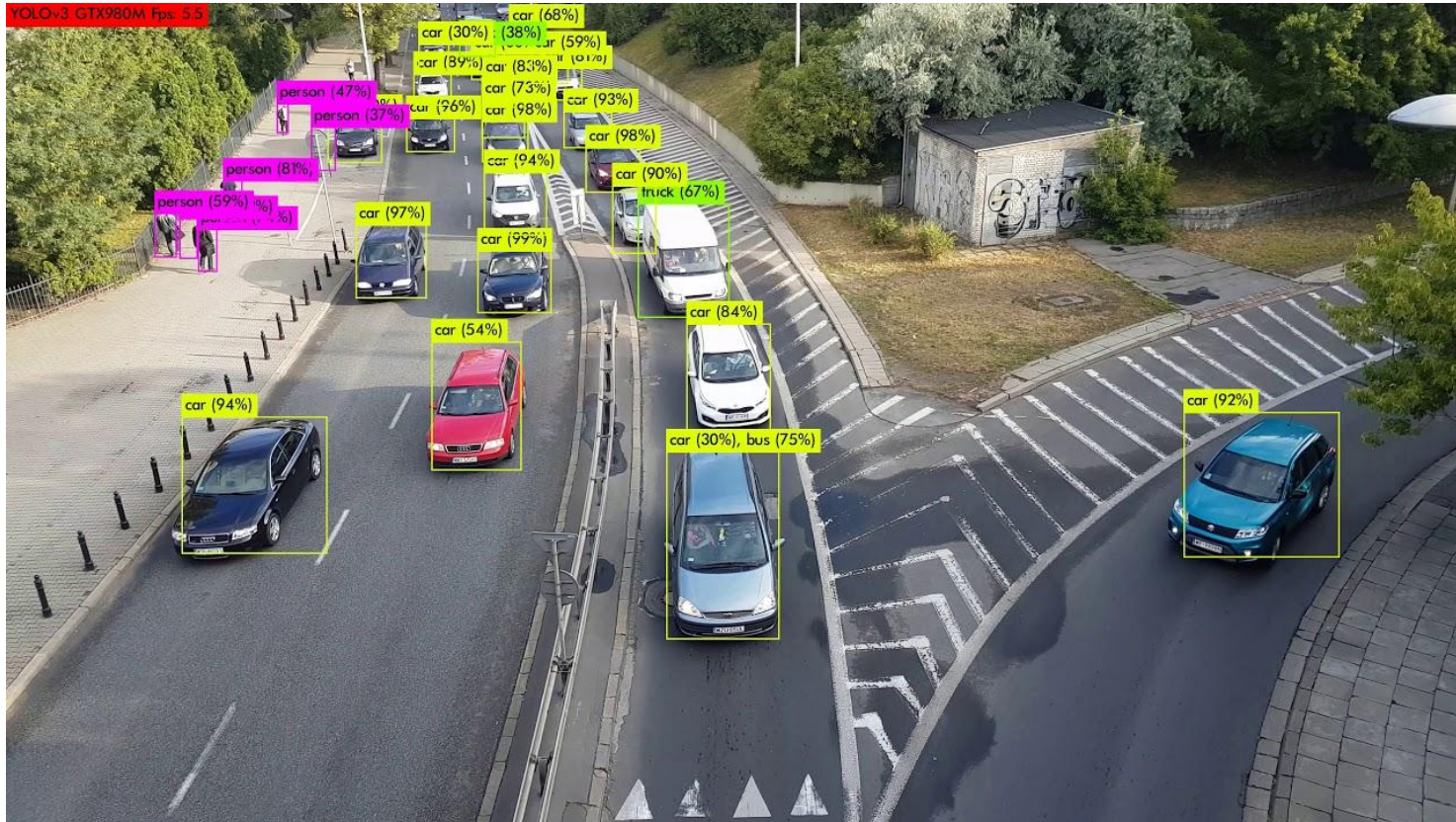
Español

×

This has been translated with neural networks like the ones you are about to learn.

Esto se ha traducido con redes neuronales como las que estás a punto de aprender.

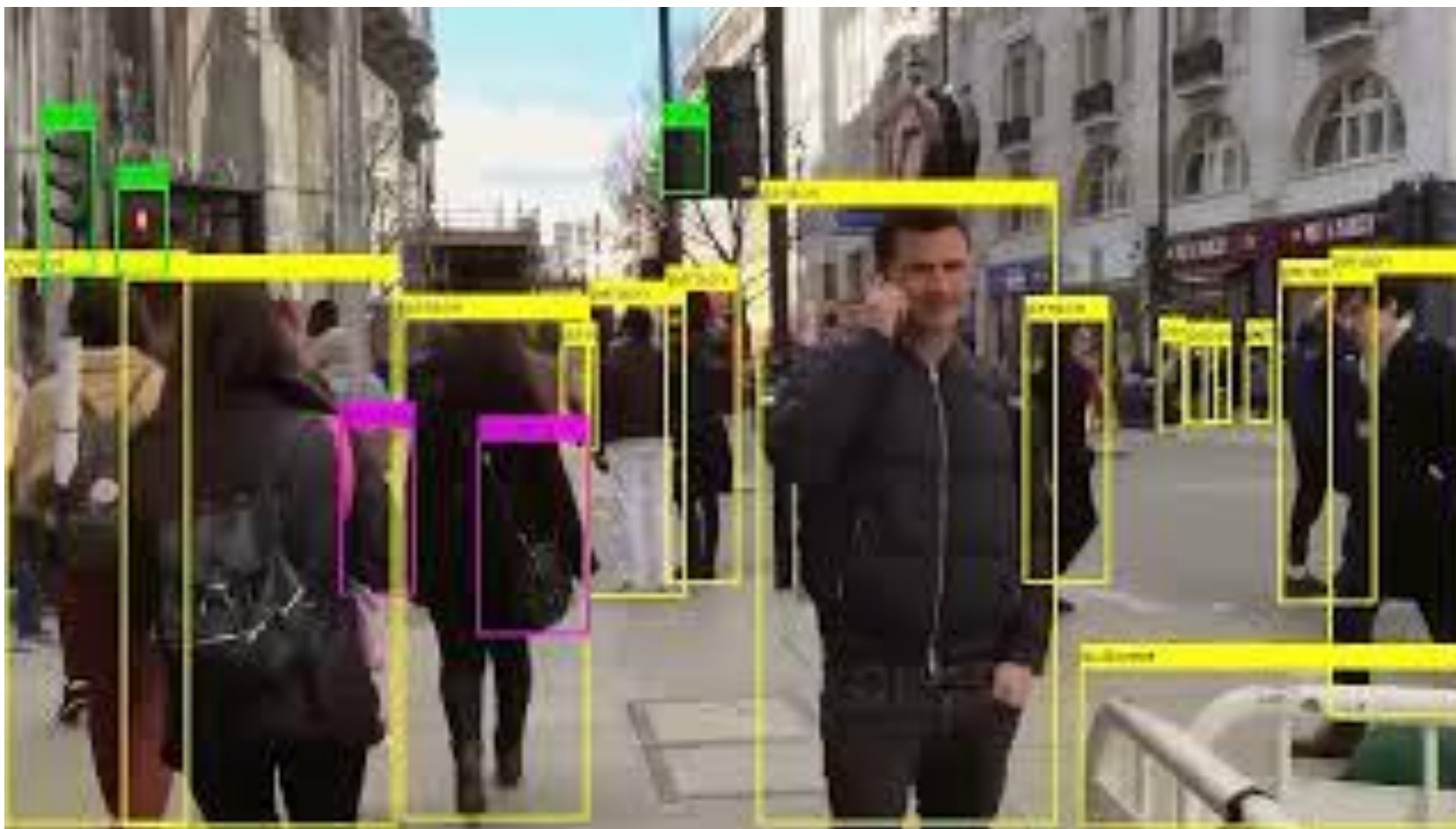
Applications:



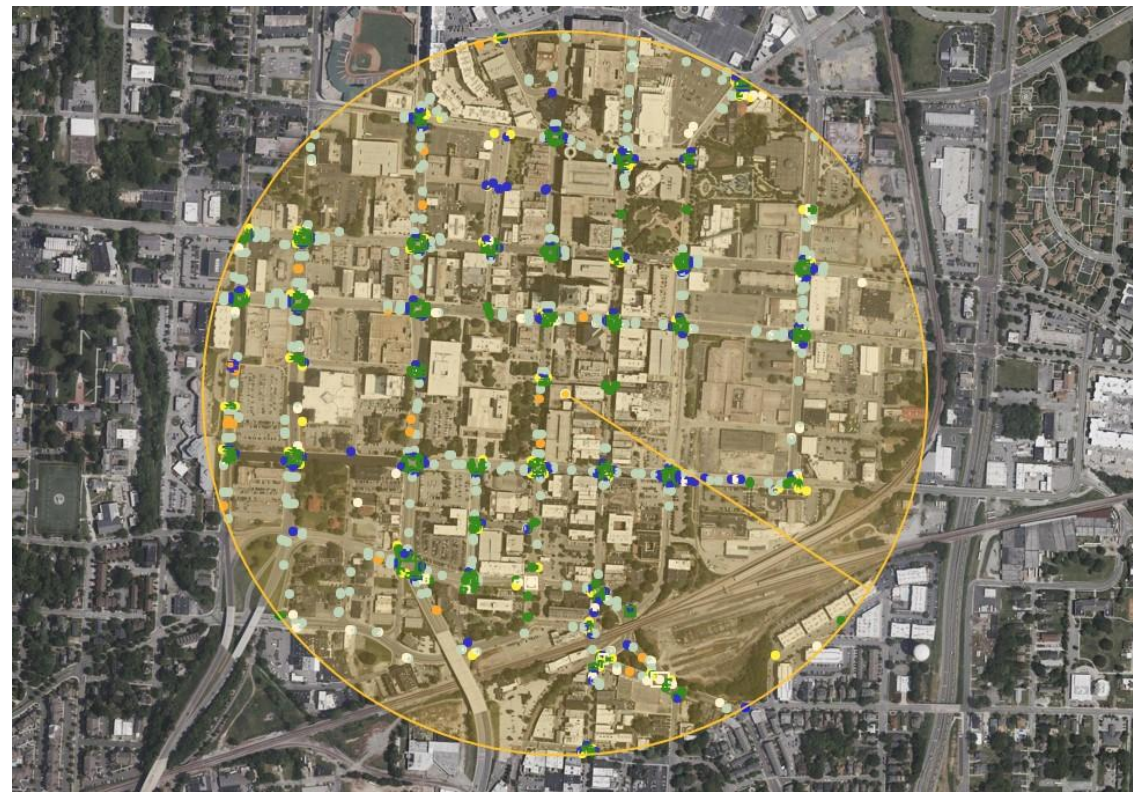
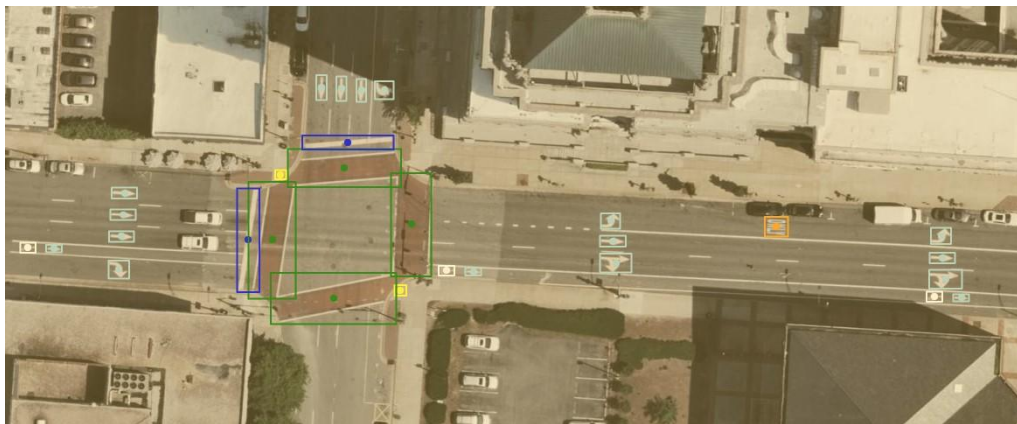
YOLO : You Only Look Once!

https://www.youtube.com/watch?v=cHDLvp_NPOk

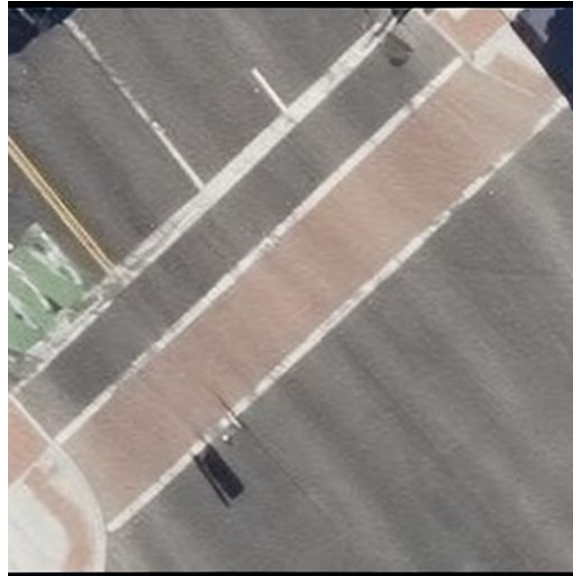
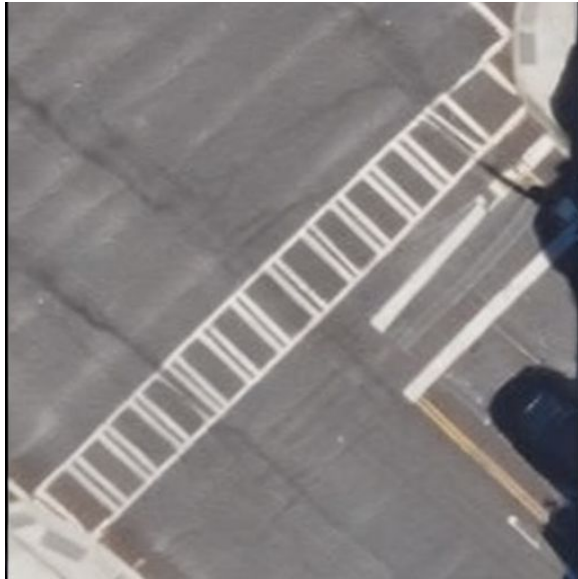
Applications:



Applications:

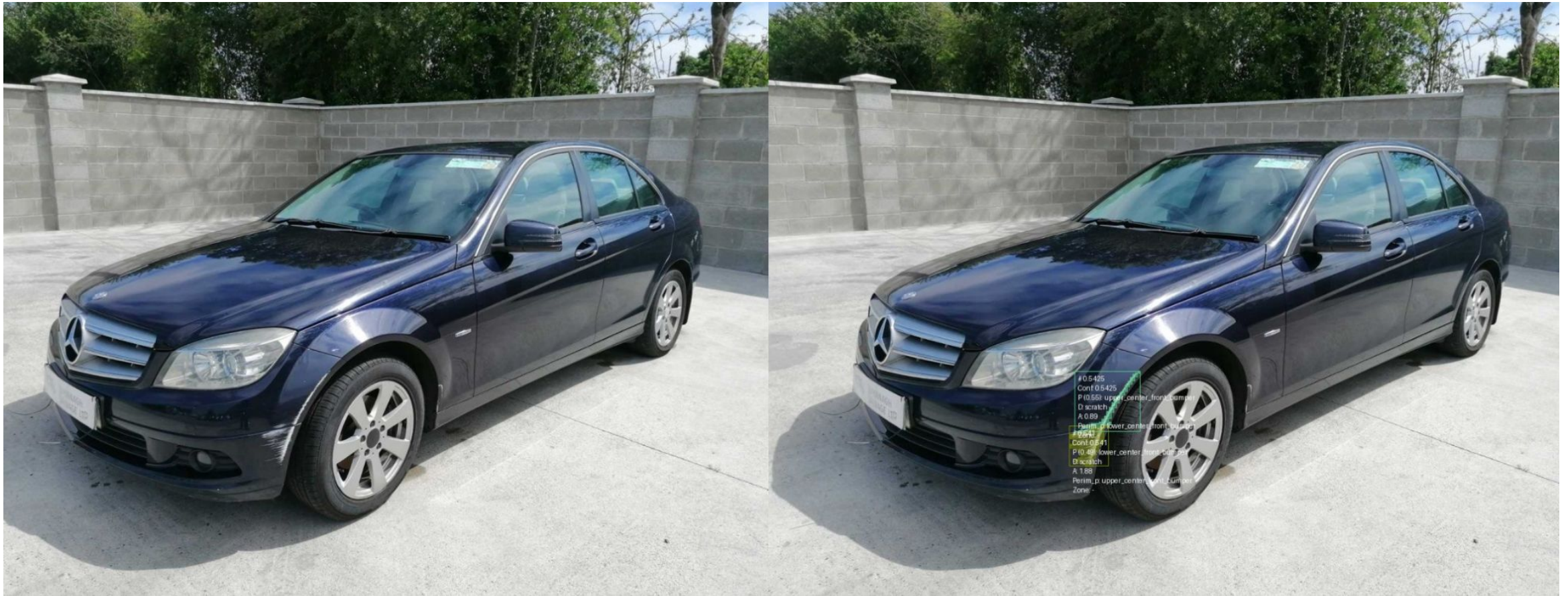


Applications:



Stable diffusion for new samples

Applications:



ChatGPT:

 Make me a complete pytorch program with a CNN that learns the MNIST dataset

 Sure! Below is a complete PyTorch program for a Convolutional Neural Network (CNN) that learns the MNIST dataset. We'll use the PyTorch framework to build the CNN, train it on the MNIST dataset, and evaluate its performance.   

Make sure you have PyTorch and torchvision installed:

```
bash
pip install torch torchvision
```

Now, let's create the PyTorch program:

```
python
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

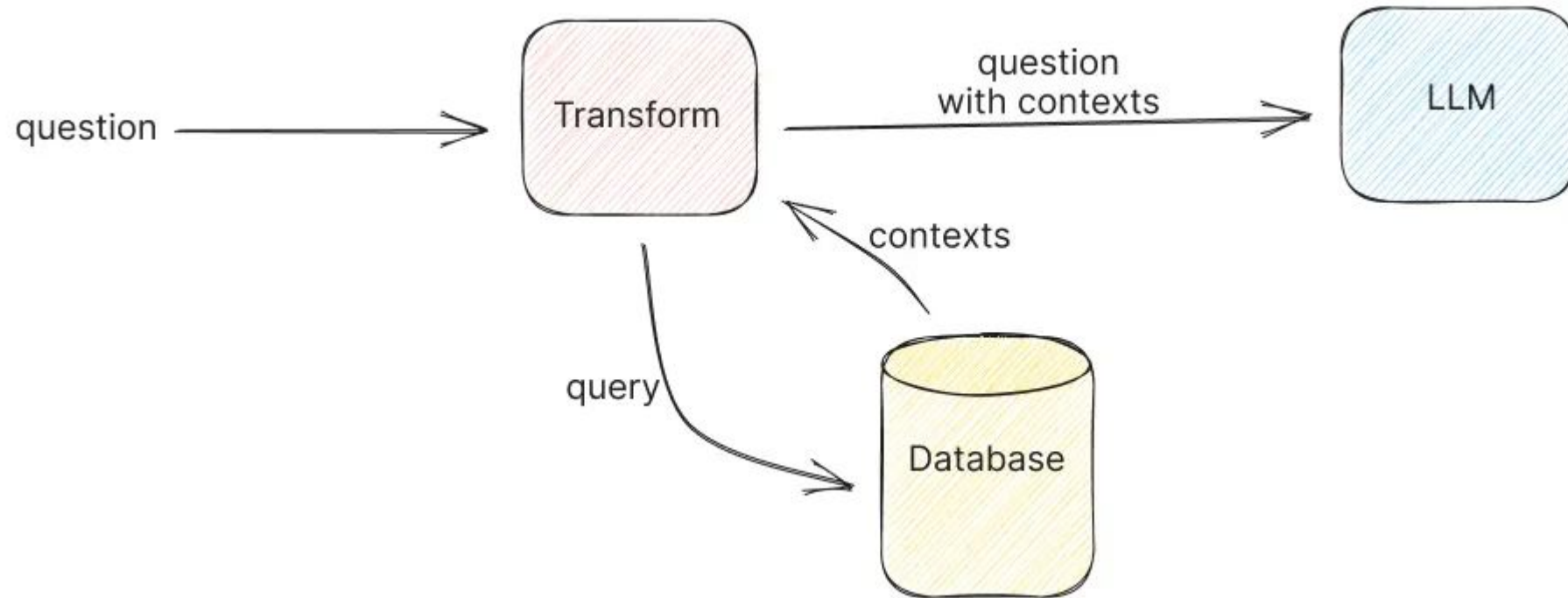
# Hyperparameters
batch_size = 64
learning_rate = 0.001
num_epochs = 10
```

 Regenerate

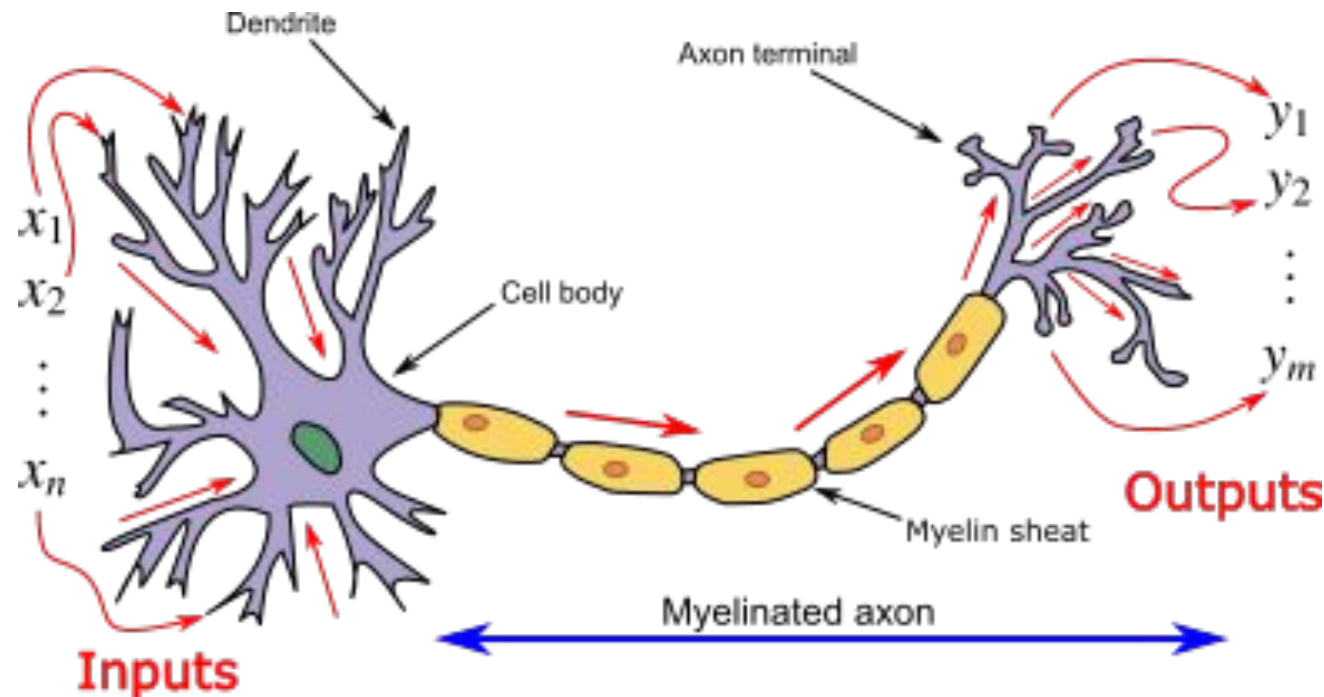
Send a message 

Free Research Preview. ChatGPT may produce inaccurate information about people, places, or facts. ChatGPT July 20 Version

RAG:



Why are they called Neural Networks?:



The brain is composed by a lot of small (and “simple”) neurons!

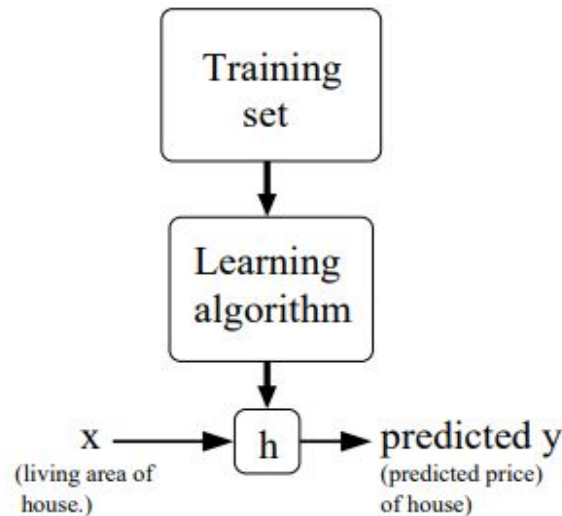
Neural Networks are composed by a lot of small arithmetic operations!

Problem Statement:

- In LinReg and LogReg we considered our models (or hypothesis $h_{\theta}(x)$) to be linear in the parameters θ .
- Neural Networks are more general models that non-linear in both the parameters θ and the inputs x .

Notation:

- $x_j^{(i)}$ feature j of sample i
- $\mathbf{x}^{(i)}$ feature vector i (size $n \times 1$)
- $y_j^{(i)}$ target value j of example i
- $\mathbf{y}^{(i)}$ target vector i
- $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ training pair i
- $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^m$ training set
- $\mathcal{X}$ input space
- $\mathcal{Y}$ output space
- $h : \mathcal{X} \rightarrow \mathcal{Y}$ hypothesis

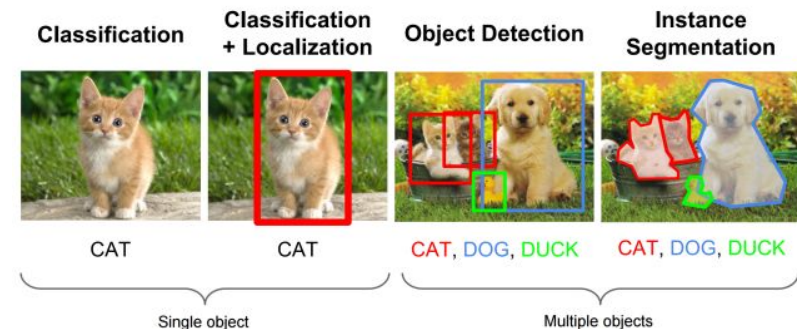


Objective:

- given a training set
- learn a function $h_{\theta}(x)$ so that $h(x)$ is a “good” predictor of y .

“good” = Minimize a Loss function!

- continuous - regression
- discrete – classification
- A combination of both



Logistic Regression is a Neural Net:

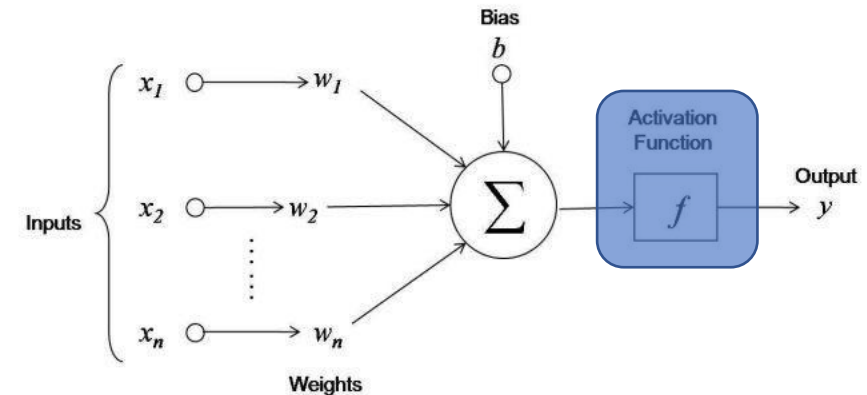
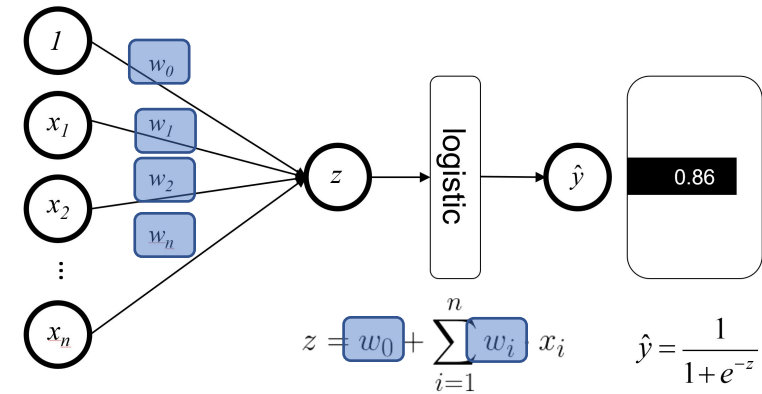
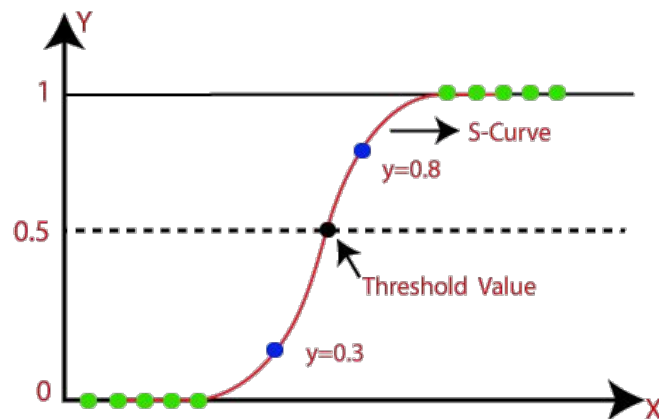
Approximate y as a nonlinear function of x

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}},$$

where

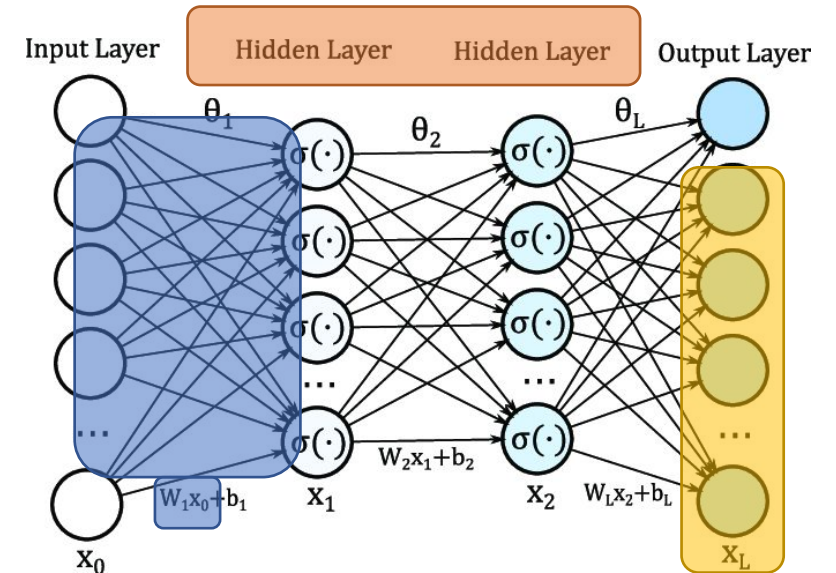
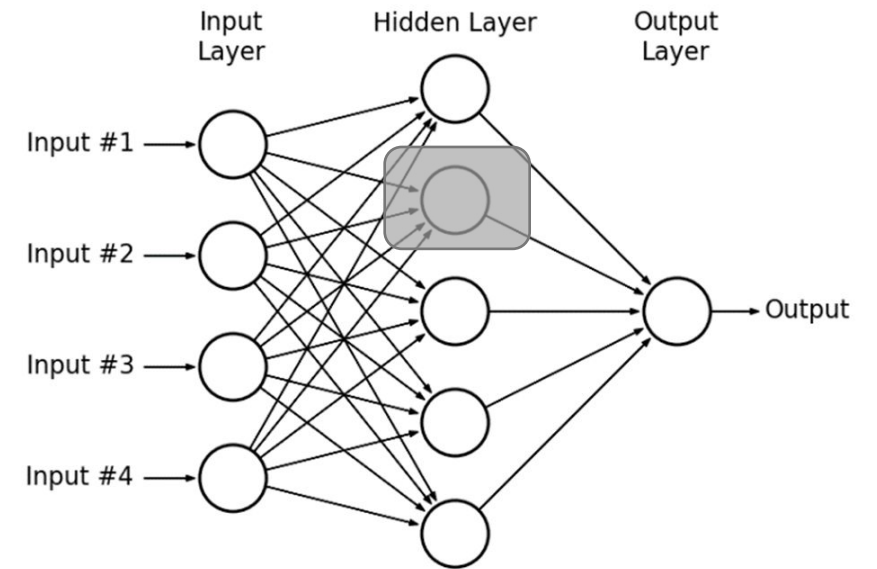
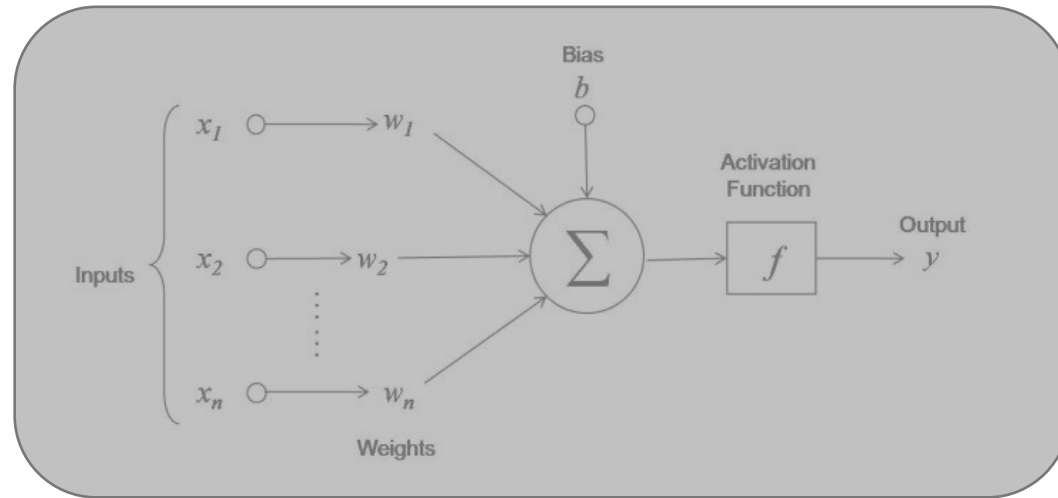
$$g(z) = \frac{1}{1 + e^{-z}}.$$

$g(z)$ is the sigmoid function.



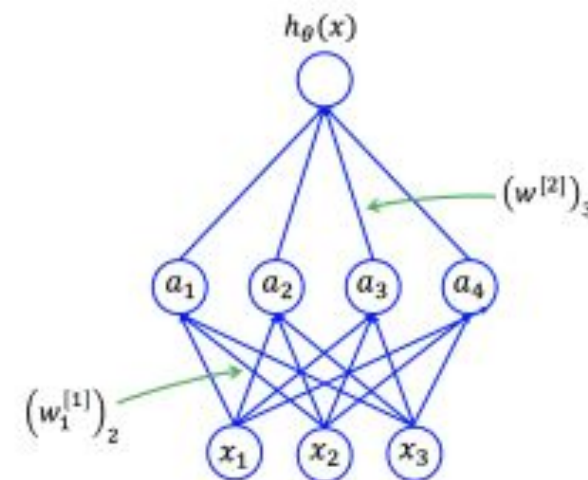
Single layer perceptron!

Multi-layer perceptron:



Multi-layer perceptron:

- $a_1 = \sigma(w_1^{[1]}x + b_1^{[1]})$, where $w_1^{[1]} \in R^{1 \times 3}$ and $b_1^{[1]} \in R$
- $a_2 = \sigma(w_2^{[1]}x + b_2^{[1]})$
- $a_3 = \sigma(w_3^{[1]}x + b_3^{[1]})$
- $a_4 = \sigma(w_4^{[1]}x + b_4^{[1]})$
- $h_\theta(x) = \sigma(w^{[2]}a + b^{[2]})$, where $w^{[2]} \in R^{1 \times 4}$, $b^{[2]} \in R$ and $a = [a_1, \dots, a_4]$
- $z_i = w_i^{[1]}x + b_i^{[1]}$

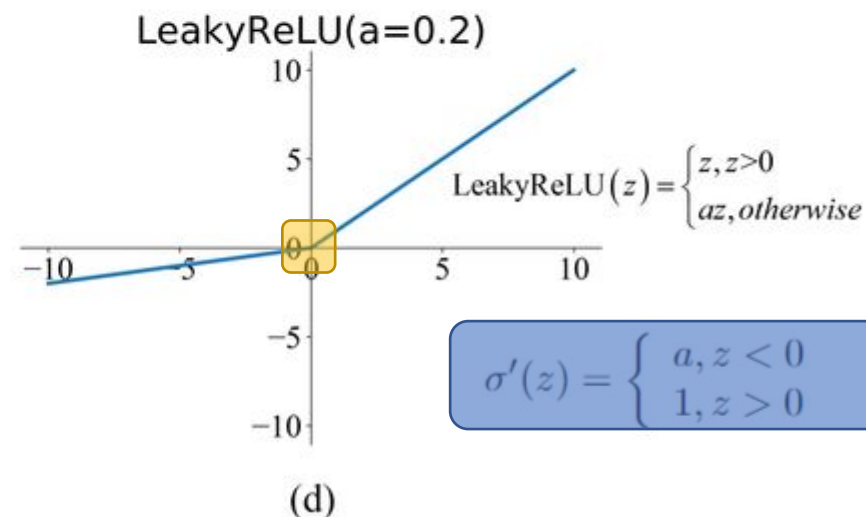
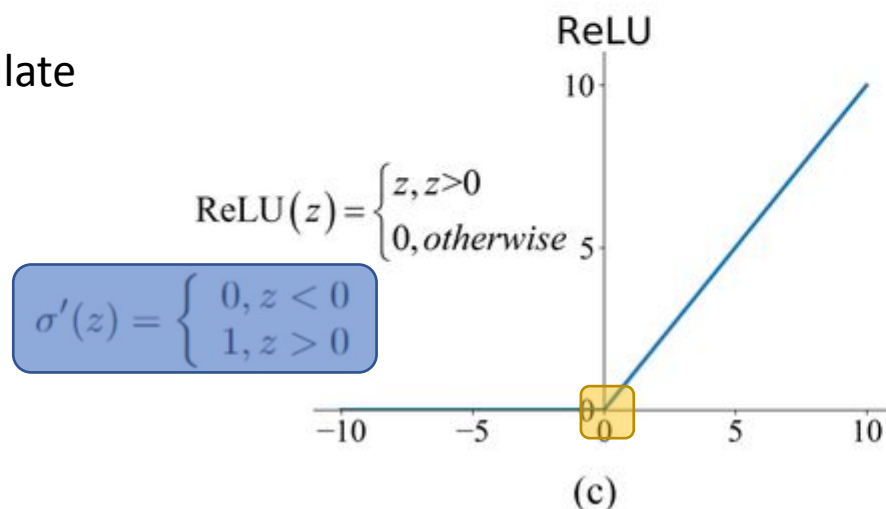
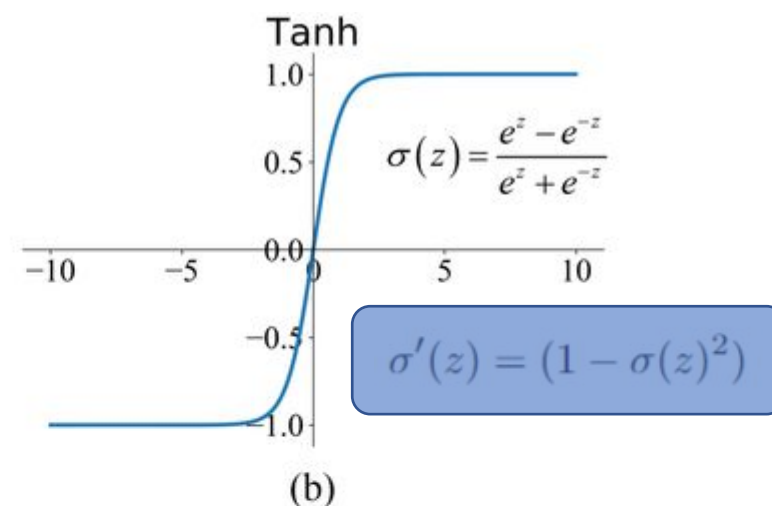
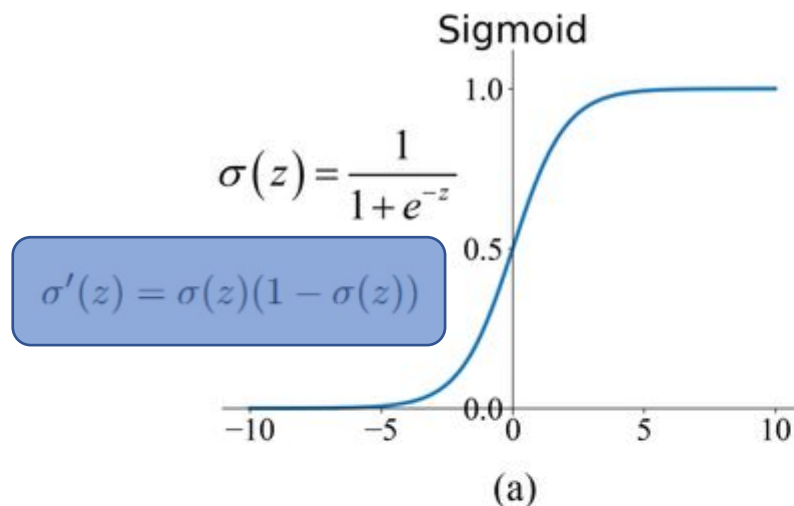


Matrix notation:

$$\underbrace{\begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \end{pmatrix}}_{z} \quad 4 \times 1 = \underbrace{\begin{pmatrix} -w_1^{[1]T} & - \\ -w_2^{[1]T} & - \\ -w_3^{[1]T} & - \\ -w_4^{[1]T} & - \end{pmatrix}}_{W} \quad 4 \times 3 \times \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_{x} \quad 3 \times 1 + \underbrace{\begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}}_{b^{[1]}} \quad 4 \times 1$$

Activation Layers:

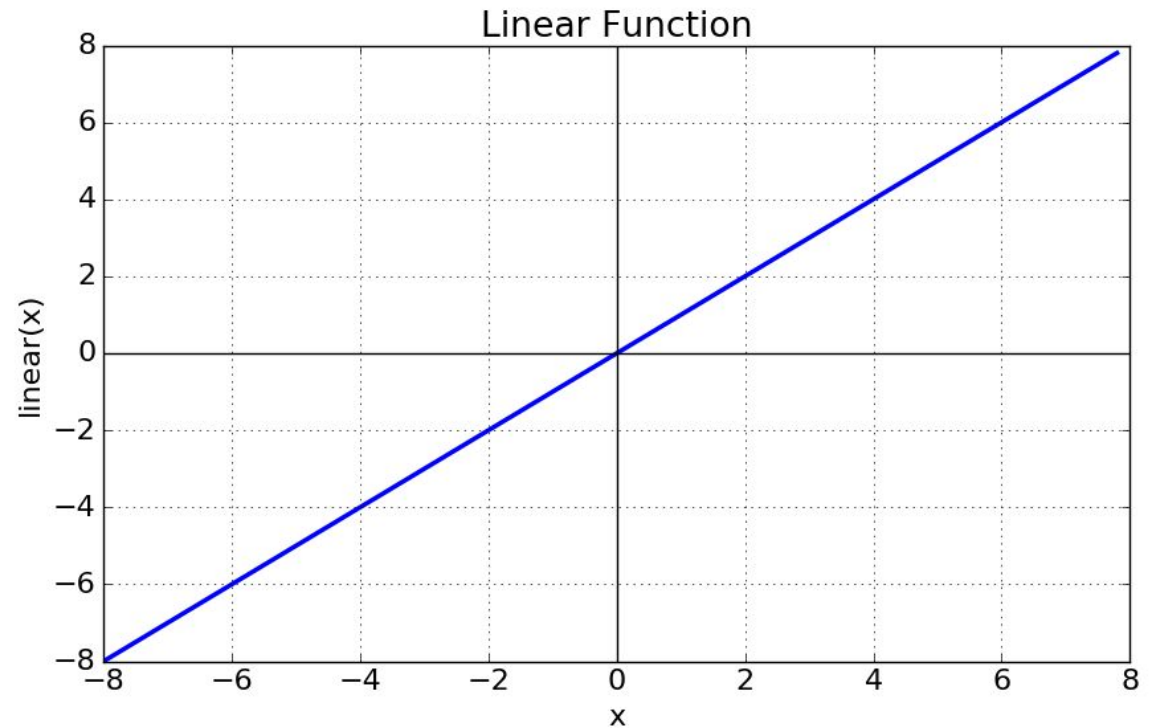
- Apply non-linearities.
- Fast to compute
- Differentiable* (to apply Gradient Descent).
- All have fast-to-calculate derivatives.



Linear activation layer:

- It is only used for the last layer and a regression problem.
- Why is it not used in hidden layers?

$$\begin{aligned}h_{\theta}(x) &= W^{[2]}a^{[1]} \\&= W^{[2]}\sigma(z^{[1]}) \\&= W^{[2]}z^{[1]} \\&= W^{[2]}W^{[1]}x \\&= \tilde{W}x\end{aligned}$$

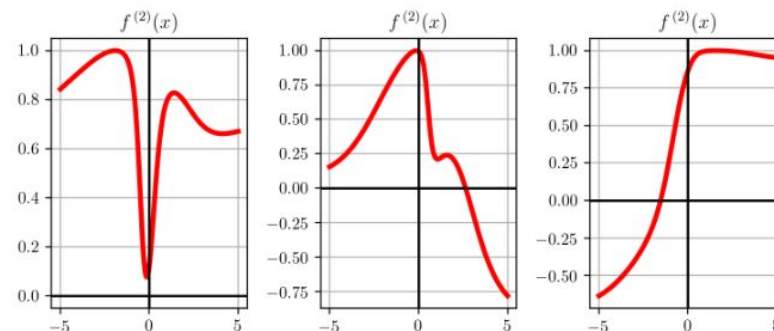
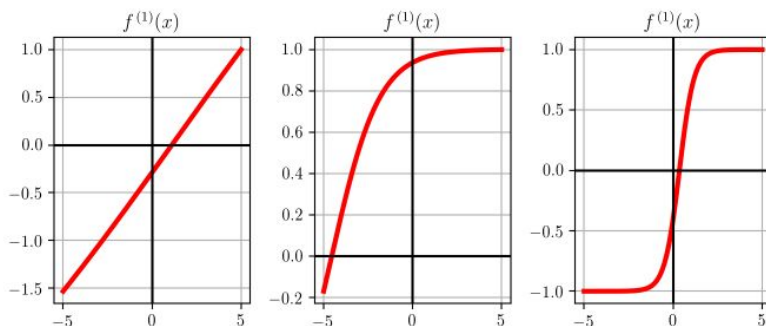


Depth and non-linearity:

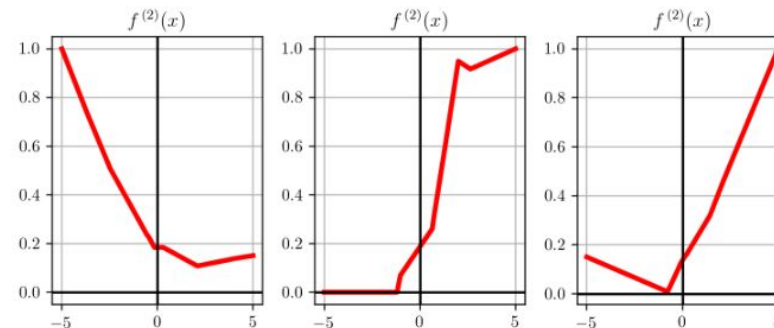
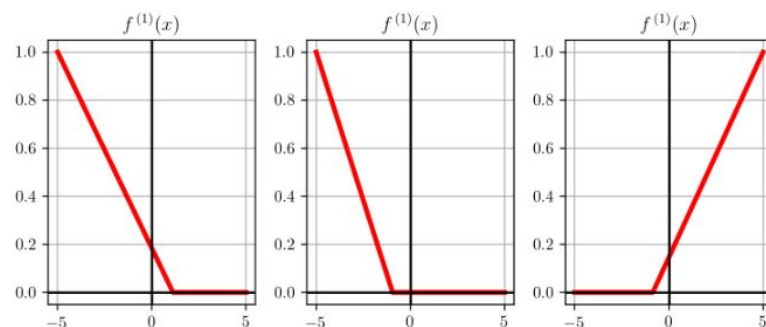
1 layer – 3 random initializations.

2 layers – 3 random initializations.

Tanh

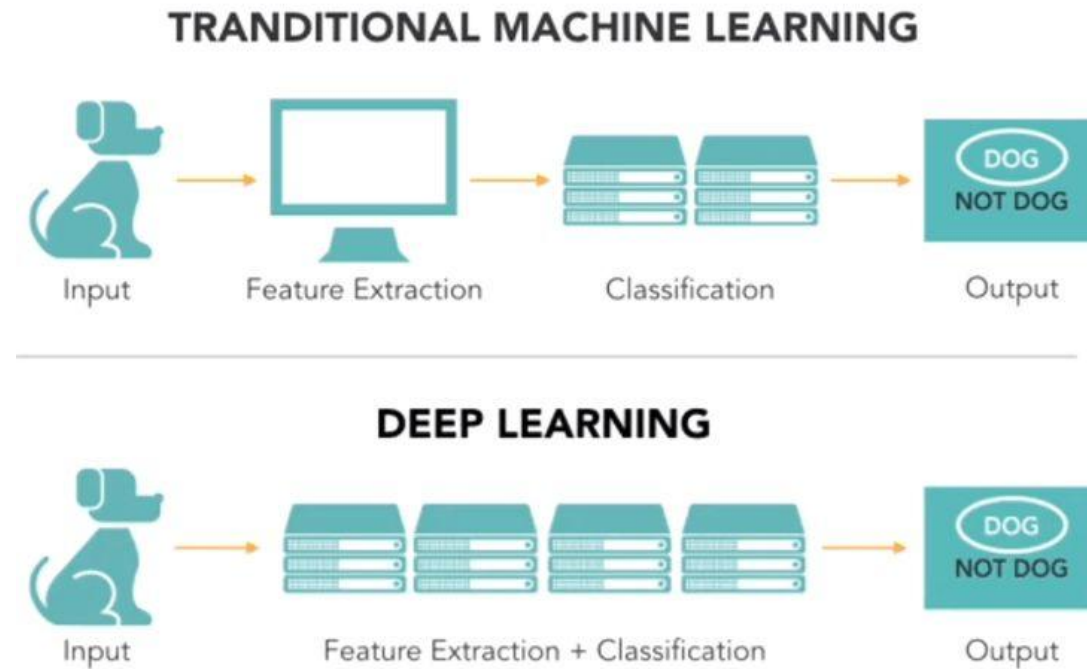


ReLu



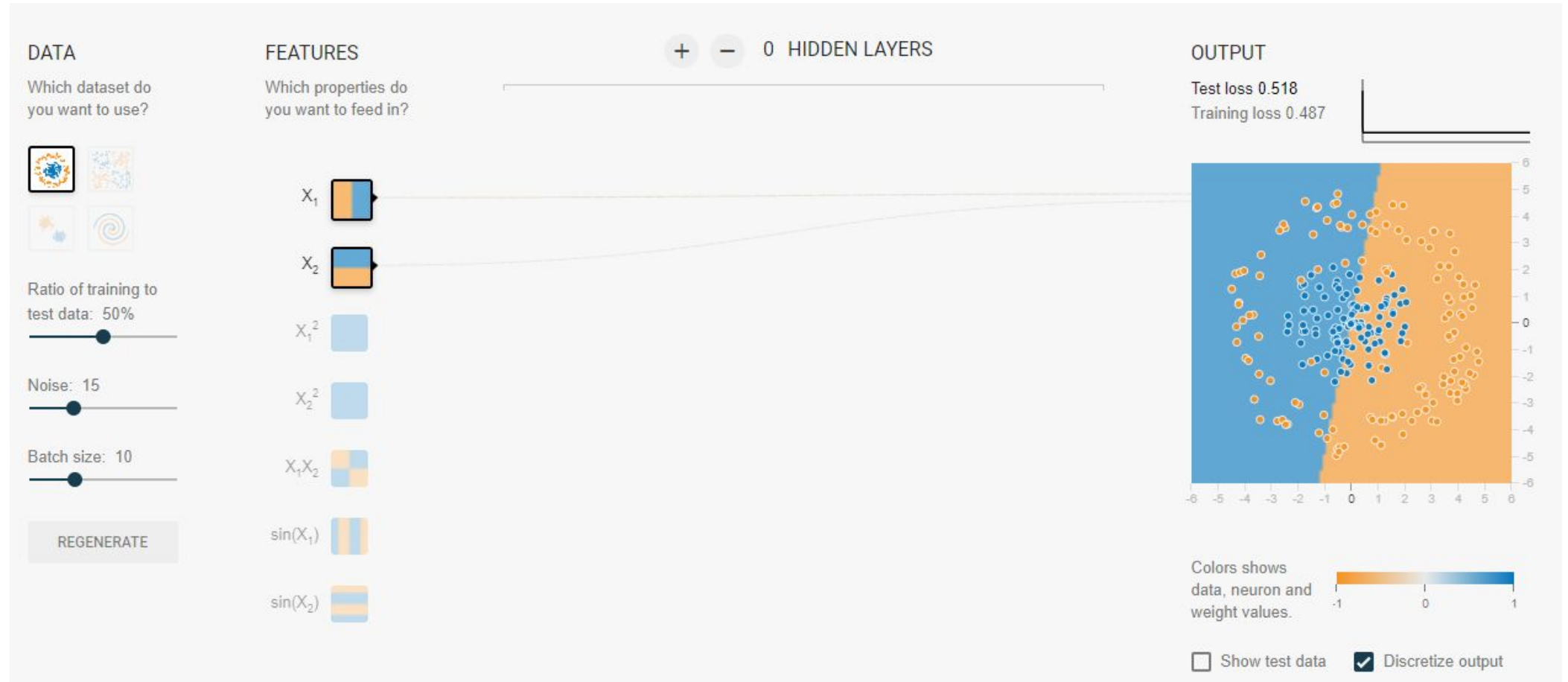
We can learn non-linear functions without performing feature engineering!

Different approach:

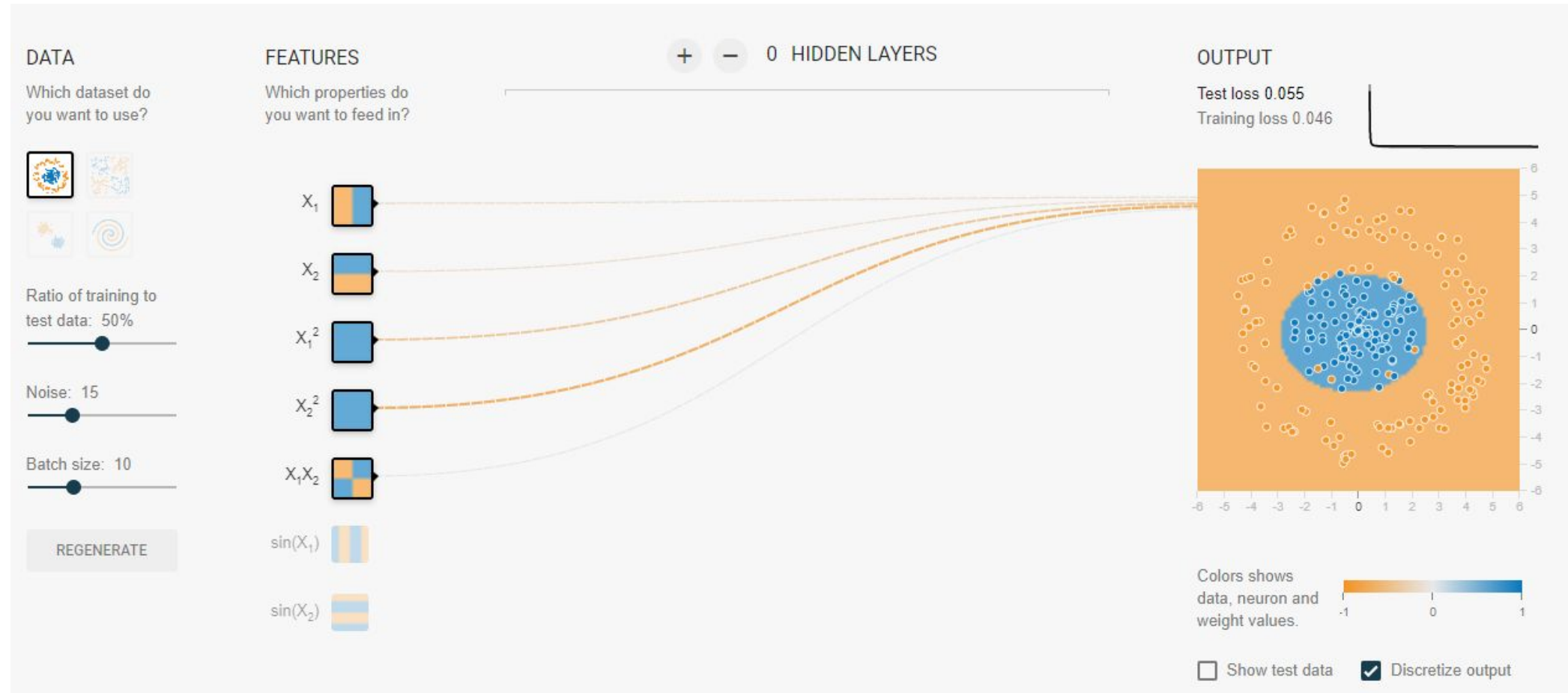


<https://playground.tensorflow.org/>

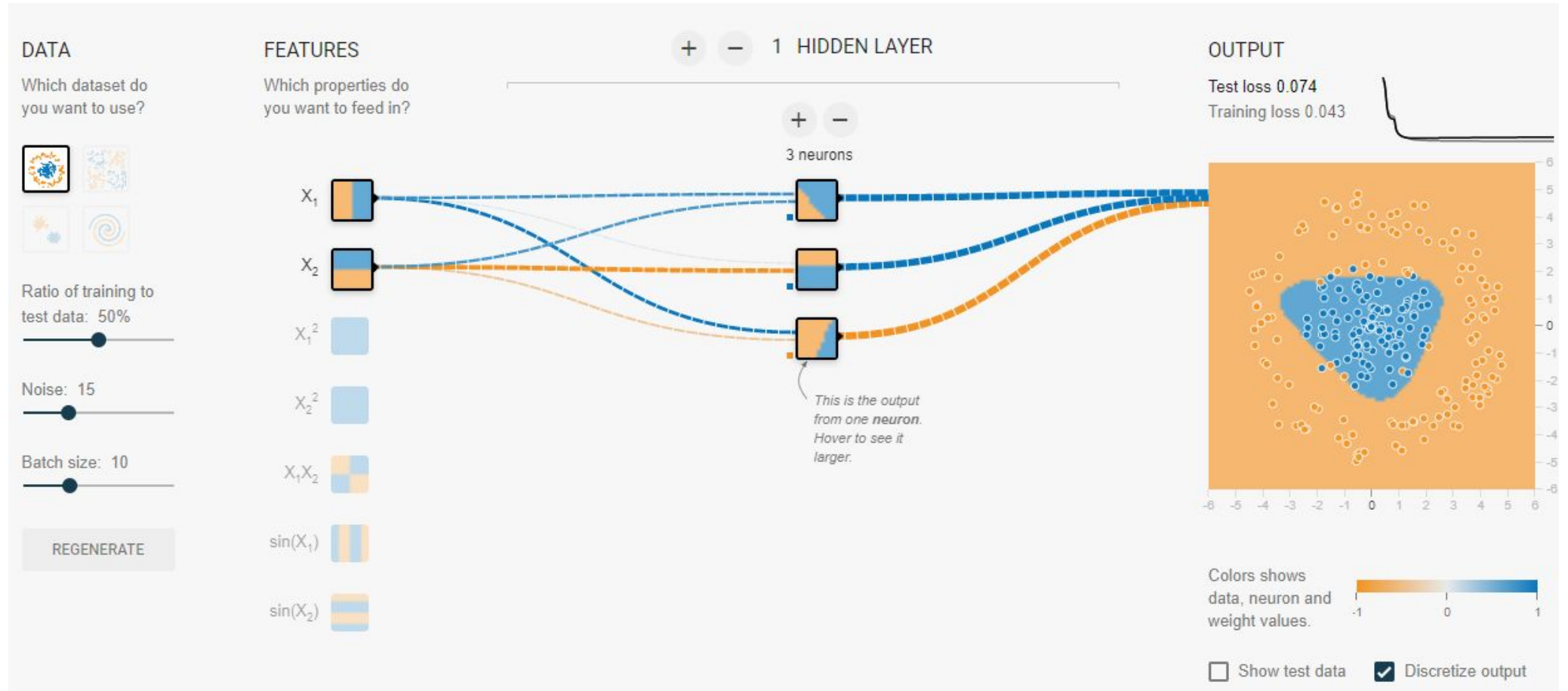
Different approach – example of ML:



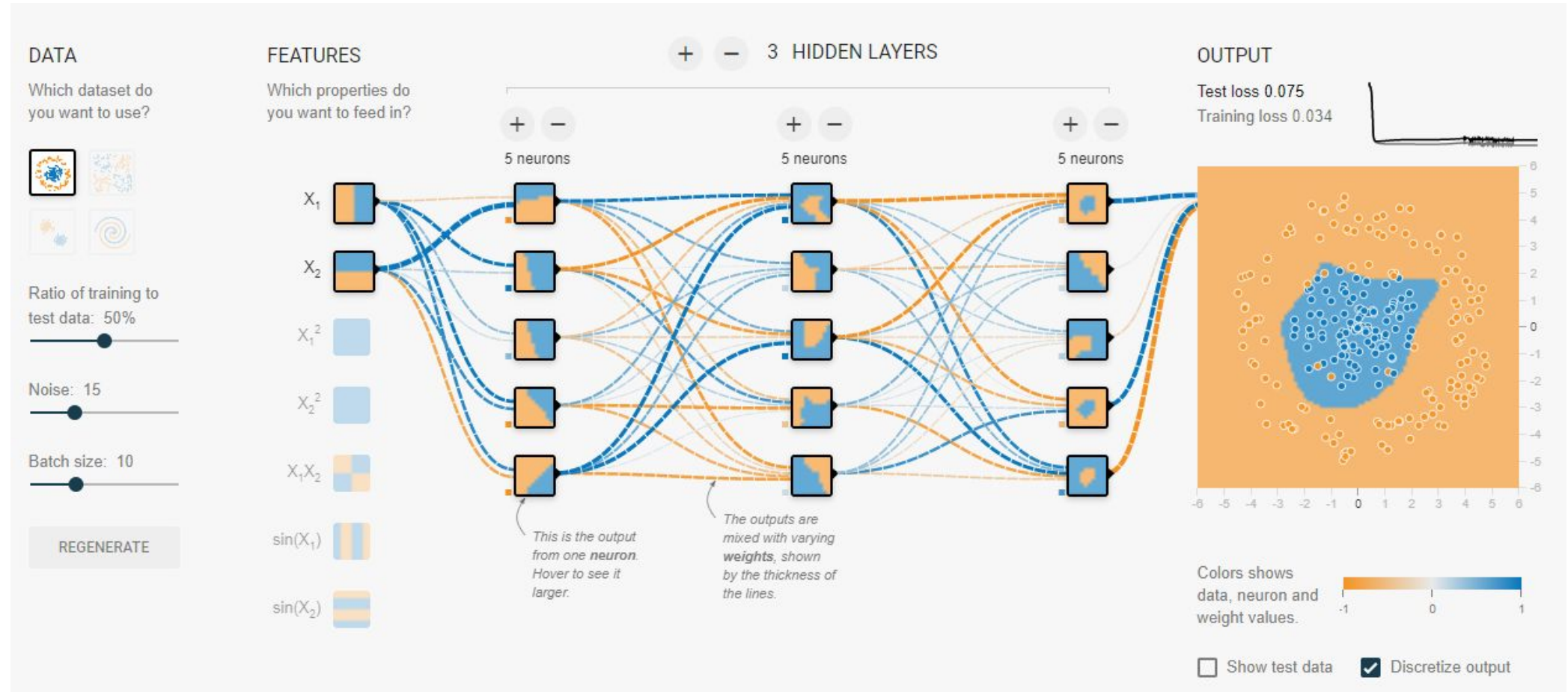
Different approach – example of ML :



Different approach – Neural Networks:



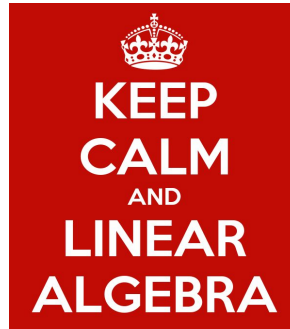
Different approach - Deep Learning:



Deep Learning is not magic:

The applications that you saw in the first part were not magic. They are simply a combination of the following concepts:

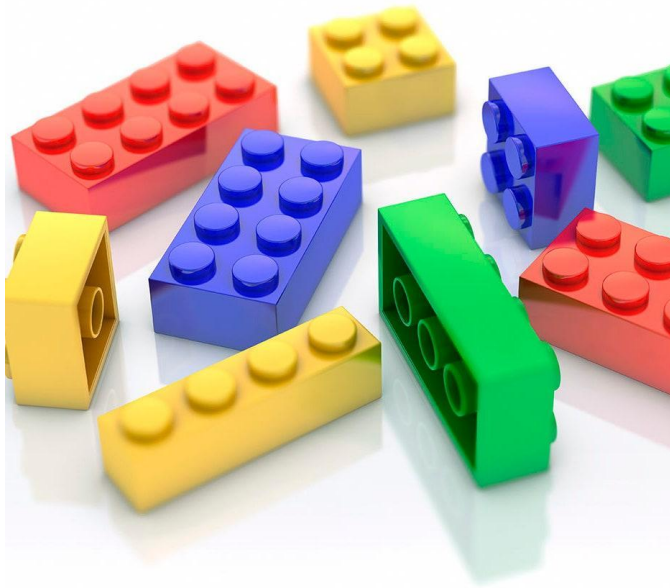
- Linear combinations
- Simple nonlinear functions
- A stack of the previous 2.



The most complex computer vision neural nets are just a combination of many “small and easy blocks” that you can understand and even program! The main ideas behind finding the best weights are the same as the tools that you already possess from Machine Learning!



Deep Learning is not magic:



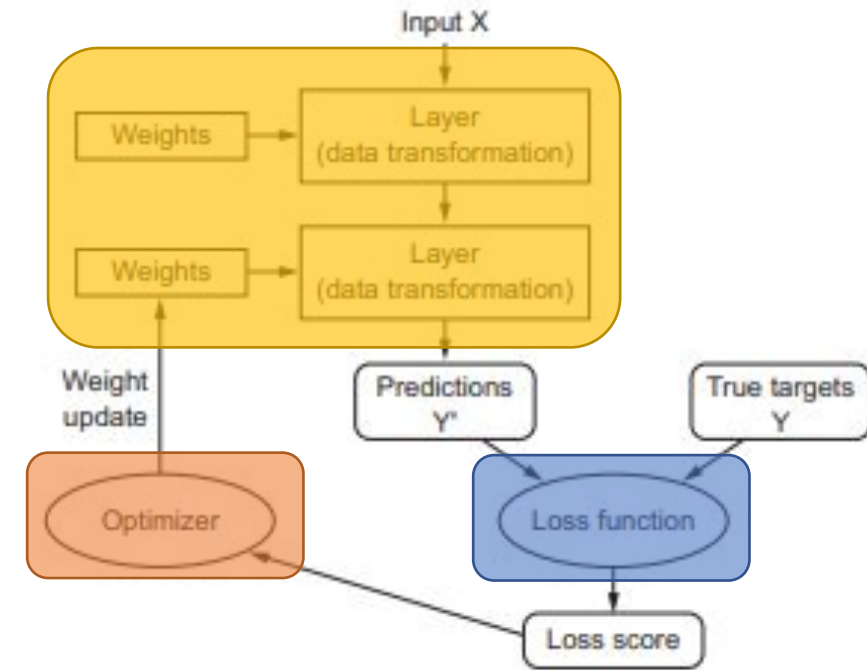
We will explain and understand small blocks



Build complex nets abstracting each block.

Learning workflow:

- Start with a classified dataset.
 - Design an architecture.
 - Decide a loss function.
 - Decide an optimizer.
 - Apply an iterative algorithm (Gradient Descent).
-
- The engine of neural networks: gradient-based optimization!
 - Note: we do not mention the word “convex”.



- The first step to perform gradient-based optimization is to find the gradients!

Loss functions:

Problem type	Last-layer activation	Loss function
Binary classification	sigmoid	binary_crossentropy
Multiclass, single-label classification	softmax	categorical_crossentropy
Multiclass, multilabel classification	sigmoid	binary_crossentropy
Regression to arbitrary values	None	mse
Regression to values between 0 and 1	sigmoid	mse or binary_crossentropy

Cross Entropy Loss

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

[nn.CrossEntropyLoss](#)

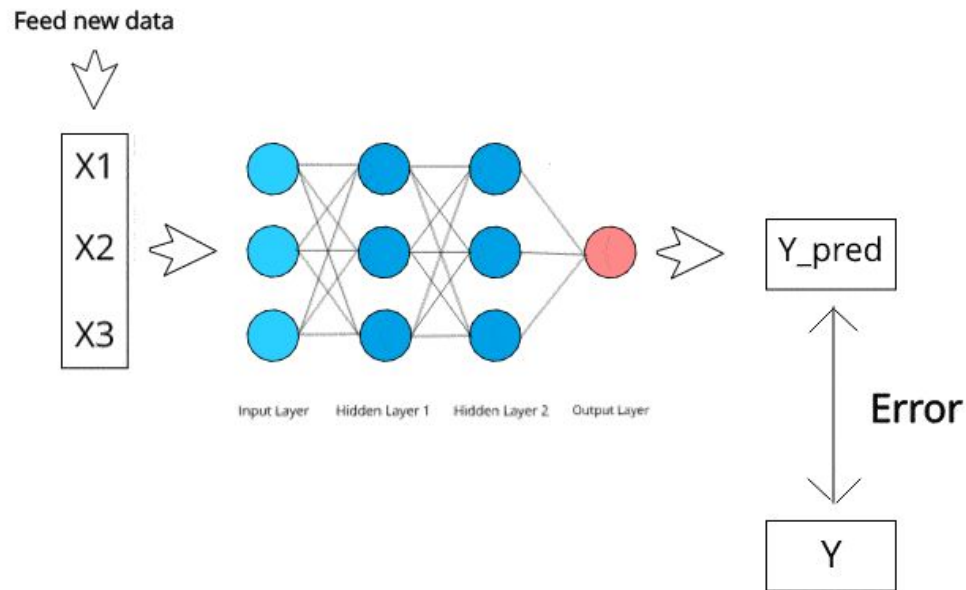
[nn.NLLLoss](#)

[nn.BCELoss](#)

[nn.BCEWithLogitsLoss](#)

Gradient Calculation:

- We could generate a very complex function and find all derivatives.
 - This is very complex and inefficient.
- We will use a trick named back-propagation.
 - Modern DL libraries (TensorFlow/Pytorch) construct computational graphs that do exactly this.



How do we calculate the derivatives? Chain rule!

- $g_i = g_i(\theta_1, \dots, \theta_p)$
- $J = J(g_1, \dots, g_k)$

$$\frac{\partial J}{\partial \theta_i} = \sum_{j=1}^k \frac{\partial J}{\partial g_j} \frac{\partial g_j}{\partial \theta_i}$$

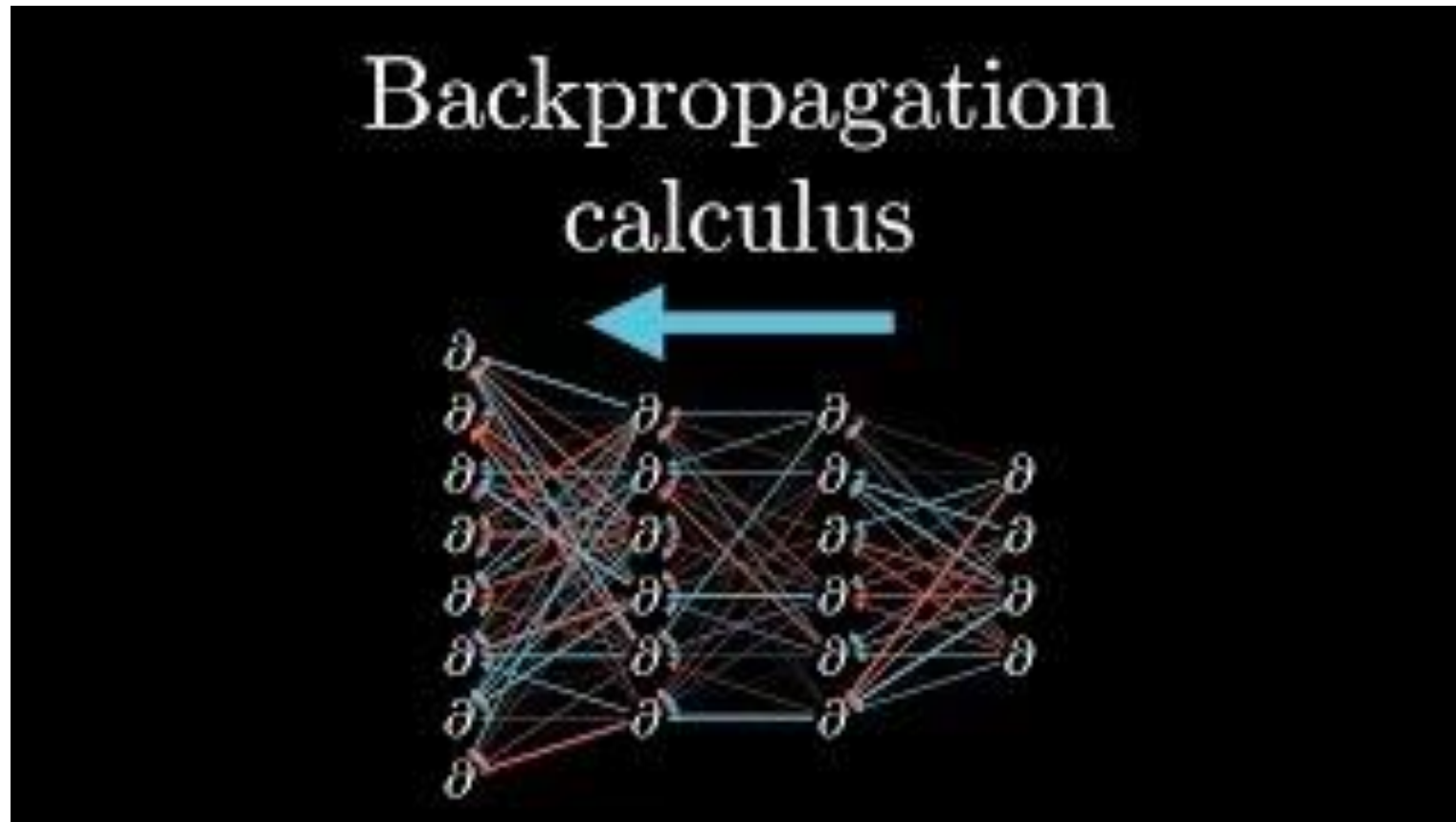
The derivative of the composition...

$$\frac{df(g(x))}{dx} = \frac{df(g(x))}{dg(x)} \cdot \frac{dg(x)}{dx}$$

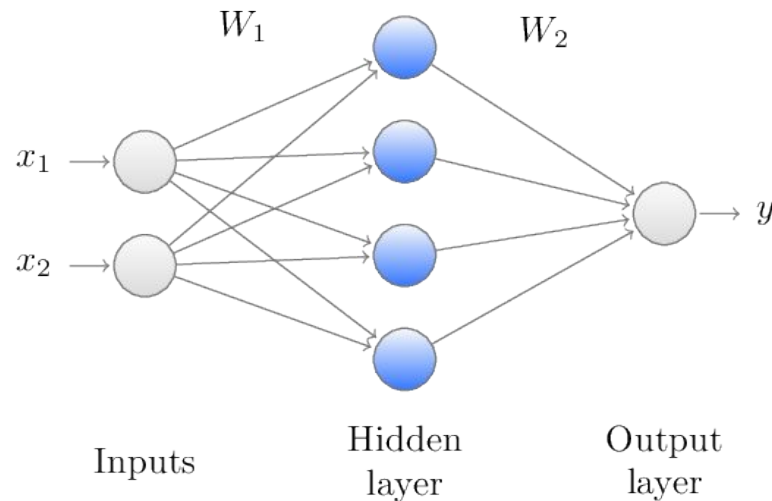
...is the derivative of the first function with respect to the second...

...by the derivative of the second function

Gradient descent - neural networks:



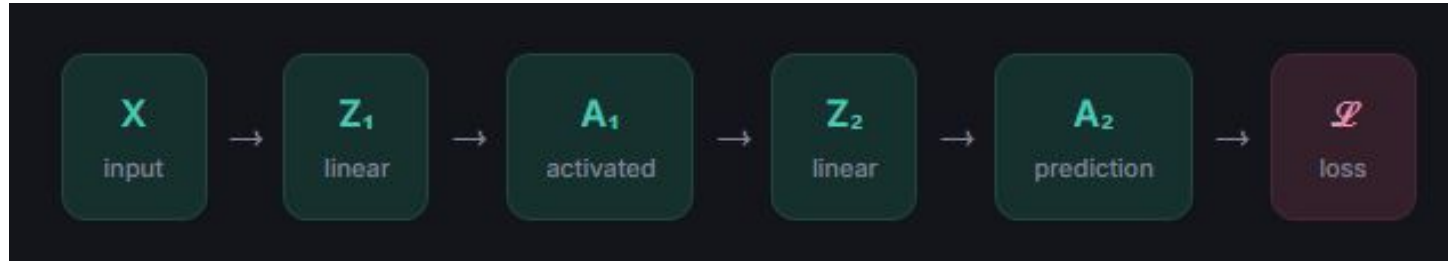
Forward and backward propagation:



Key Notations:

- X : Input matrix, shape $(m \times n_{\text{input}})$, where m is the number of samples.
- W_1 : Weight matrix between input and hidden layer, shape $(n_{\text{input}} \times n_{\text{hidden}})$.
- b_1 : Bias vector for the hidden layer, shape $(1 \times n_{\text{hidden}})$.
- Z_1 : Pre-activation output of the hidden layer, shape $(m \times n_{\text{hidden}})$.
- A_1 : Post-activation output of the hidden layer, shape $(m \times n_{\text{hidden}})$, where $A_1 = \phi(Z_1)$.
- W_2 : Weight matrix between hidden and output layer, shape $(n_{\text{hidden}} \times n_{\text{output}})$.
- b_2 : Bias vector for the output layer, shape $(1 \times n_{\text{output}})$.
- Z_2 : Pre-activation output of the output layer, shape $(m \times n_{\text{output}})$.
- A_2 : Post-activation output (predictions), shape $(m \times n_{\text{output}})$, where $A_2 = \phi(Z_2)$.
- $\mathcal{L}$: The loss function (can be any differentiable function).

Forward and backward propagation:



1 Hidden layer — linear step

$$Z_1 = XW_1 + b_1$$

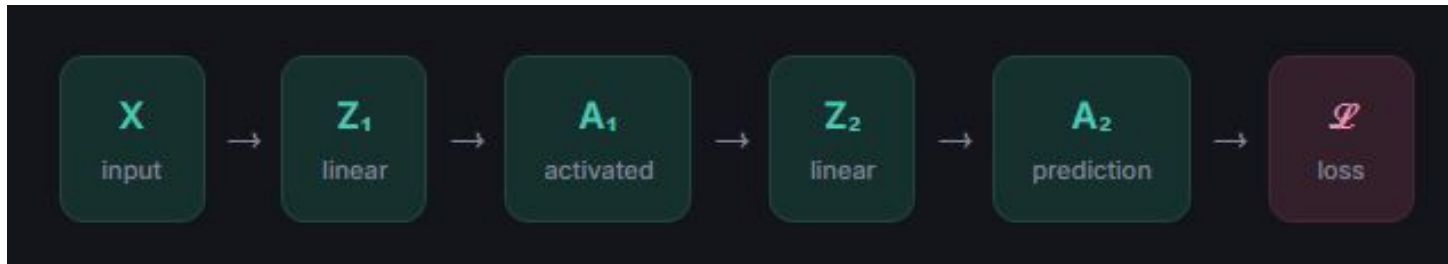
A matrix multiplication of the input X with weights W_1 , plus the bias b_1 .

2 Hidden layer — activation step

$$A_1 = \phi(Z_1)$$

ϕ can be any differentiable function — sigmoid, ReLU, tanh, etc.

Forward and backward propagation:



3 Output layer — linear step

$$Z_2 = A_1 W_2 + b_2$$

Same move as step 1, just one layer over: matrix multiply, then add the bias.

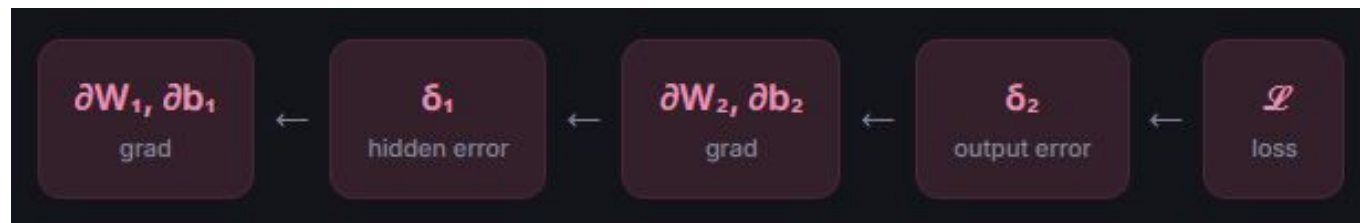
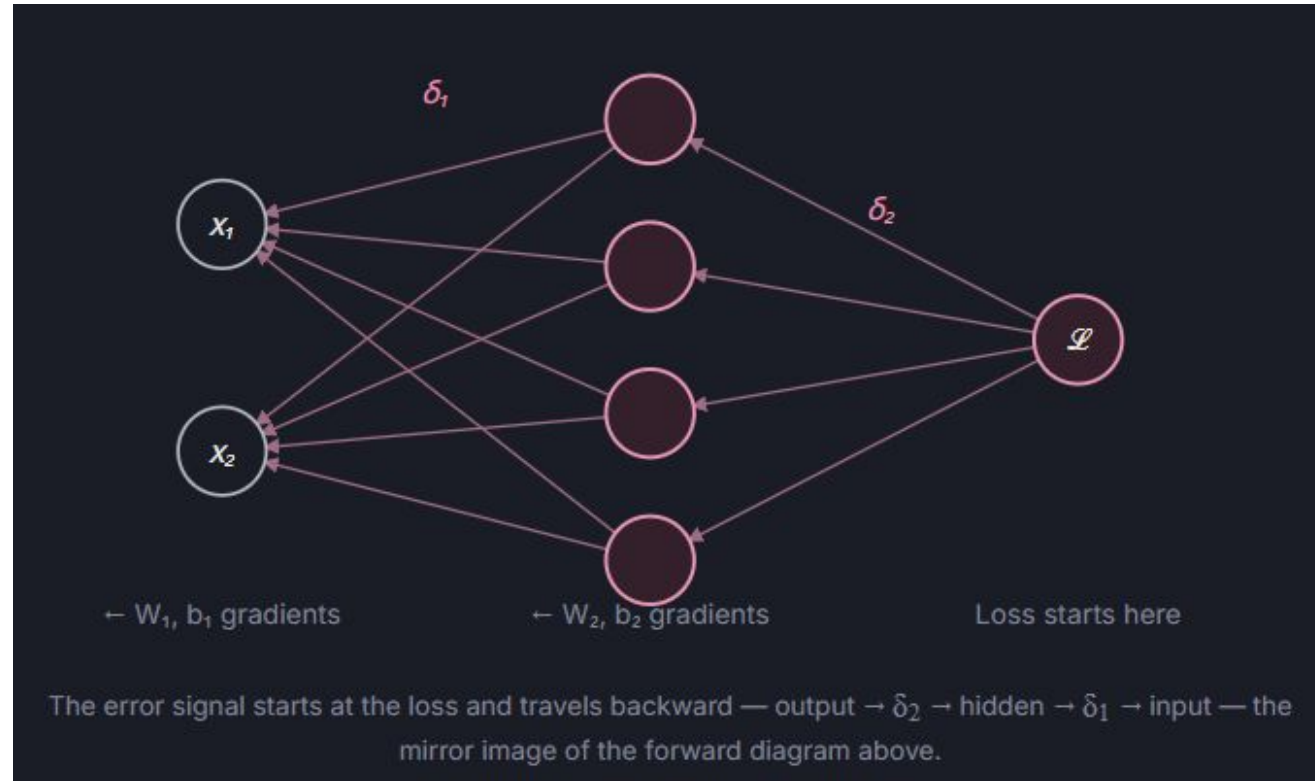
4 Output layer — activation step

$$A_2 = \phi(Z_2)$$

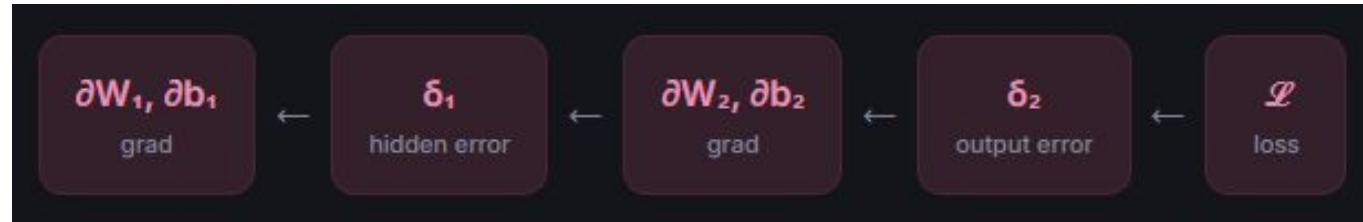
A_2 is the network's final prediction.

Loss
 $L(A_2, y)$

Forward and backward propagation:



Forward and backward propagation:



Error term at the output, δ_2

By the chain rule, the derivative of the loss with respect to Z_2 decomposes as:

$$\frac{\partial L}{\partial Z_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

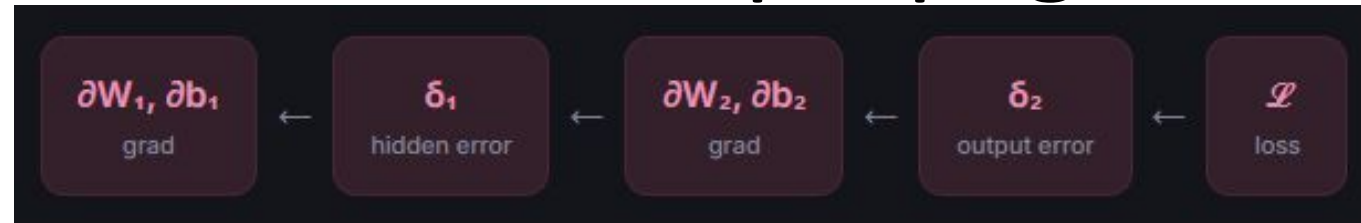
$\partial L / \partial A_2$ depends on which loss function you picked. $\partial A_2 / \partial Z_2 = \phi'(Z_2)$, the activation's own derivative.

We give this combination a name, δ_2 , since we'll reuse it constantly:

$$\delta_2 = \frac{\partial L}{\partial A_2} \cdot \phi'(Z_2)$$

Read δ_2 as "how sensitive the loss is to the output layer's raw input" — its error signal.

Forward and backward propagation:



Gradient with respect to W_2 and b_2

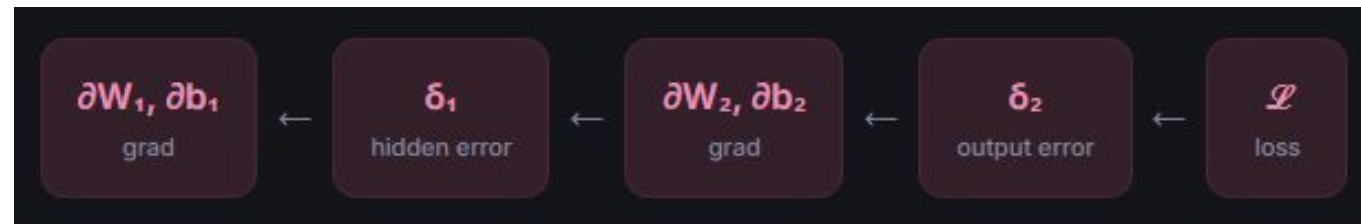
$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}$$

Since $Z_2 = A_1 W_2 + b_2$, we have $\partial Z_2 / \partial W_2 = A_1$. So:

$$\frac{\partial L}{\partial W_2} = A_1^T \delta_2 \qquad \frac{\partial L}{\partial b_2} = \sum_{i=1}^m \delta_2^{(i)}$$

The bias gradient sums δ_2 over all m samples, since every sample shares the same bias.

Forward and backward propagation:



Error term at the hidden layer, δ_1

Same chain rule, one layer further back:

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1}$$

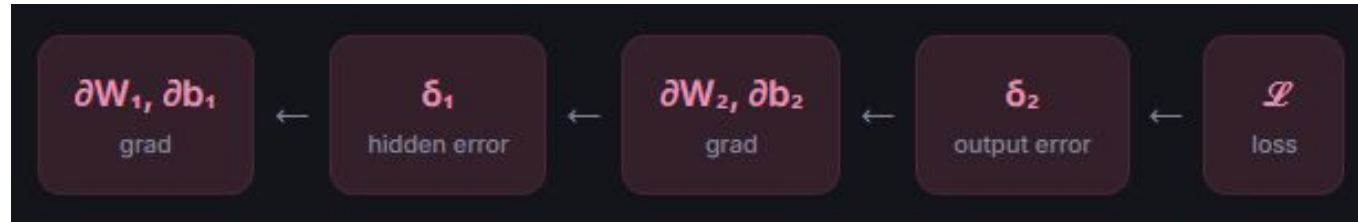
We already have $\partial L / \partial A_1$ from the output layer's work: it's δ_2 projected back through W_2 .

$$\frac{\partial L}{\partial A_1} = \delta_2 W_2^T$$

This is why it's called *back*-propagation: δ_2 flows backward through W_2^T to reach the hidden layer.

$$\delta_1 = (\delta_2 W_2^T) \cdot \phi'(Z_1)$$

Forward and backward propagation:



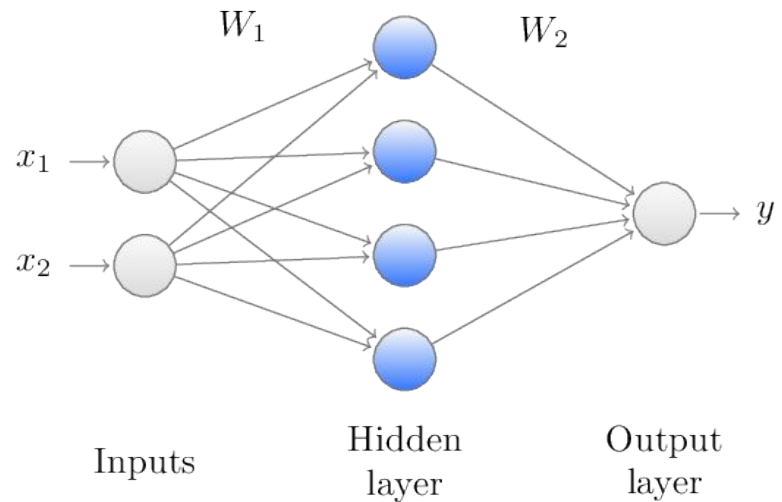
Gradient with respect to W_1 and b_1

Exactly the same shape as step 2, one layer earlier:

$$\frac{\partial L}{\partial W_1} = X^T \delta_1 \qquad \frac{\partial L}{\partial b_1} = \sum_{i=1}^m \delta_1^{(i)}$$

Since $Z_1 = XW_1 + b_1$, $\partial Z_1 / \partial W_1 = X$ plays the role A_1 played for W_2 .

Forward and backward propagation:



4. **Update Parameters:** Using gradient descent, update the weights and biases:

$$W_1 = W_1 - \eta \frac{\partial \mathcal{L}}{\partial W_1}, \quad W_2 = W_2 - \eta \frac{\partial \mathcal{L}}{\partial W_2}$$

$$b_1 = b_1 - \eta \frac{\partial \mathcal{L}}{\partial b_1}, \quad b_2 = b_2 - \eta \frac{\partial \mathcal{L}}{\partial b_2}$$

Forward and backward propagation:

Forward →

Start

$$A_0 = X$$

For each layer $k = 1, \dots, K+1$, linear

$$Z_k = A_{k-1}W_k + b_k$$

...then activation

$$A_k = \phi_k(Z_k)$$

Loss, at the last layer

$$L(A_{K+1}, y)$$

Backward ←

Start at the output, $k = K+1$

$$\delta_{K+1} = \frac{\partial L}{\partial A_{K+1}} \cdot \phi'_{K+1}(Z_{K+1})$$

For each layer $k = K, \dots, 1$, error term

$$\delta_k = (\delta_{k+1} W_{k+1}^T) \cdot \phi'_k(Z_k)$$

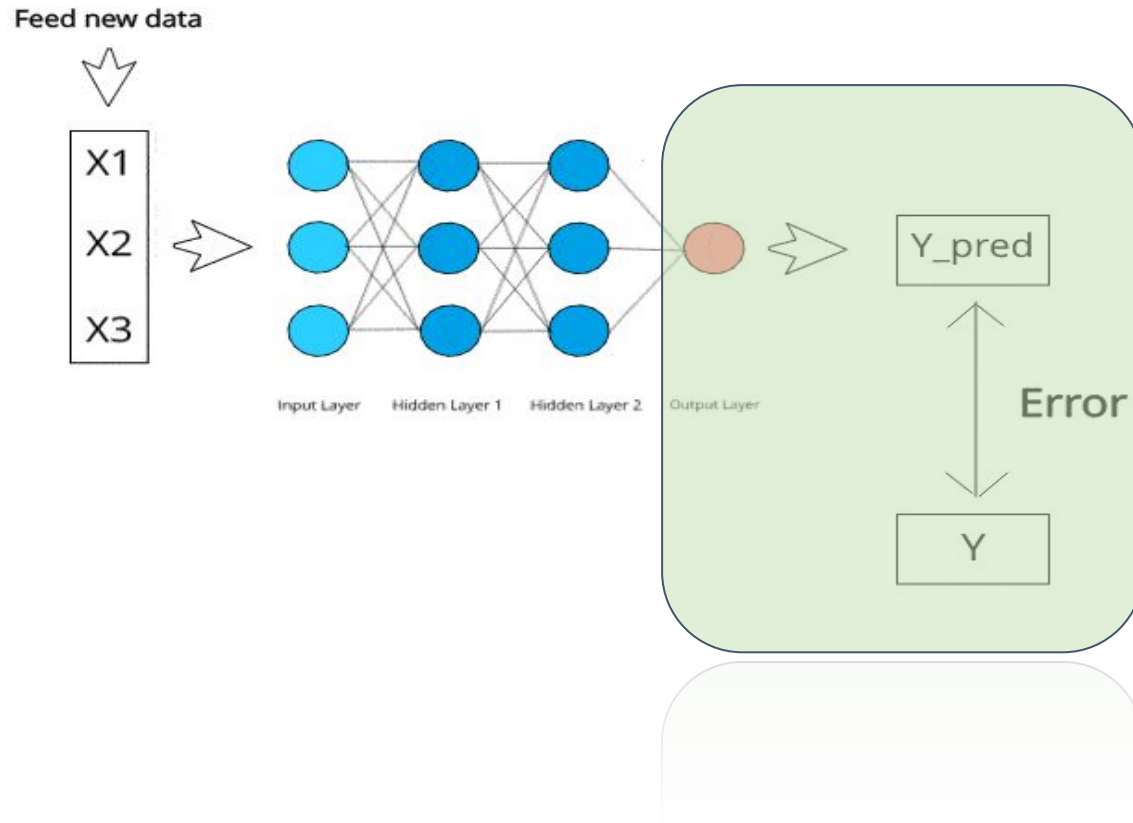
...weight gradient

$$\frac{\partial L}{\partial W_k} = A_{k-1}^T \delta_k$$

...bias gradient

$$\frac{\partial L}{\partial b_k} = \sum_i \delta_k^{(i)}$$

Forward and backward propagation:



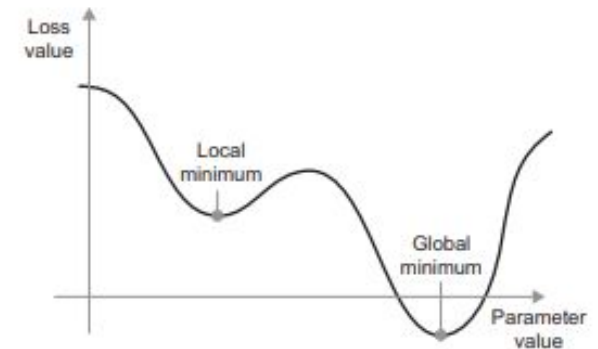
From the ML classes:

- You know how to start for Linear Regression
- You know how to start for Logistic Regression
- Let's see how to start for multiclass classification (later)

Gradient Descent for NN:

In order to optimize the weights of a neural network, we will apply gradient descent, but!:

- Multi-layer NN are not convex.
 - Small NN -> seed it at random points
 - Large NN have been more freedom it has and hence will likely escape from potential local minima traps. (Empirical observations)
 - <https://www.quora.com/Are-neural-nets-always-convex-with-respect-to-the-weights-And-if-not-how-does-gradient-descent-work-so-well>
 - Optimizers -> Different from vanilla Gradient Descent.
- Some activation function are not differentiable.
 - Only at some points (i.e., 0 for ReLU).
 - We still have sub-derivatives.
 - <https://www.quora.com/Why-does-ReLU-work-with-backprops-if-its-non-differentiable>

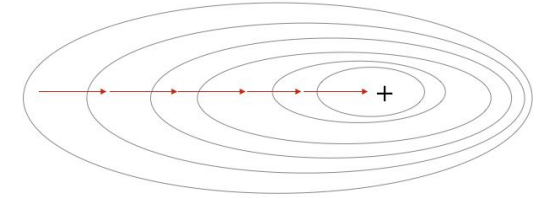


Gradient Descent variations:

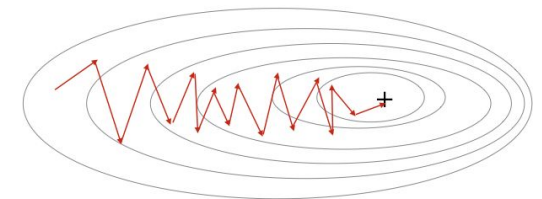
- Batch Gradient Descent:
 - Feed the entire dataset, average of cost, calculate derivatives and update.
 - Used in LinReg and LogReg (convex problems).
- Stochastic Gradient Descent:
 - Feed the 1 sample, calculate the cost and derivatives, and update.
 - Many more updates.
 - Not always towards the minima (could be good due to non-convexity).
- Mini-Batch Gradient Descent:
 - Feed the a mini-batch of samples dataset, average of cost, calculate derivatives and update.
 - Same benefits from Stochastic but we can vectorize the calculation for the mini-batches. This improves **speed**.
 - Typically, 10, 16, 32, 64 or 128 (computers are prepared to parallelize this).

MB-SGD algorithm is that updates of weight are very noisy -> Optimizers!

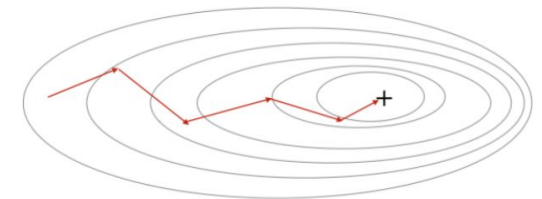
Gradient Descent



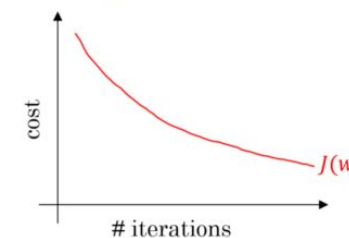
Stochastic Gradient Descent



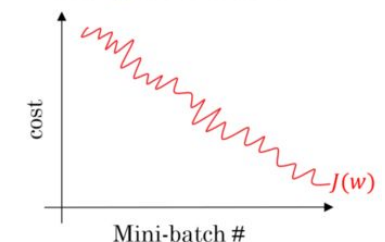
Mini-Batch Gradient Descent



Batch gradient descent



Mini-batch gradient descent



Demo MLP with numpy:

Construct a simple MLP with numpy and do mini-batch gradient descent (we will repeat it with pytorch later).

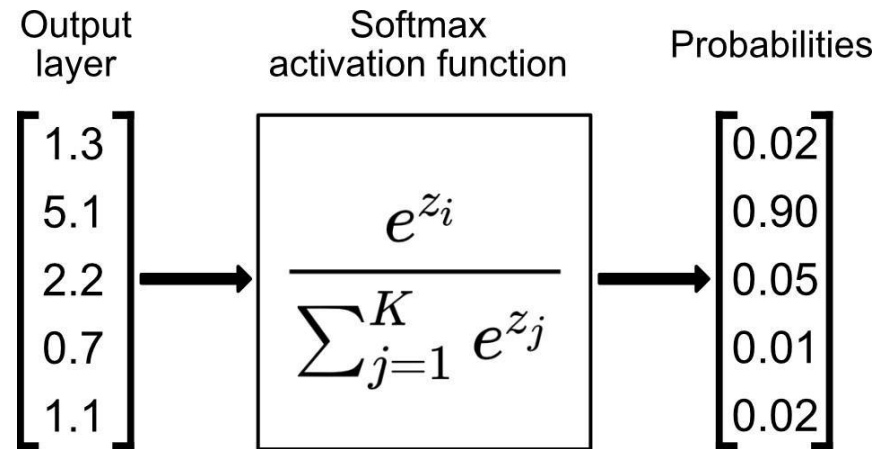
[Lab - MLP in numpy](#)

[Lab - MLP in pytorch](#)

Softmax activation layer:

- Used for multi-class classification problems (excluding categories).
- It is applied to a vector input.
- Probabilistic interpretation of the output:
 - Looks like one-hot encoded.
 - Used with Cross-Entropy Loss.

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$



Class	Value	One-Hot Encoding
0		[1, 0, 0]
1		[0, 1, 0]
2		[0, 0, 1]

Softmax - gradient calculation:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

$$L = - \sum_j y_j \log(\hat{y}_j)$$

Step 1: Compute Derivative of the Loss with Respect to z_j

We want to find $\frac{\partial L}{\partial z_j}$. Start by substituting the softmax expression into the cross-entropy loss.

$$L = - \sum_j y_j \log \left(\frac{e^{z_j}}{\sum_k e^{z_k}} \right)$$

Simplifying:

$$L = - \sum_j y_j \left(z_j - \log \left(\sum_k e^{z_k} \right) \right)$$

Since y_j is one-hot encoded, this simplifies to:

$$L = - \left(z_i - \log \left(\sum_k e^{z_k} \right) \right)$$

where i is the index of the correct class. Now, let's differentiate this with respect to z_j .

Softmax - gradient calculation:

$$L = - \left(z_i - \log \left(\sum_k e^{z_k} \right) \right)$$

We will treat two cases separately:

1. $j = i$ (the correct class)
2. $j \neq i$ (all other classes)

Case 1: $j = i$ (Correct Class)

First, differentiate the loss with respect to z_i :

$$\frac{\partial L}{\partial z_i} = - \frac{\partial}{\partial z_i} \left(z_i - \log \left(\sum_k e^{z_k} \right) \right)$$

This becomes:

$$\frac{\partial L}{\partial z_i} = - \left(1 - \frac{e^{z_i}}{\sum_k e^{z_k}} \right)$$

Since $\frac{e^{z_i}}{\sum_k e^{z_k}} = \hat{y}_i$ (the softmax output for class i):

$$\frac{\partial L}{\partial z_i} = -(1 - \hat{y}_i)$$

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - 1$$

Softmax - gradient calculation:

$$L = - \left(z_i - \log \left(\sum_k e^{z_k} \right) \right)$$

We will treat two cases separately:

1. $j = i$ (the correct class)
2. $j \neq i$ (all other classes)

Case 2: $j \neq i$ (Incorrect Classes)

Now differentiate with respect to z_j where $j \neq i$:

$$\frac{\partial L}{\partial z_j} = - \frac{\partial}{\partial z_j} \left(- \log \left(\sum_k e^{z_k} \right) \right)$$

This becomes:

$$\frac{\partial L}{\partial z_j} = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Again, recognizing that $\hat{y}_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$, we have:

$$\frac{\partial L}{\partial z_j} = \hat{y}_j$$

Softmax - gradient calculation:

$$\frac{\partial L}{\partial z_j} = \begin{cases} \hat{y}_i - 1 & \text{if } j = i \\ \hat{y}_j & \text{if } j \neq i \end{cases}$$

Step 4: Vectorized Form

In practice, this derivative can be written in a more concise vectorized form for all classes j :

$$\frac{\partial L}{\partial z_j} = \hat{y}_j - y_j$$

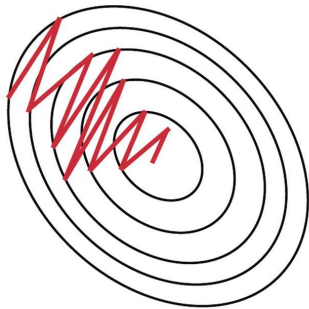
Where:

- $\hat{y}_j$ is the softmax probability for class j ,
- y_j is the one-hot encoded true label for class j .

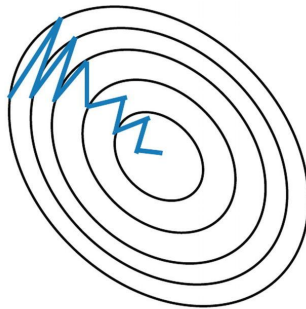
This shows that the gradient of the loss with respect to the logits is simply the difference between the predicted probability $\hat{y}_j$ and the true label y_j , for each class j .

Optimizers - Idea:

- Use momentum!
 - It accelerates the convergence towards the relevant direction and reduces the fluctuation to the irrelevant direction

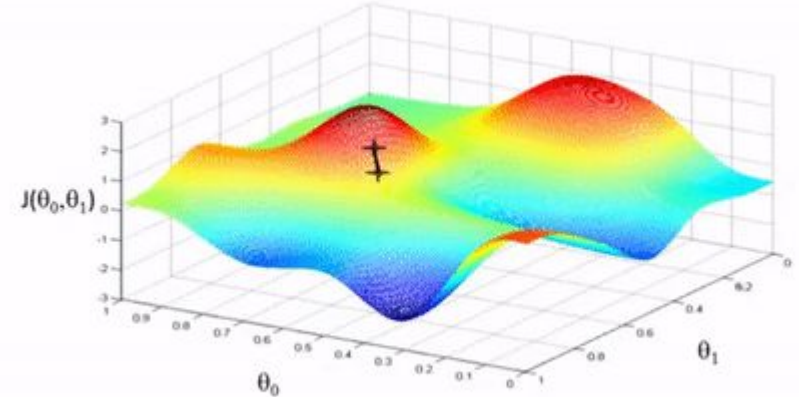


Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Help with plateaus and local minima!



```
past_velocity = 0.  
momentum = 0.1  
while loss > 0.01:  
    w, loss, gradient = get_current_parameters()  
    velocity = past_velocity * momentum + learning_rate * gradient  
    w = w + momentum * velocity - learning_rate * gradient  
    past_velocity = velocity  
    update_parameter(w)
```

Constant momentum factor
Optimization loop

$$v \leftarrow \beta v + (1 - \beta) \nabla_w J(w)$$

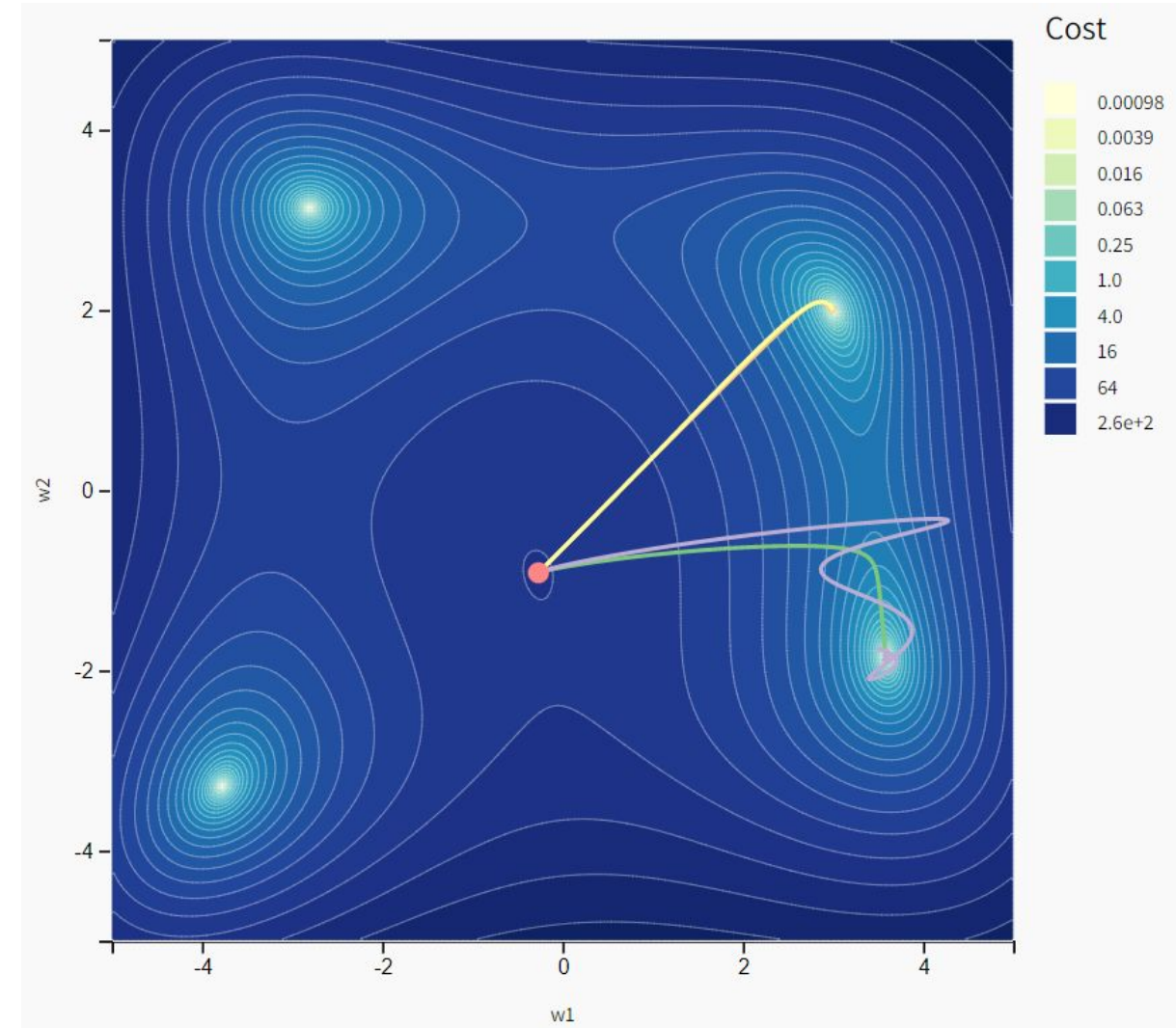
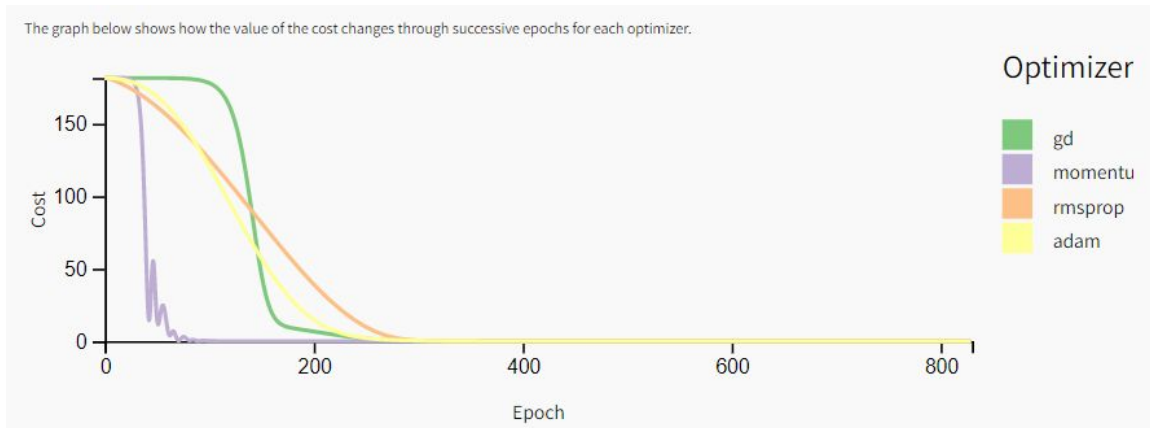
$$w \leftarrow w - \eta v$$

Optimizers - Idea:

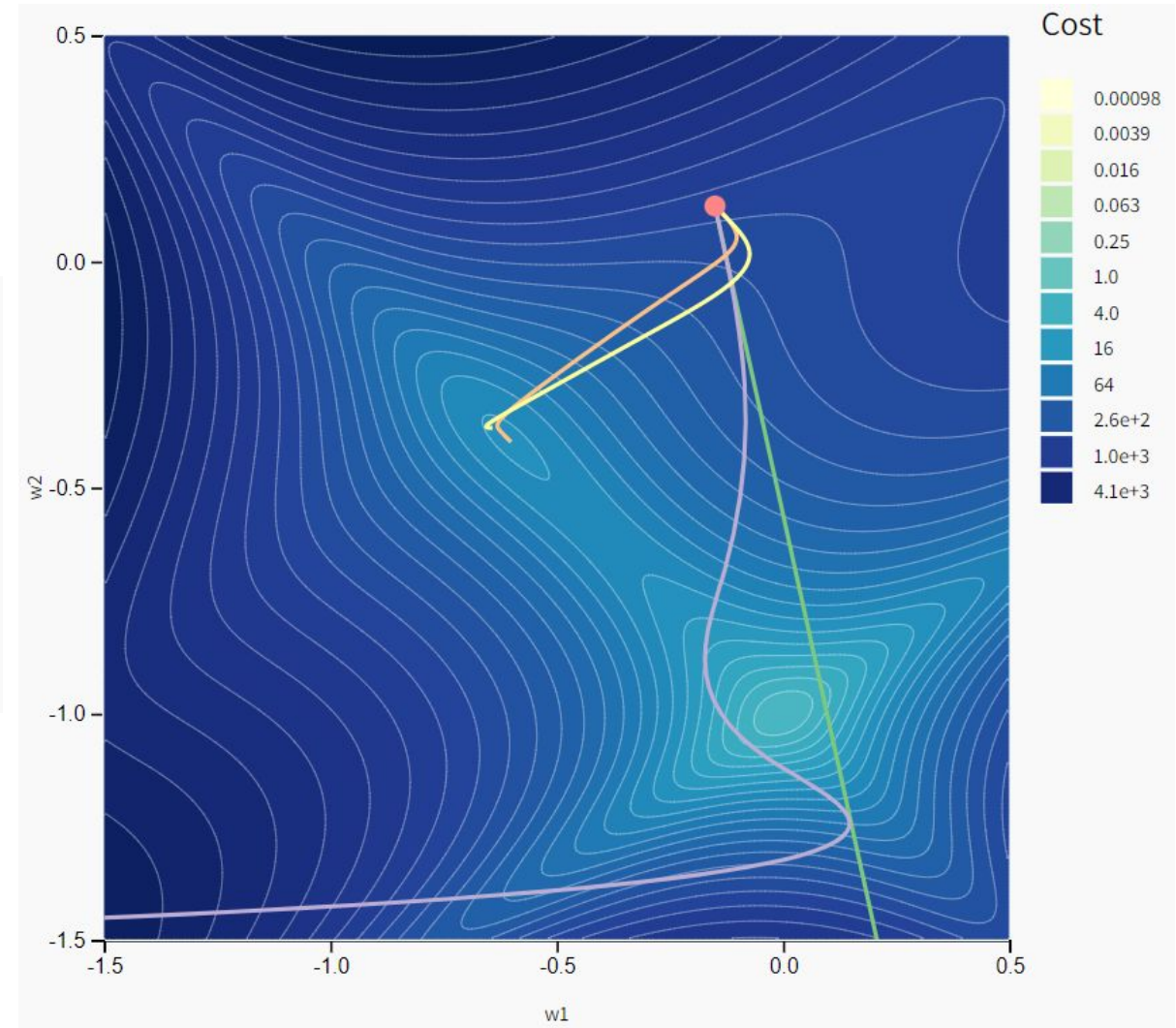
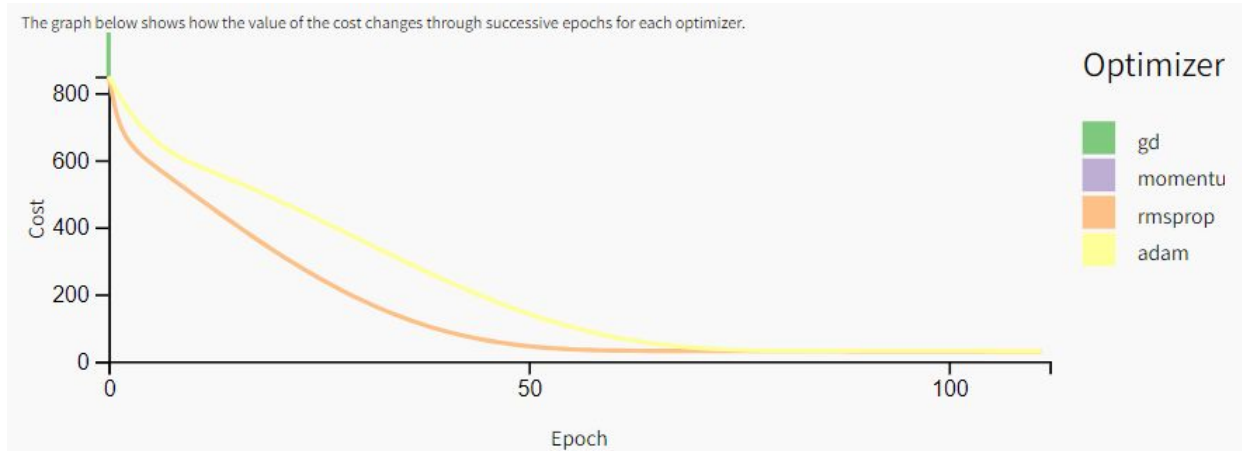
Method	Explanation	Update of w	Update of b
Momentum	• Dampens oscillations • Improvement to SGD • 2 parameters to tune	$w - \alpha v_{dw}$	$b - \alpha v_{db}$
RMSprop	• Root Mean Square propagation • Speeds up learning algorithm by controlling oscillations	$w - \alpha \frac{dw}{\sqrt{s_{dw}}}$	$b \leftarrow b - \alpha \frac{db}{\sqrt{s_{db}}}$
Adam	• Adaptive Moment estimation • Most popular method • 4 parameters to tune	$w - \alpha \frac{v_{dw}}{\sqrt{s_{dw}} + \epsilon}$	$b \leftarrow b - \alpha \frac{v_{db}}{\sqrt{s_{db}} + \epsilon}$

<https://www.deeplearning.ai/ai-notes/optimization/index.html#l>

Optimizers - Examples:

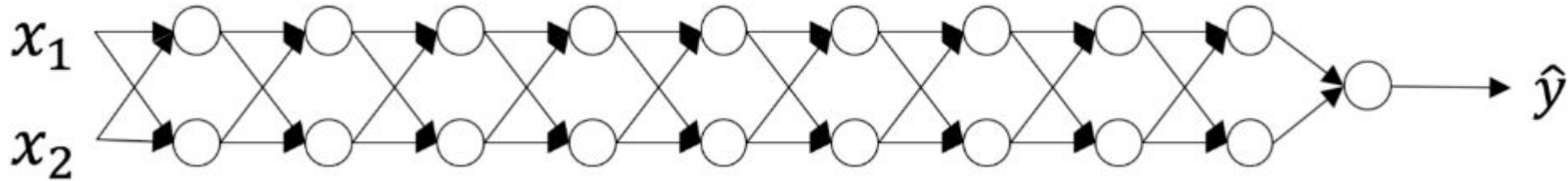


Optimizers - Examples:



Weight Initialization:

- How would you initialize your weights?



$$\hat{y} = a^{[L]} = W^{[L]}W^{[L-1]}W^{[L-2]} \dots W^{[3]}W^{[2]}W^{[1]}x$$

- Too large initialization leads to exploding gradients
 - $\hat{y} = W^{[L]} * 1.5^{L-1} * x$, the values of $a^{[l]}$ increase exponentially.
- Too small initialization leads to vanishing gradients
 - $\hat{y} = W^{[L]} * 0.5^{L-1} * x$, the values of $a^{[l]}$ decrease exponentially.

$$W^{[1]} = W^{[2]} = \dots = W^{[L-1]} = \begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$$

$$W^{[1]} = W^{[2]} = \dots = W^{[L-1]} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

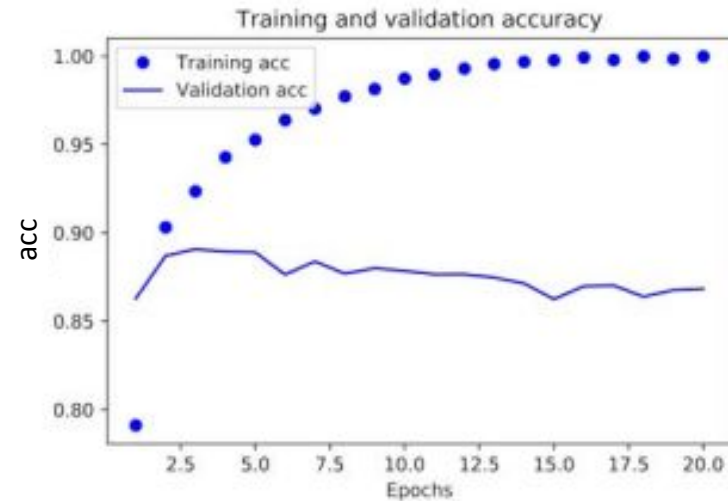
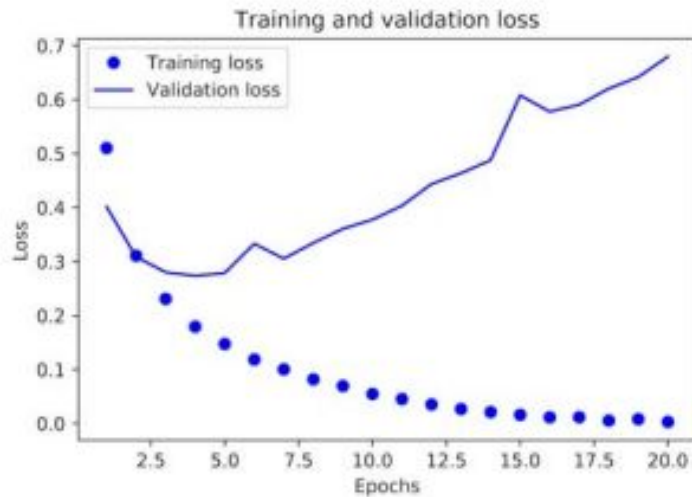
The mean of the activations should be 0 and the variance remain constant.

$$W^{[l]} \sim \mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{n^{[l-1]}})$$
$$b^{[l]} = 0$$

<https://www.deeplearning.ai/ai-notes/initialization/index.html>

Overfitting vs underfitting in NN:

In neural networks we should always verify if the model is generalizing well. It is quite easy to overfit your data.



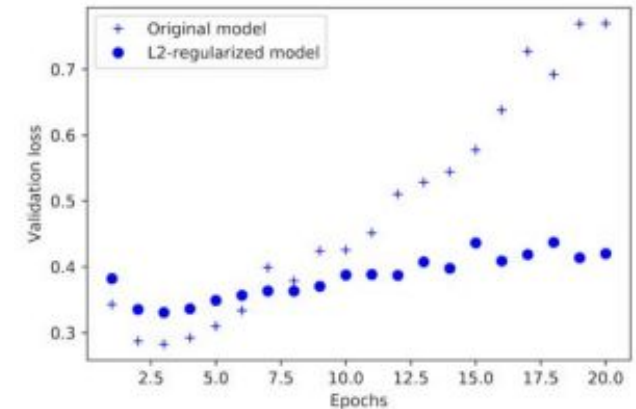
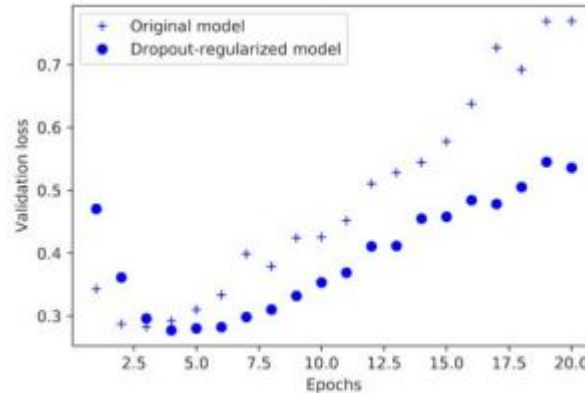
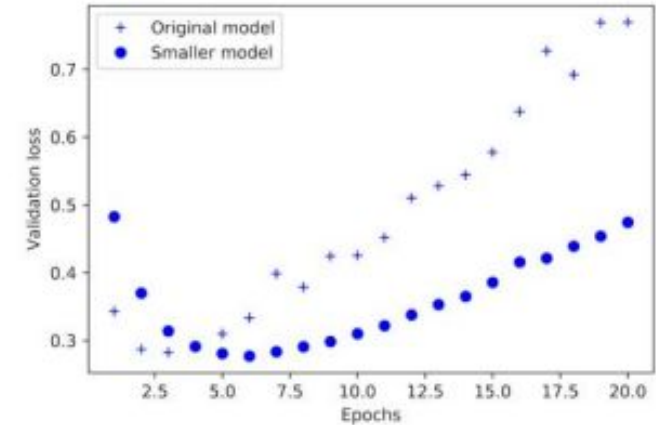
Overfitting vs underfitting in NN:

If your model under-fits your data:

- Make your model deeper.
- Try new architectures.

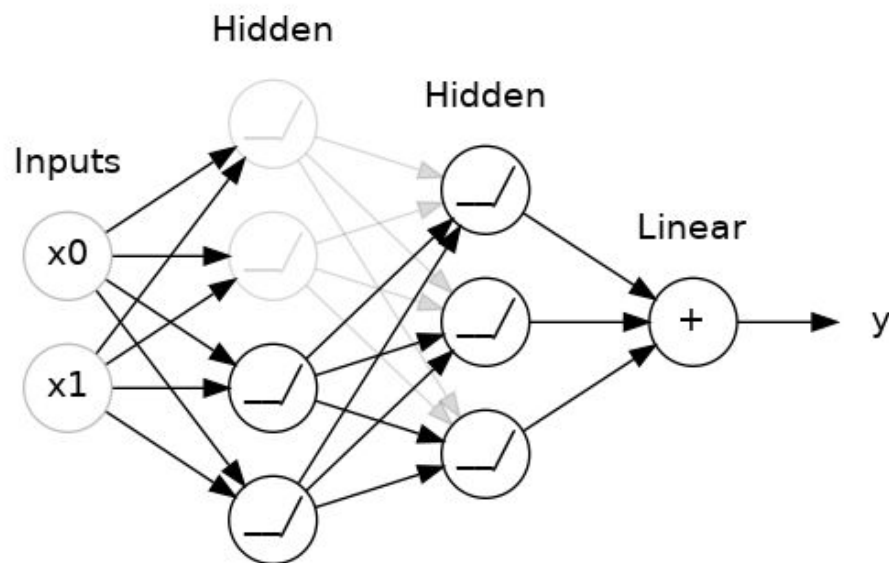
If your model overfits your data:

- Use more data (also data augmentation).
- Simplify your architecture (decrease your depth or number of units).
- Apply L1 and/or L2 regularization to the weights.
- Apply Dropout (explained later).
- Combine many of the above.

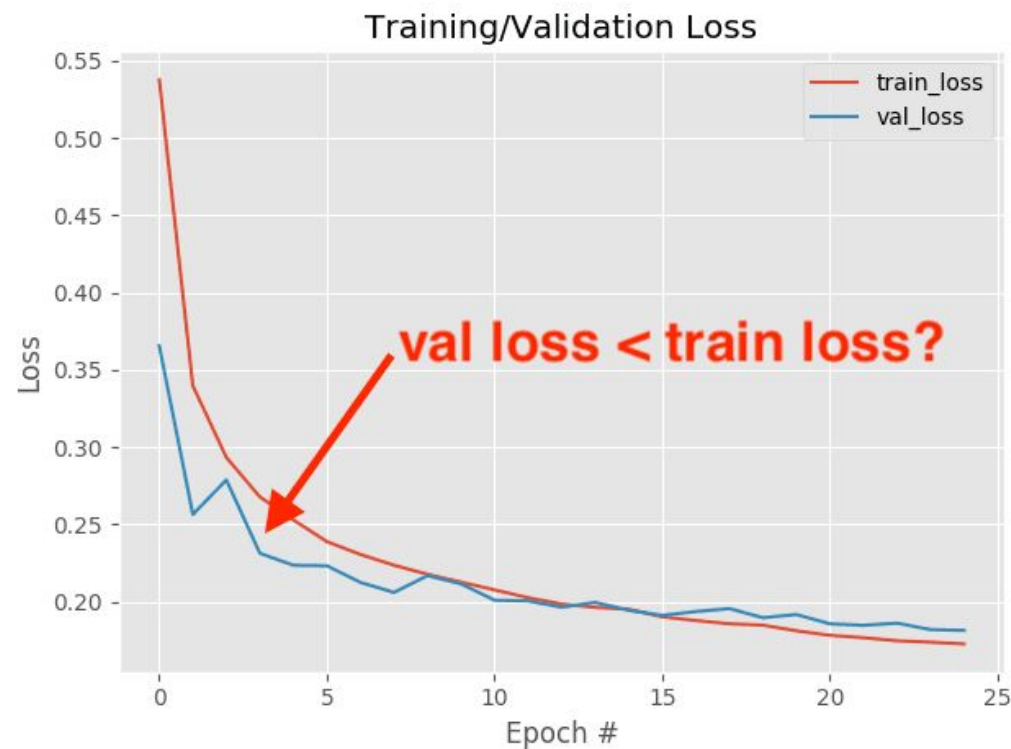


Dropout- CNN:

Inactivate some neurons in training (Helps with regularization). Controlled by a “rate”.



All neurons are used in validation/prediction!



Batchnorm layer:

Helps correct training that is slow or unstable.

- In gradient based algos, it's generally a good idea to put all your data on a common scale
- Maybe also normalizing inside the network would be better!

A batch normalization layer looks at each batch as it comes in, first normalizing the batch with its own mean and standard deviation, and then also putting the data on a new scale with two trainable scaling parameters.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Batchnorm layer:

Helps correct training that is slow or unstable.

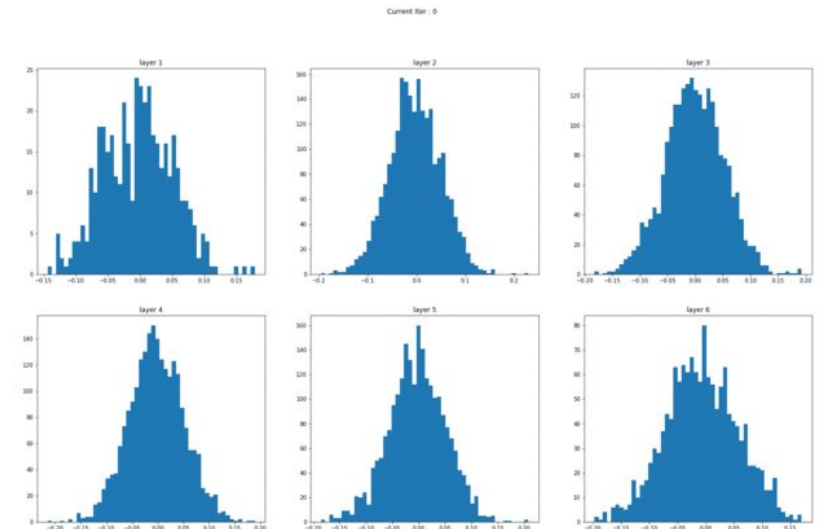
- In gradient based algos, it's generally a good idea to put all your data on a common scale
- Maybe also normalizing inside the network would be better!

A batch normalization layer looks at each batch as it comes in, first normalizing the batch with its own mean and standard deviation, and then also putting the data on a new scale with two trainable scaling parameters.

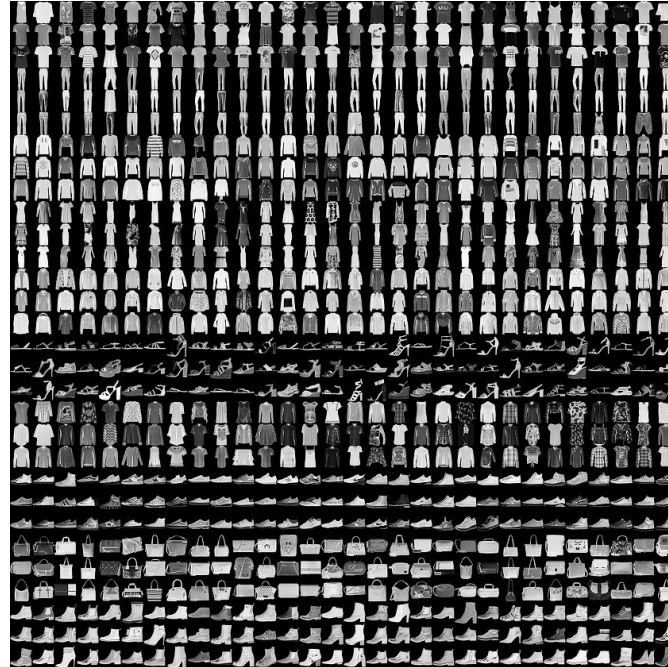
Batch-norm also adds some regularization..

Where to place it?

- After the activation function if for s-shaped functions like the hyperbolic tangent and logistic function
- Before the activation function for activations that may result in non-Gaussian distributions like the rectified linear activation function



Lab – Multiclass classification:



[Lab - multiclass classification](#)

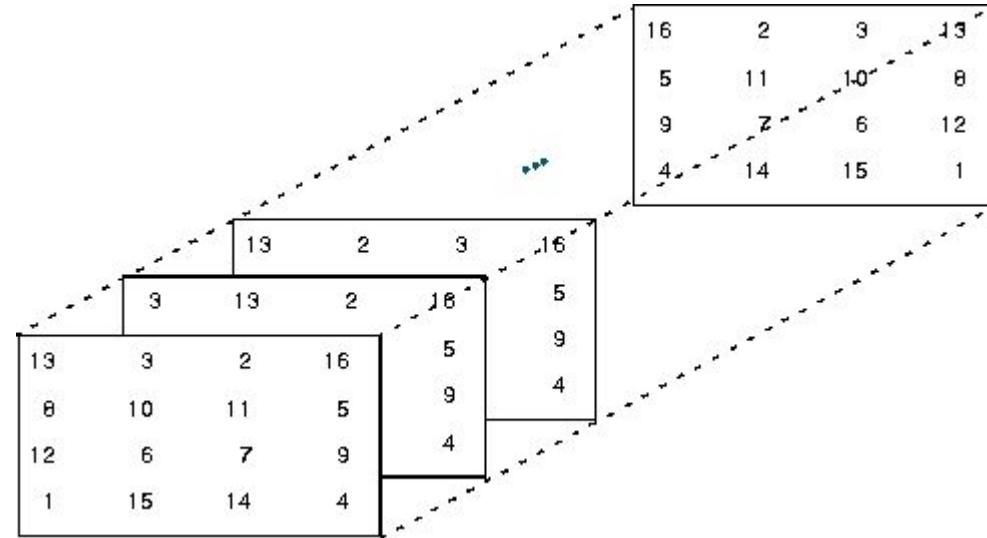
Non-vectorial inputs to NN:

Neural Networks are not restricted to inputs in form of a vector of features. They also accept formatted data, for example:

- 1D-Vectors containing temporal data.
- 2D-Matrices containing (black and white) images.
- 3D-Arrays containing colored pictures.
- 4D-Arrays containing colored video.

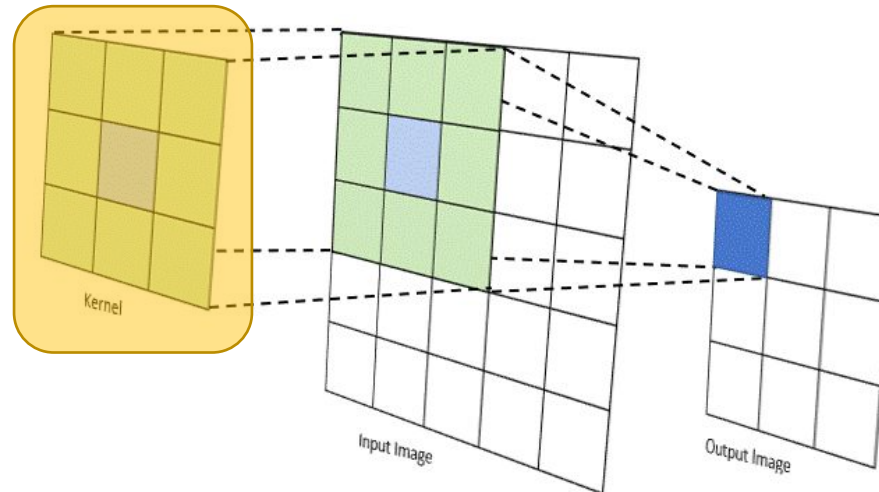
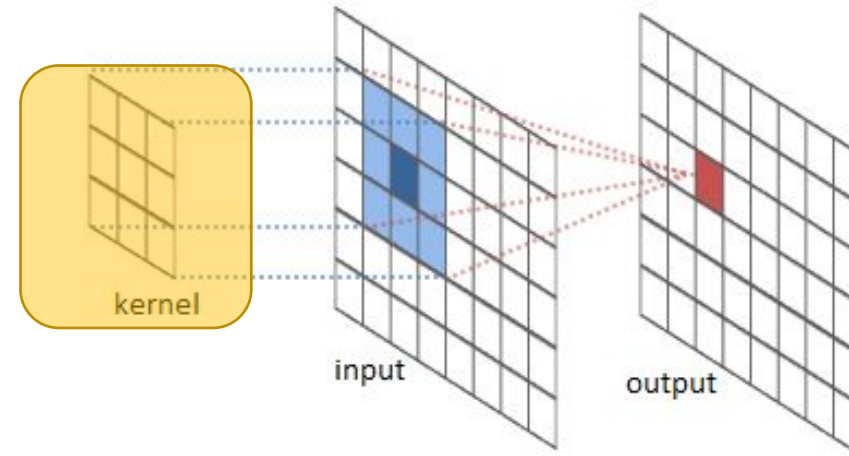
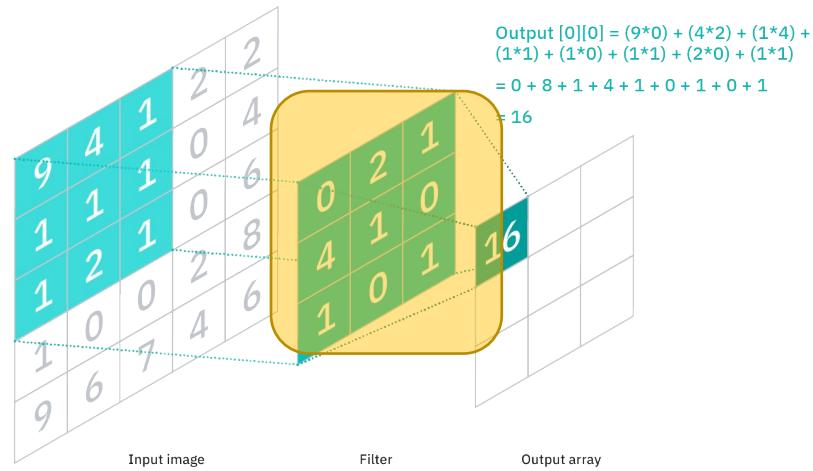
Since neural networks are trained using mini-batch gradient descent, we usually add an extra dimension (0) with the batch size:

- `Input_size = [batch_size, ... ]`
- Sometimes NN require this extra dimension for prediction, and therefore, we will need to reshape new data.



Up to now, we only saw fully connected layers. We will see that there are smarter (or more efficient) ways to learn features for formatted data!

Convolutional layers - CNN:



$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

Are the parameters that we need to learn!

Convolutional layers - CNN:

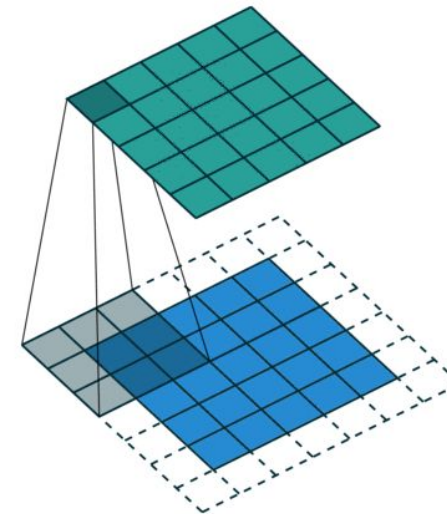
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

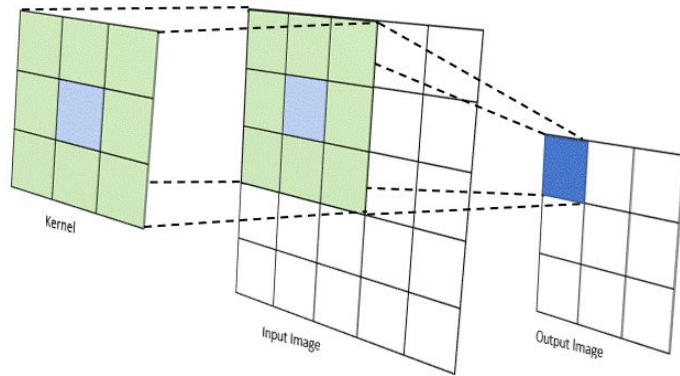
4		

Convolved
Feature

- Same filter (small, few parameters) is applied throughout the image.
 - Parameter 1 -> Kernel size! ((3,3) in the example).
 - Parameter 2 -> Stride, the “jump” between calculations ((1,1) in the example)
 - Parameter 3 -> Padding (add zeros in the border).
- The stride and the padding determine the size of the output matrix.
- The idea is that a filter “learns” certain features of the input tensor/array.



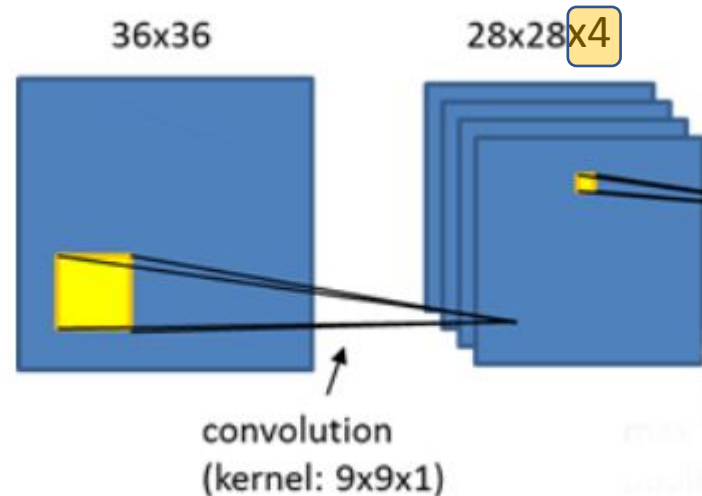
Convolutional layers - CNN:



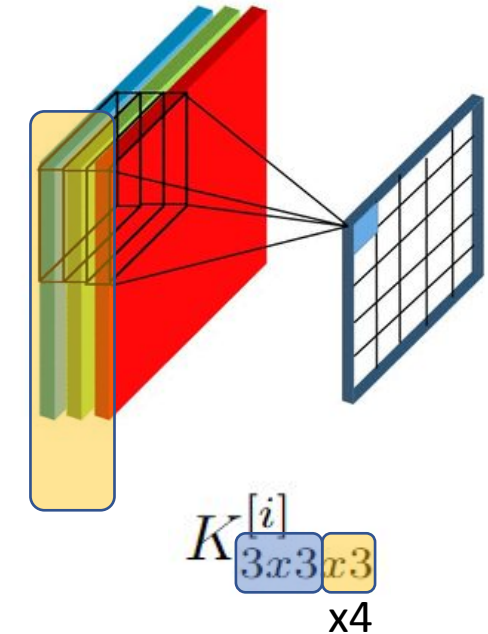
$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

We can have many filters on a single layer (Parameter – “filters”) to learn multiple features of the formatted data.

$$K^{[i]} = \begin{bmatrix} w_1^{[i]} & w_2^{[i]} & w_3^{[i]} \\ w_4^{[i]} & w_5^{[i]} & w_6^{[i]} \\ w_7^{[i]} & w_8^{[i]} & w_9^{[i]} \end{bmatrix}$$



What if I wanted to apply another conv layer? (Third dimension)

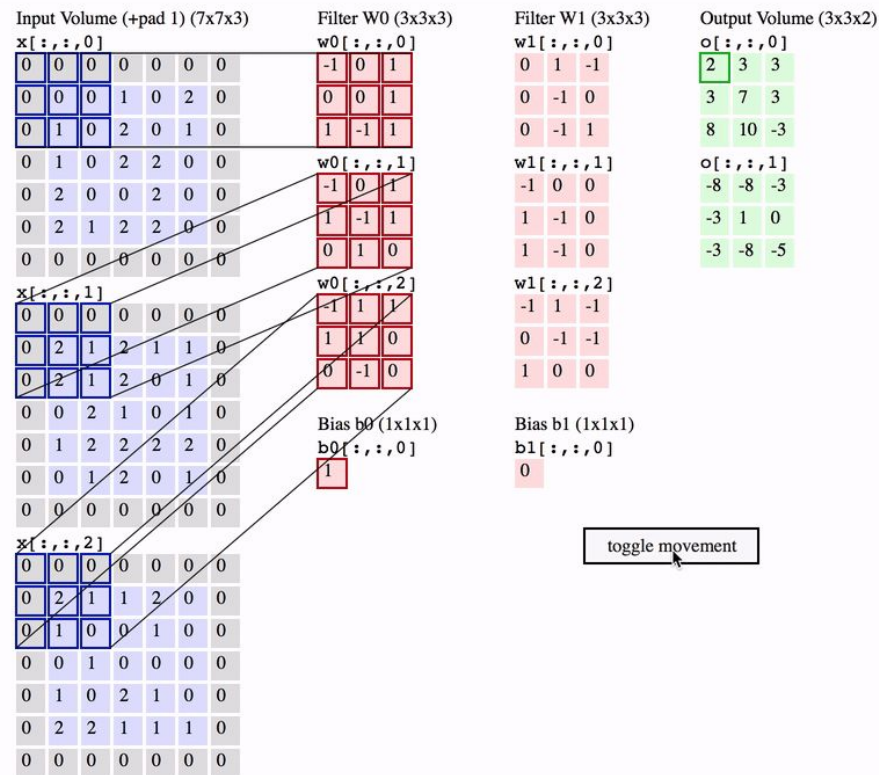


Kernel_size

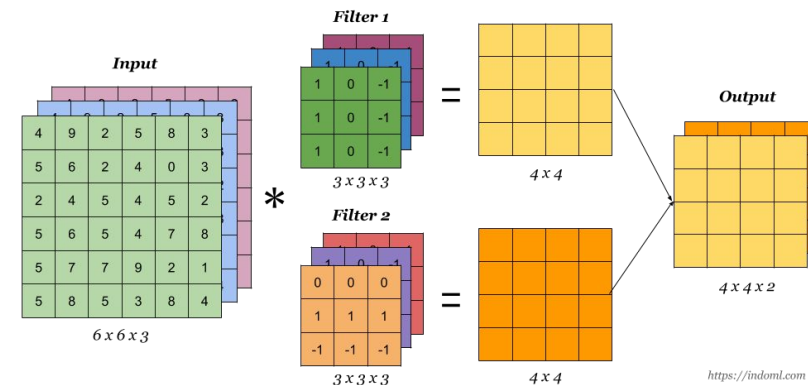
Third dimension to match previous third dimension!

Convolutional layers - CNN:

We can have many filters on a single layer (Parameter – “filters”) to learn multiple features of the formatted data.



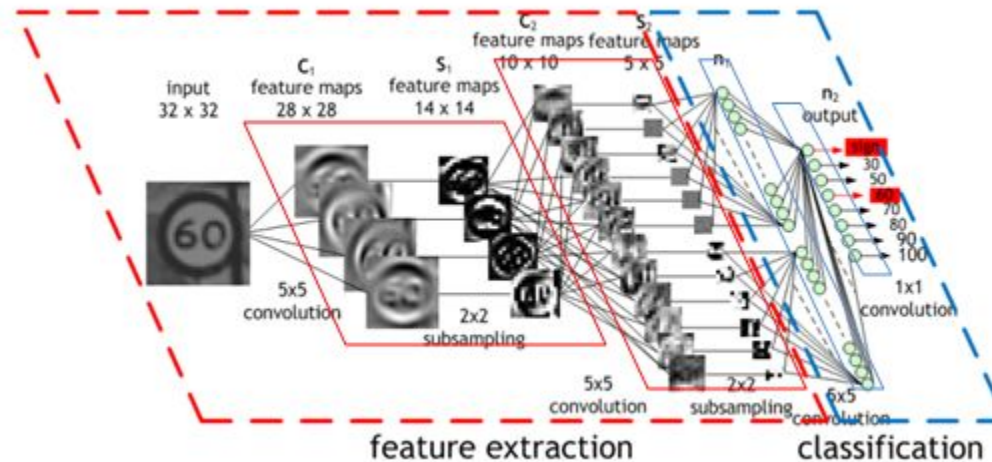
- Input volume (7x7x3).
- Kernel Size (3,3) – 2D convolutional layers take the rest of the dimensions automatically.
 - Kernel size is (3x3x3)
 - Bias (not shown, but there may be)
- Stride (2,2) -> reduces dimensionality.
- Output is (3x3x2) – 2 because we apply 2 filters.



Convolutional layers - CNN:

Stacking convolutional layers in deep models allows layers:

- close to the input to learn low-level features (e.g., lines)
- layers deeper in the model to learn high-order or more abstract features, like shapes or specific objects.

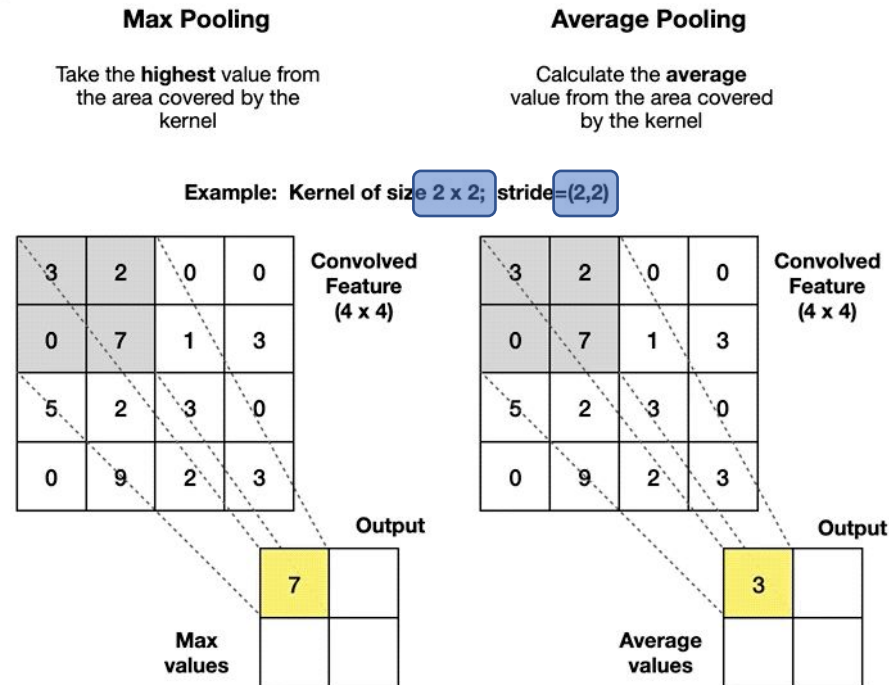


A limitation of the feature map output of convolutional layers is that they record the precise position of features in the input. A common approach to addressing this problem from signal processing is called down-sampling. This is where a lower resolution version of an input signal is created that still contains the large or important structural elements, without the fine detail that may not be as useful to the task.

Pooling layers - CNN:

Down-sampling can be performed with the parameter Stride. For example, a stride = (2,2) will divide your size by 2 every successive conv-layer.

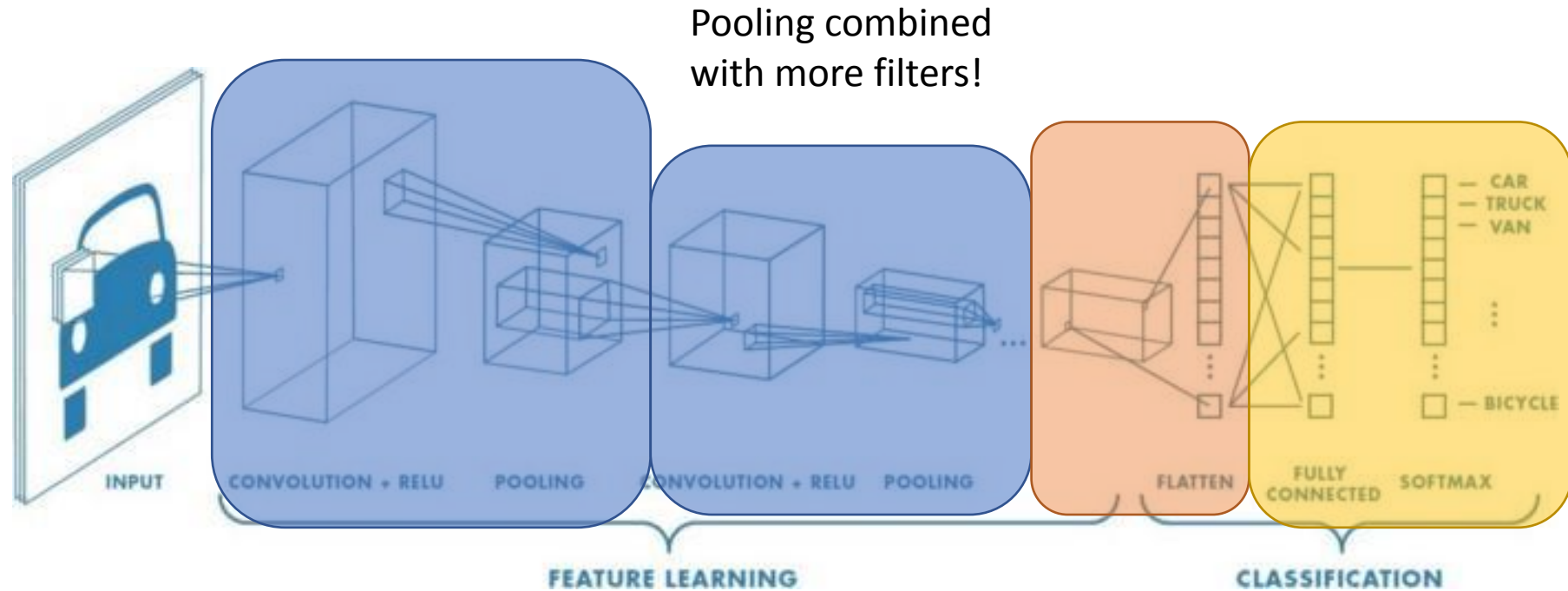
A more robust and common approach is to use a pooling layer. (After the non-linear activation).



The result of using a pooling layer and creating down sampled or pooled feature maps is a summarized version of the features detected in the input.

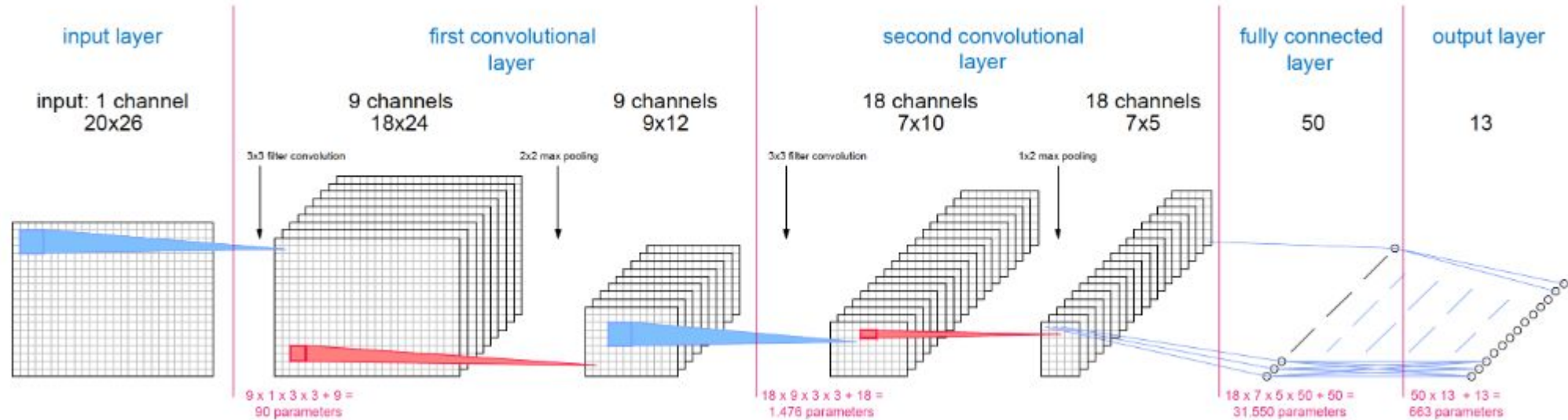
“ In a nutshell, the reason is that features tend to encode the spatial presence of some pattern or concept over the different tiles of the feature map (hence, the term feature map), and it's more informative to look at the maximal presence of different features than at their average presence.

Basic CNN architecture:



How to select the exact hyper-parameters?

Basic CNN architecture:



How to select the exact hyper-parameters?

See what has worked for other people solving similar problems!

Many reference architectures:

1. LeNet
2. AlexNet
3. VGGNet
4. GoogLeNet
5. ResNet
6. ZFNet

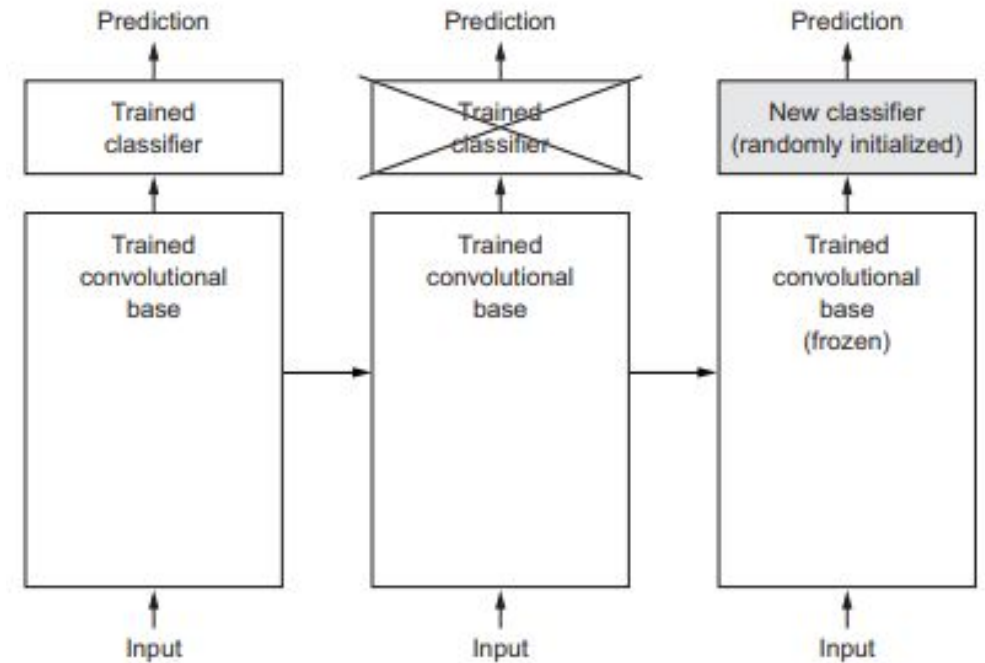
Millions of
parameters to
learn!

Transfer learning:

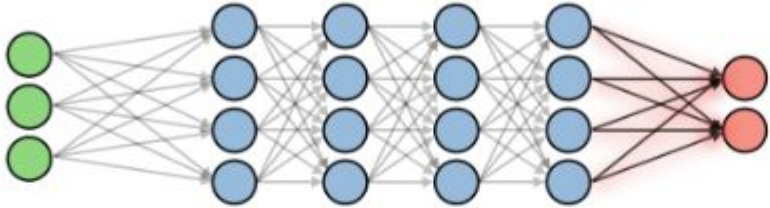
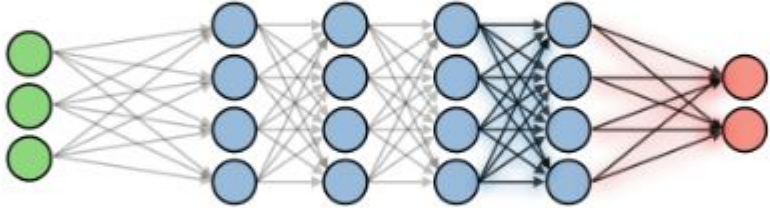
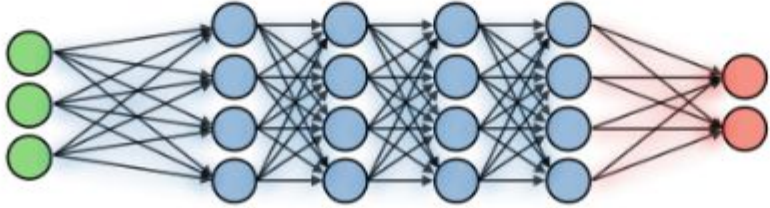
You can use the same neural net as Google, but you do not have the same amount of data!

Since the first conv layers learn to extract complex features from the data (i.e., images), you can use that knowledge to new problems!

- Download a trained net (with the learned weights).
- Fix the first layers, where the complex feature engineering happens.
- Modify or simply random initialize the last Fully connected layers. This layers usually learn the more specific features of your problem, the “high level features”.
- Train these (fewer) weights with your (smaller amount of) data.



Transfer learning:

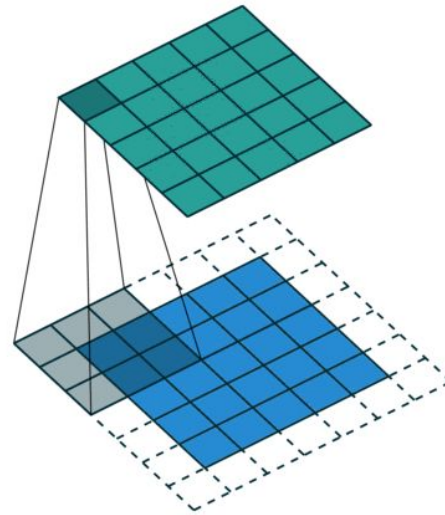
Training size	Illustration	Explanation
Small		Freezes all layers, trains weights on softmax
Medium		Freezes most layers, trains weights on last layers and softmax
Large		Trains weights on layers and softmax by initializing weights on pre-trained ones

Data augmentation:

Original	Flip	Rotation	Random crop
			
• Image without any modification	• Flipped with respect to an axis for which the meaning of the image is preserved	• Rotation with a slight angle • Simulates incorrect horizon calibration	• Random focus on one part of the image • Several random crops can be done in a row
Color shift	Noise addition	Information loss	Contrast change
			
• Nuances of RGB is slightly changed • Captures noise that can occur with light exposure	• Addition of noise • More tolerance to quality variation of inputs	• Parts of image ignored • Mimics potential loss of parts of image	• Luminosity changes • Controls difference in exposition due to time of day

<https://albumentations.ai/>

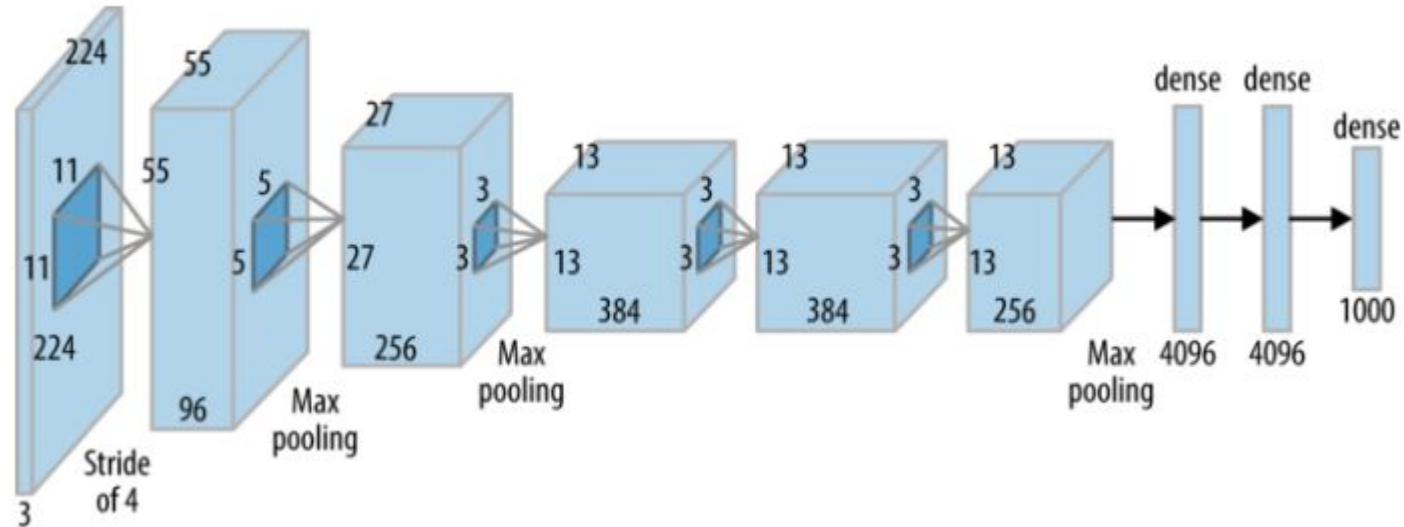
Lab – Convolutional Neural Networks:



[Lab - CNN](#)

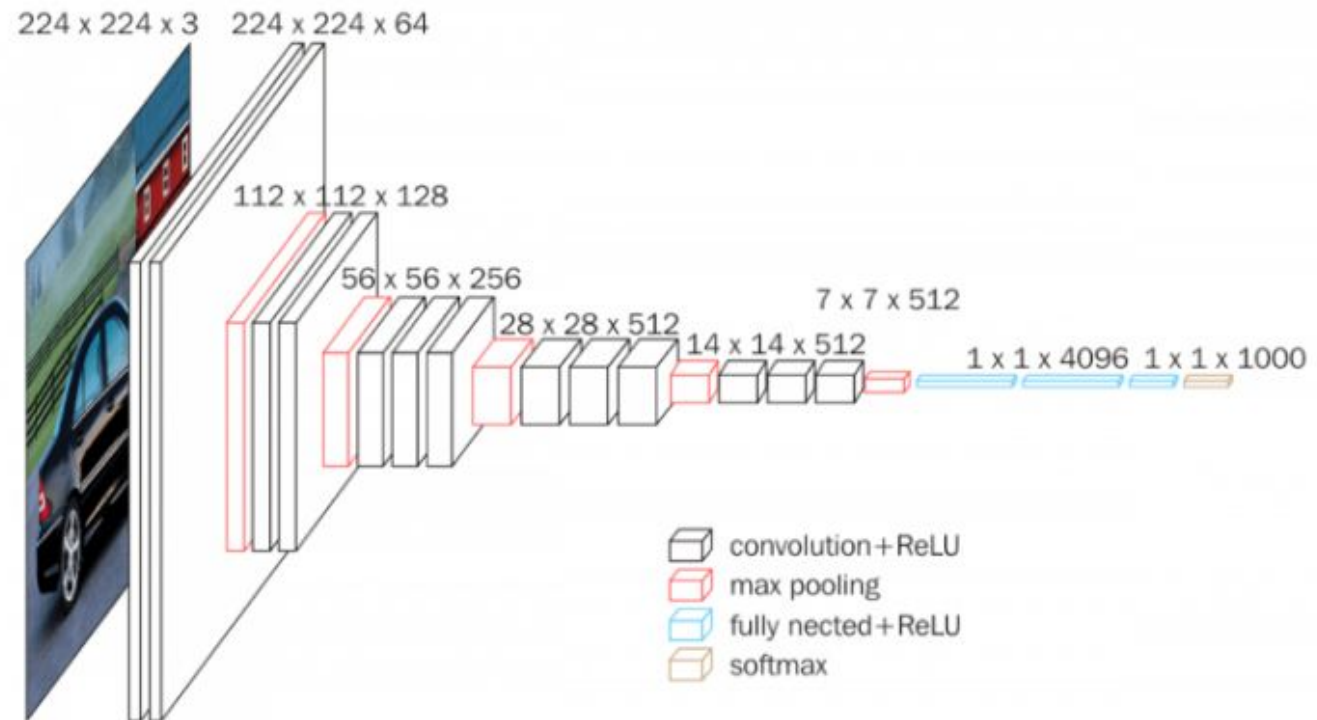
AlexNet:

- First to use ReLu!
- 2012 ImageNet LSVRC-2012 challenge -> 84% acc (73.8%)
- 5 convolutional layers.
- Overlapped max-pooling layer
- 1000 outputs and 62 million trainable variables.
- Local Response Normalization (LRN) for vanishing gradients.
- Use of Dropout.



VGGnet:

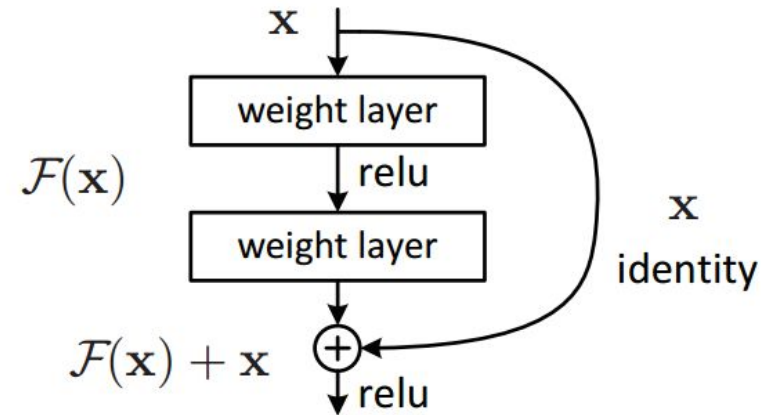
- All the conv kernels are of size 3x3 and Maxpool kernels are of size 2x2 with a stride of two5 convolutional layers.
 - For a 5x5 conv layer filter, the number of variables is 25. On the other hand, two conv layers of kernel size 3x3 have a total of $3 \times 3 \times 2 = 18$ variables (a reduction of 28%).
- VGGNet (VGG16, VGG19, etc.)



Residual Networks and ResNet:

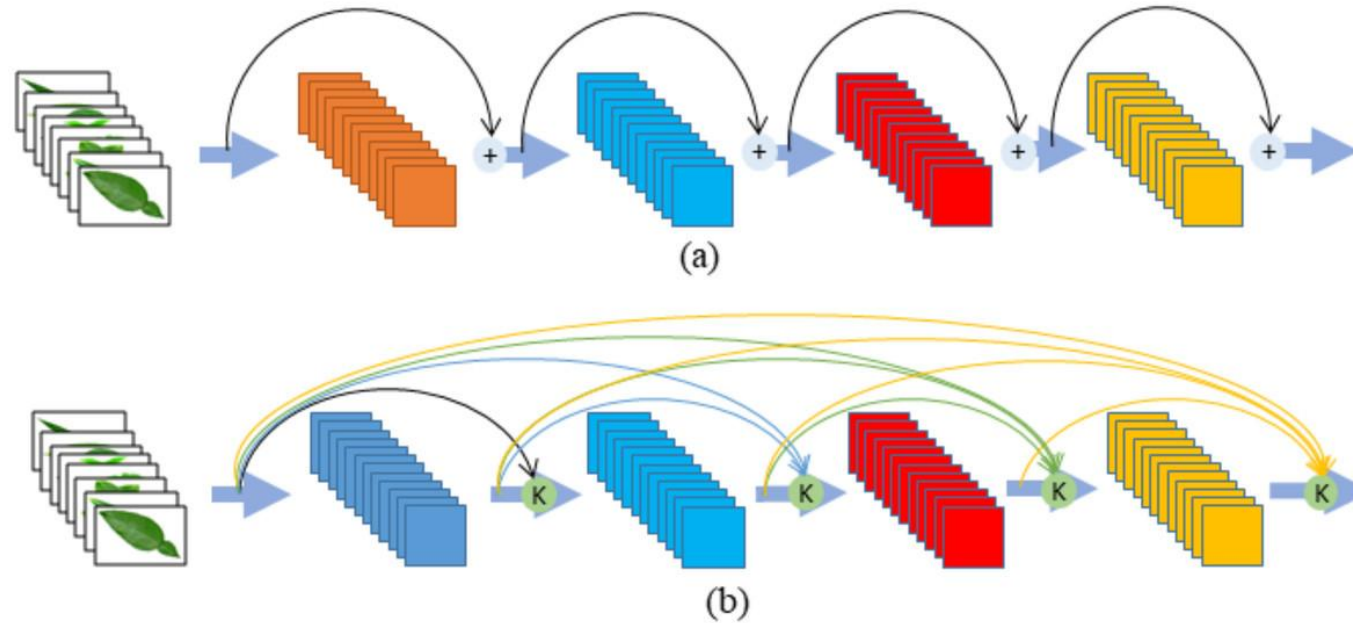
When training very deep networks they saw that 57-layer CNN performed worse than a 20-layer one in both training and validation data. (More layers in theory should do at least as good as less layers).

- Not overfitting because also happened in training data.
- Vanishing gradient made it difficult to learn!



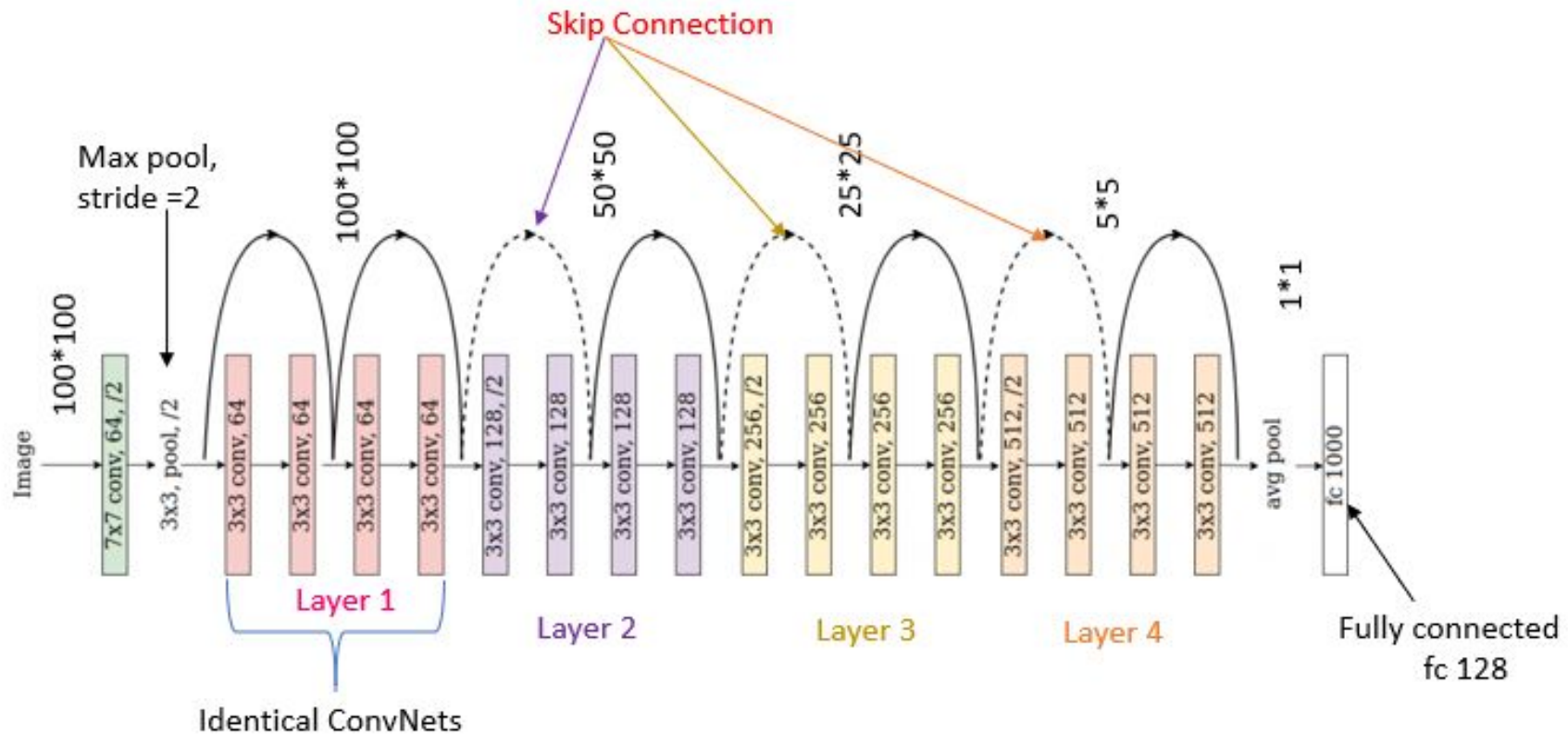
The skip connections in ResNet solve the problem of vanishing gradient in deep neural networks by allowing this alternate shortcut path for the gradient to flow through. The other way that these connections help is by allowing the model to learn the identity functions which ensures that the higher layer will perform at least as good as the lower layer, and not worse.

Residual Networks and ResNet:



ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-110, ResNet-152, ResNet-164, ResNet-1202

ResNet 18:



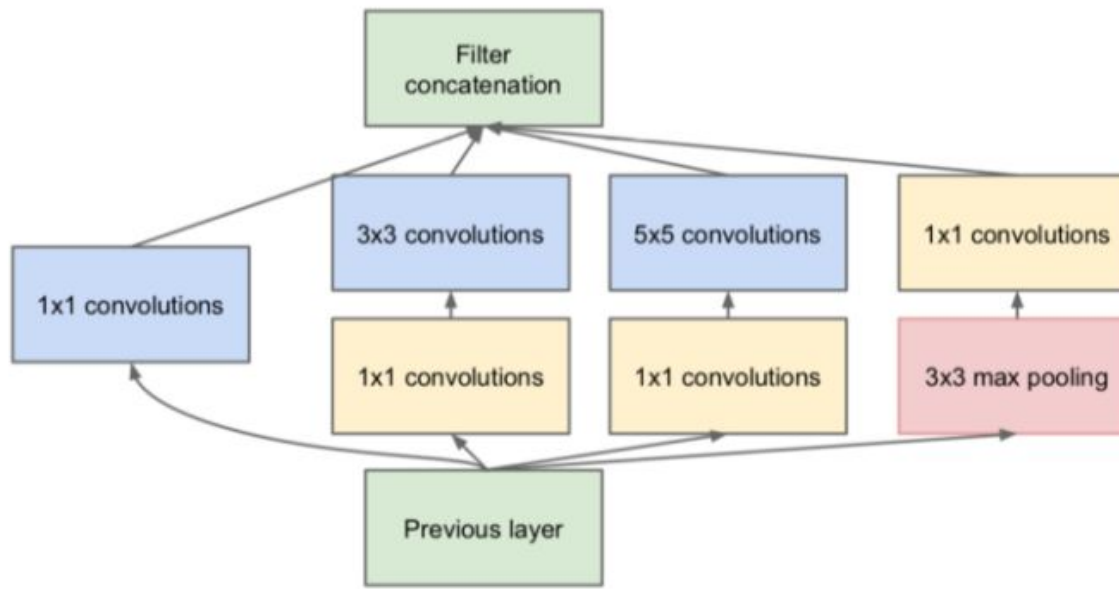
ResNet-18 Architecture

Fruit 360 Input Image size= 100*100 px

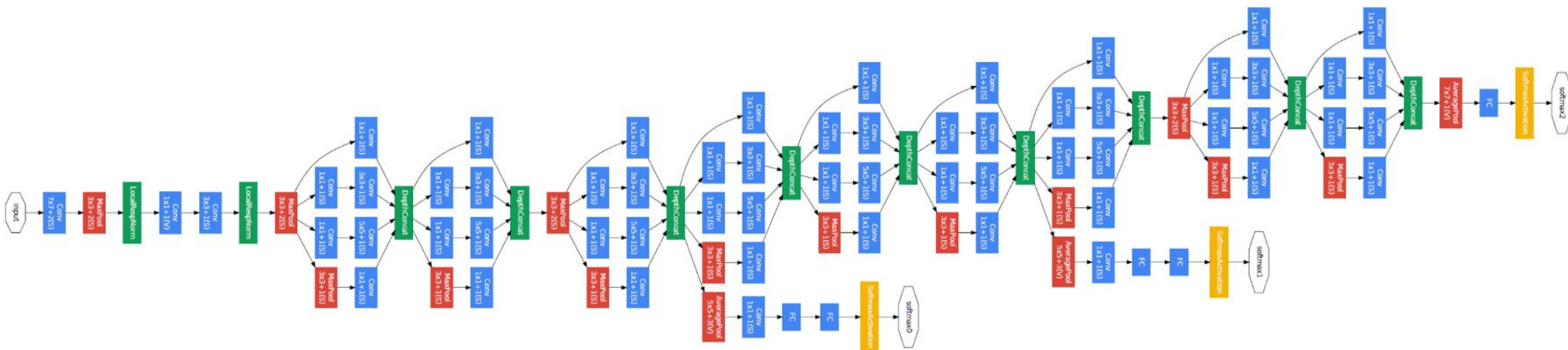
Inception Module:

The size of the salient feature can considerably vary within the image frame. Hence, deciding on a fixed kernel size is rather difficult.

Larger kernels are preferred for more global features that are distributed over a large area of the image, on the other hand, smaller kernels provide good results in detecting area-specific features that are distributed across the image frame.



Inception network:



Global features are captured by the 5x5 conv layer, while the 3x3 conv layer is prone to capturing distributed features. The max-pooling operation is responsible for capturing low-level features that stand out in a neighborhood.

Transpose convolutions:

- Transposed convolutions are the evolution of classical up-sampling methods.
- Instead of representing the data in “smaller” (but deeper) dimensions, with transpose convolutions we can expand the dimensions.
- Controlled by the same parameters: `kernel_size`, `strides`, `padding`...

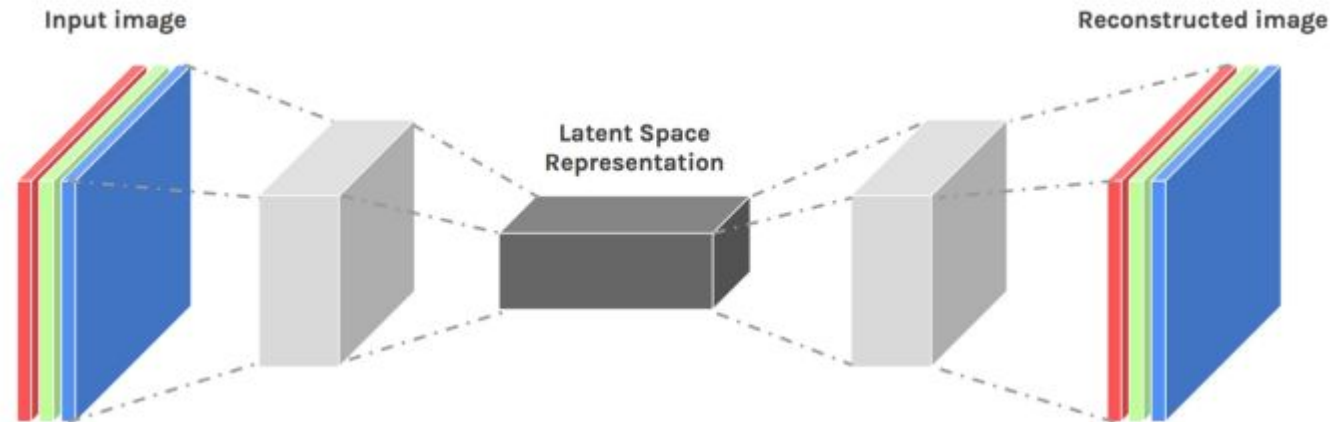
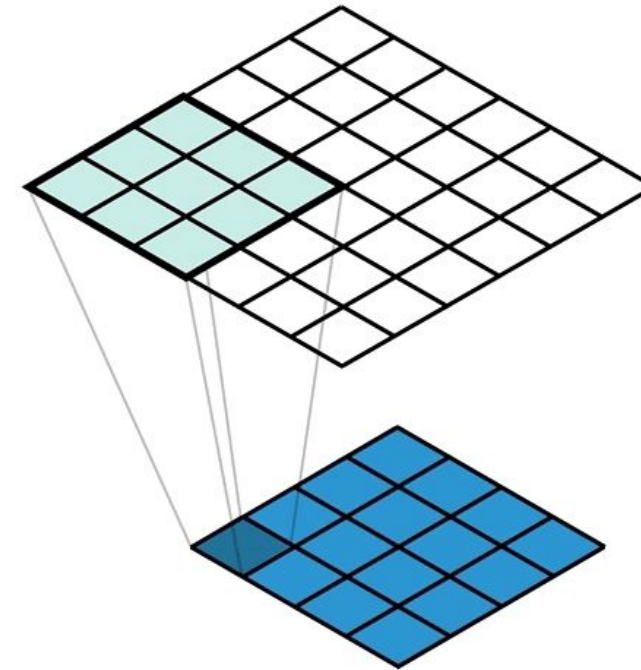
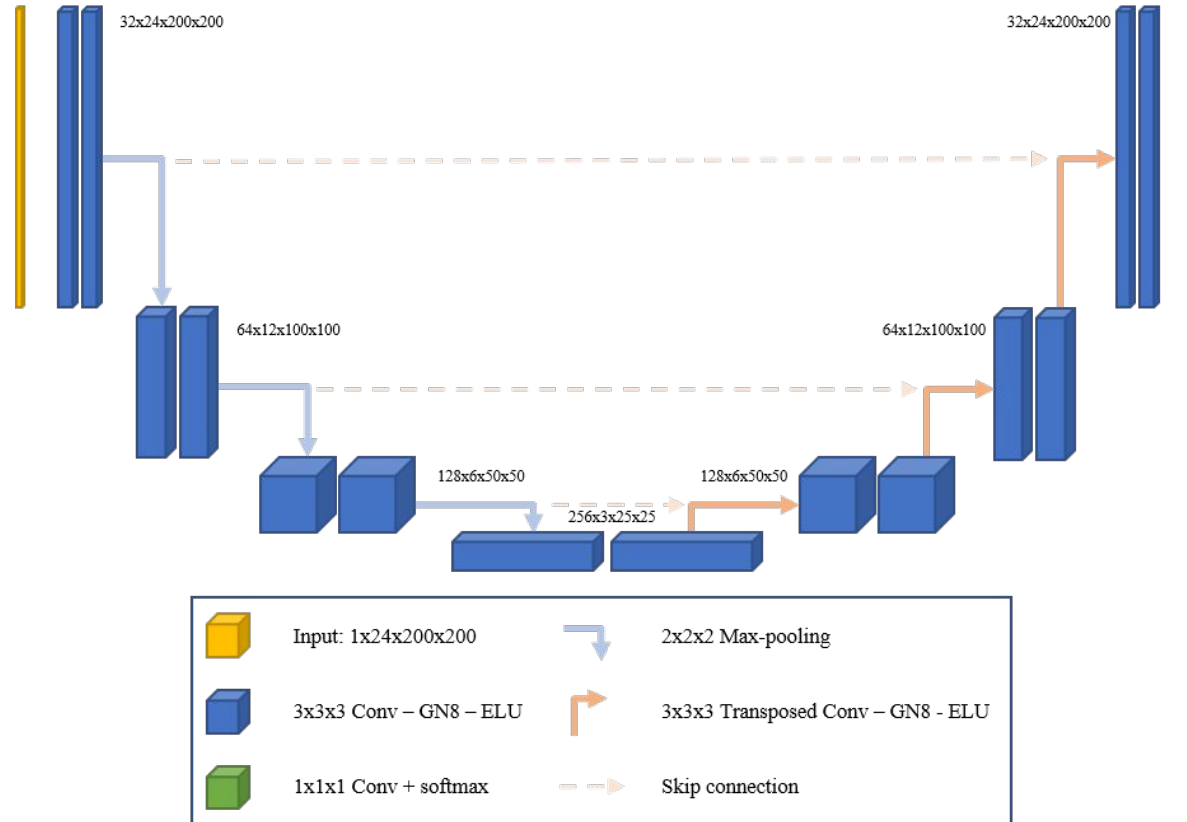
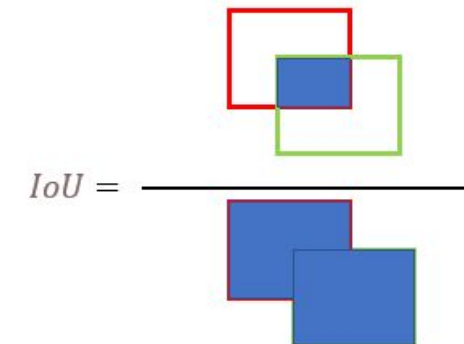


Image segmentation:



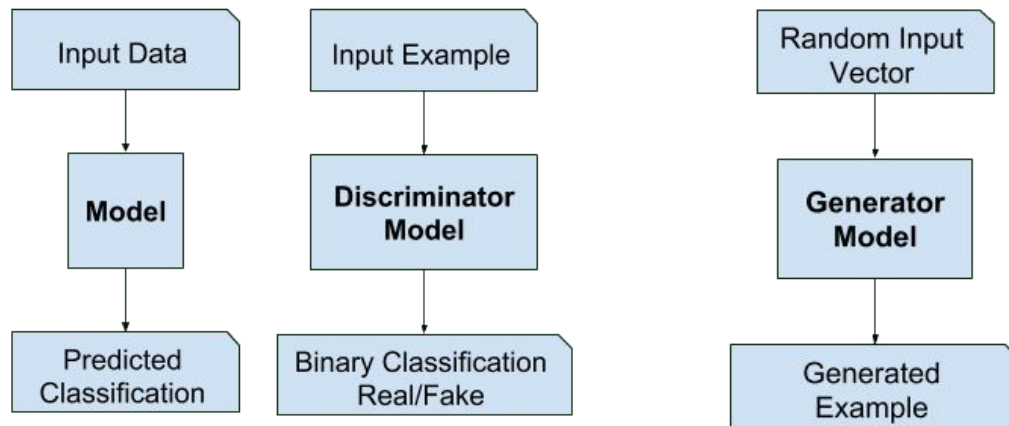
$$IoU = \frac{|P \cap G|}{|P \cup G|}$$



Generative Adversarial Networks:

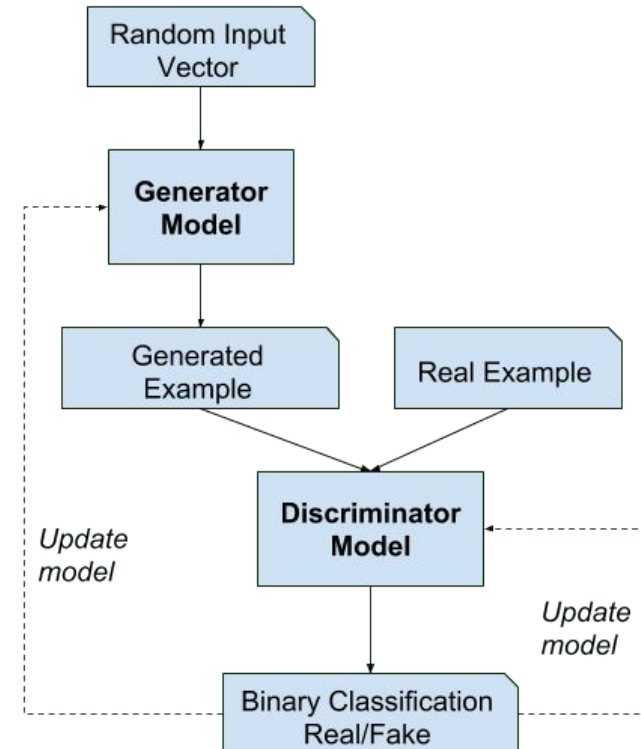
GANs are a clever way of training a generative model by framing the problem as a supervised learning problem with two sub-models:

- the generator model that we train to generate new examples
- discriminator model that tries to classify examples as either real (from the domain) or fake (generated)



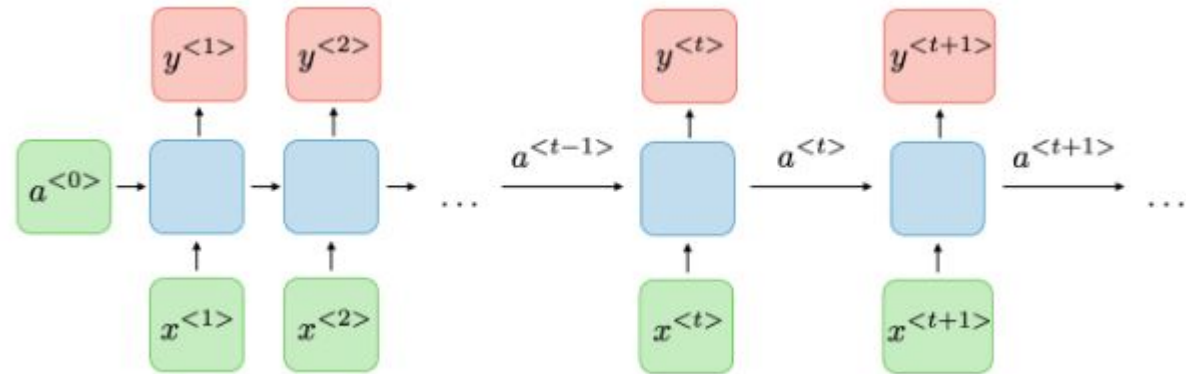
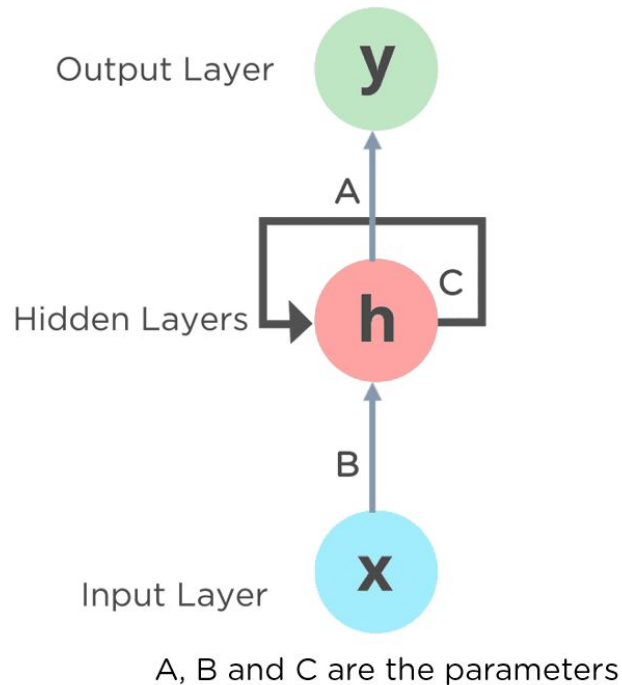
[Demo - GAN faces](#)

<https://www.youtube.com/watch?v=Sw9r8CL98N0>



Recurrent Neural Networks:

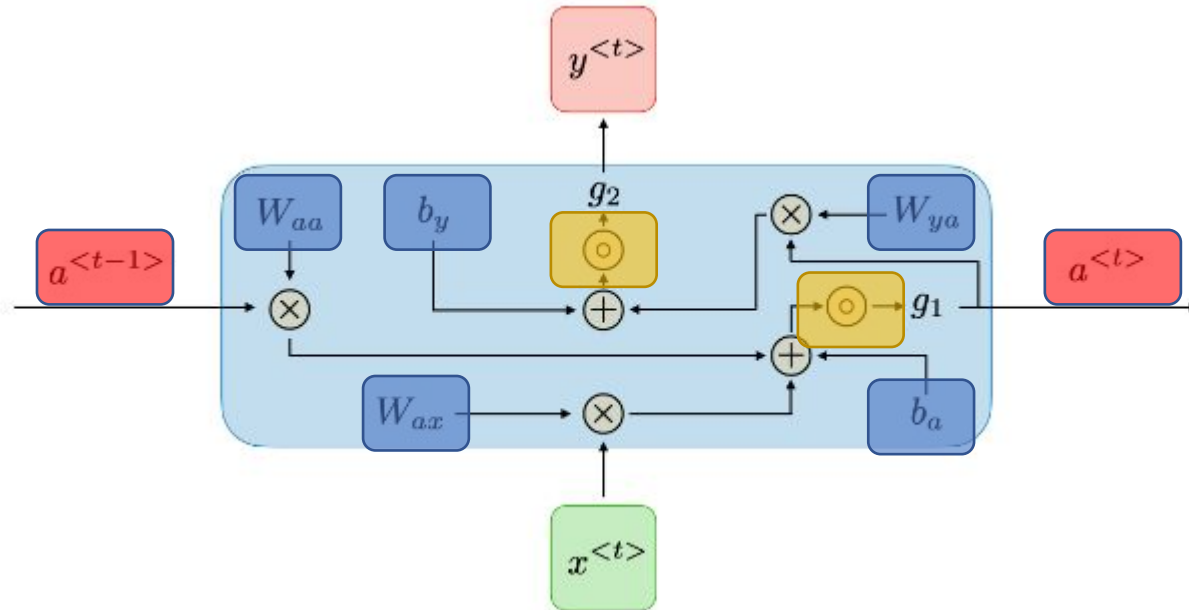
If we want to build a NN for time-series data, CNN may not be the best fit. (Speech, text, SP500, Bitcoin...).



A class of neural networks that allow previous outputs to be used as inputs while having hidden states.

Typical Recurrent Neural Networks:

Output at time t. (e.g., True or false if the word is an entity)

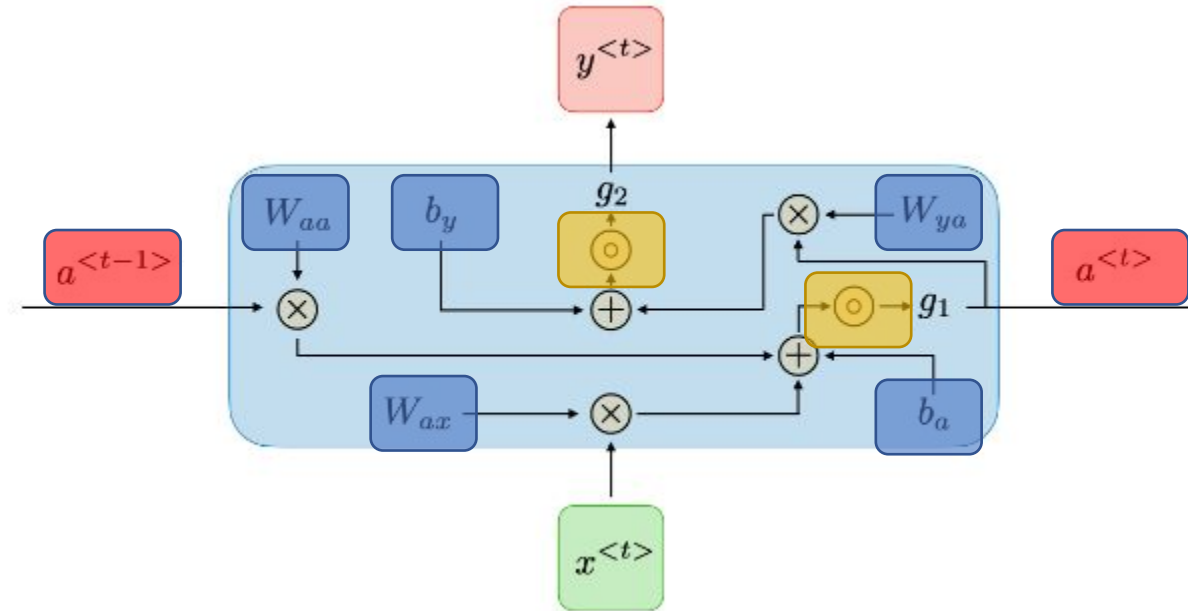


Feature vector for time stamp t.
(e.g., new word of a sentence)

$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$$

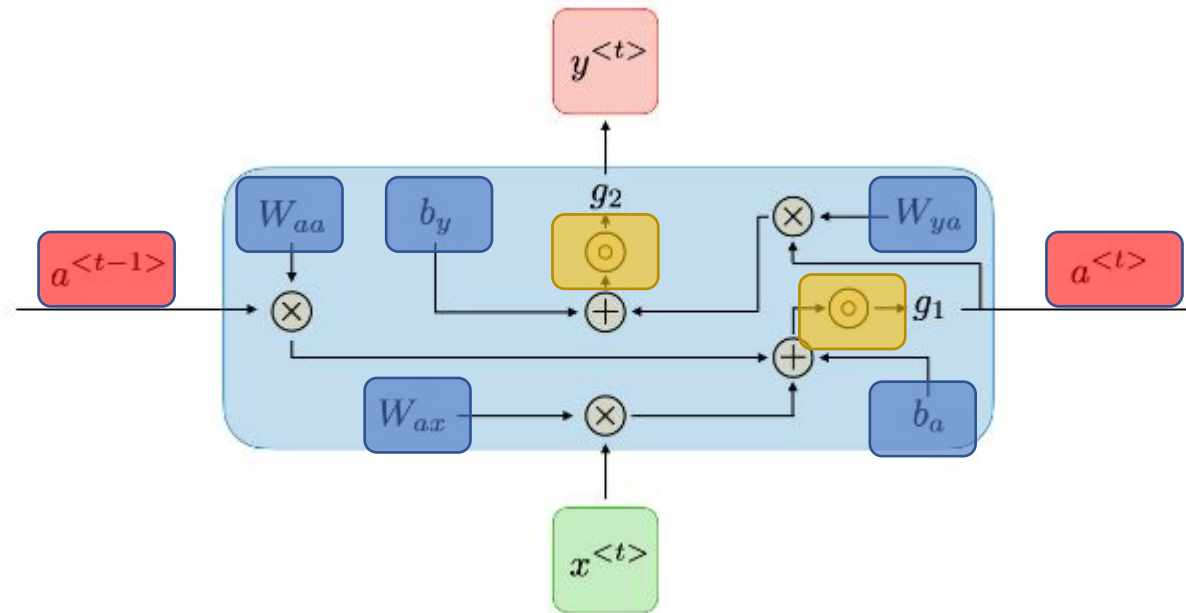
$$y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

Typical Recurrent Neural Networks:



Advantages	Drawbacks
• Possibility of processing input of any length • Model size not increasing with size of input • Computation takes into account historical information • Weights are shared across time	• Computation being slow • Difficulty of accessing information from a long time ago • Cannot consider any future input for the current state

Typical Recurrent Neural Networks:



Loss function

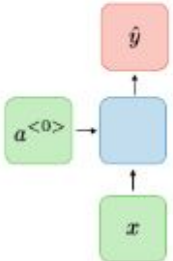
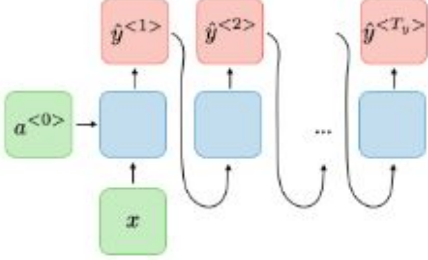
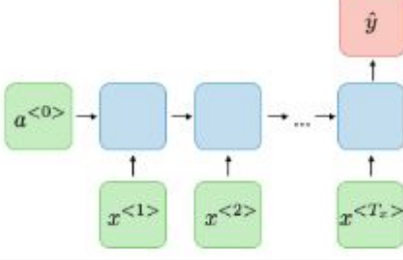
$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_2} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

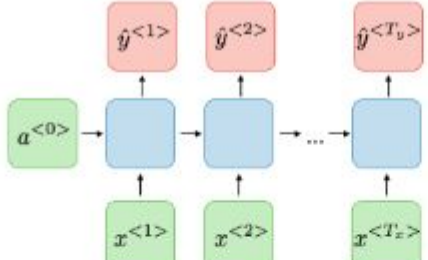
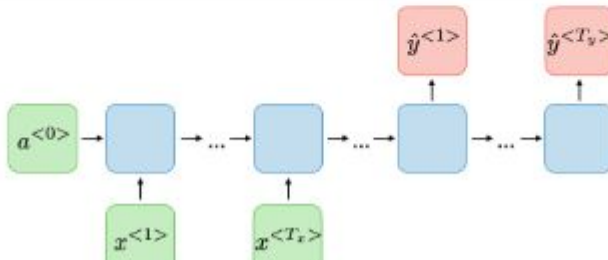
How do we learn?

Backpropagation is done at each point in time.

$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}^{(t)}}{\partial W} \Big|_{(t)}$$

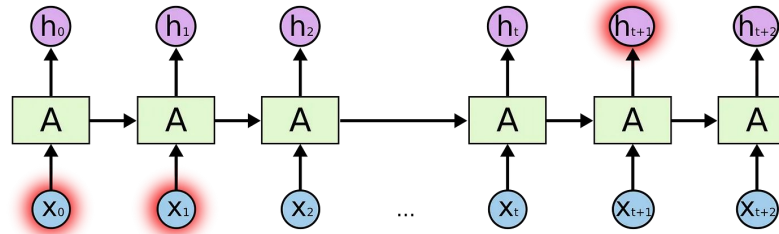
Recurrent Neural Networks - Applications:

Type of RNN	Illustration	Example
One-to-one $T_x = T_y = 1$		Traditional neural network
One-to-many $T_x = 1, T_y > 1$		Music generation
Many-to-one $T_x > 1, T_y = 1$		Sentiment classification

Many-to-many $T_x = T_y$		Name entity recognition
Many-to-many $T_x \neq T_y$		Machine translation

Recurrent Neural Networks:

Recurrent neural networks tend to fail when accessing information from a long time ago. (This is believed to be mainly due to vanishing gradients)



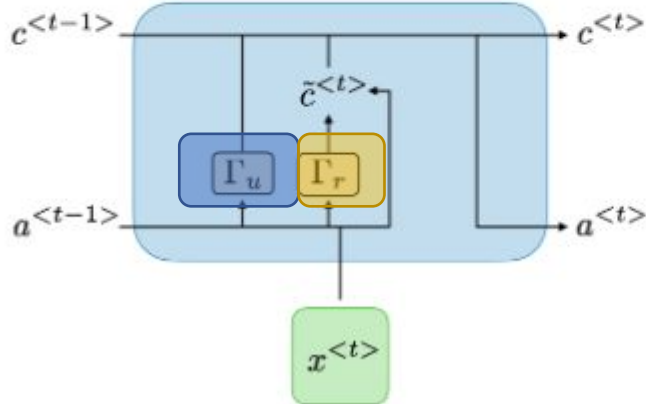
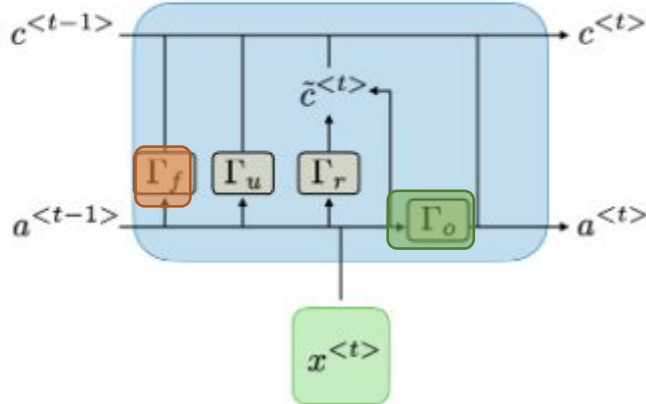
In order to remedy this, several versions of Gates have been designed, each with its own purpose (but all with similar structure):

$$\Gamma = \sigma(Wx^{<t>} + Ua^{<t-1>} + b)$$

Type of gate	Role	Used in
Update gate Γ_u	How much past should matter now?	GRU, LSTM
Relevance gate Γ_r	Drop previous information?	GRU, LSTM
Forget gate Γ_f	Erase a cell or not?	LSTM
Output gate Γ_o	How much to reveal of a cell?	LSTM

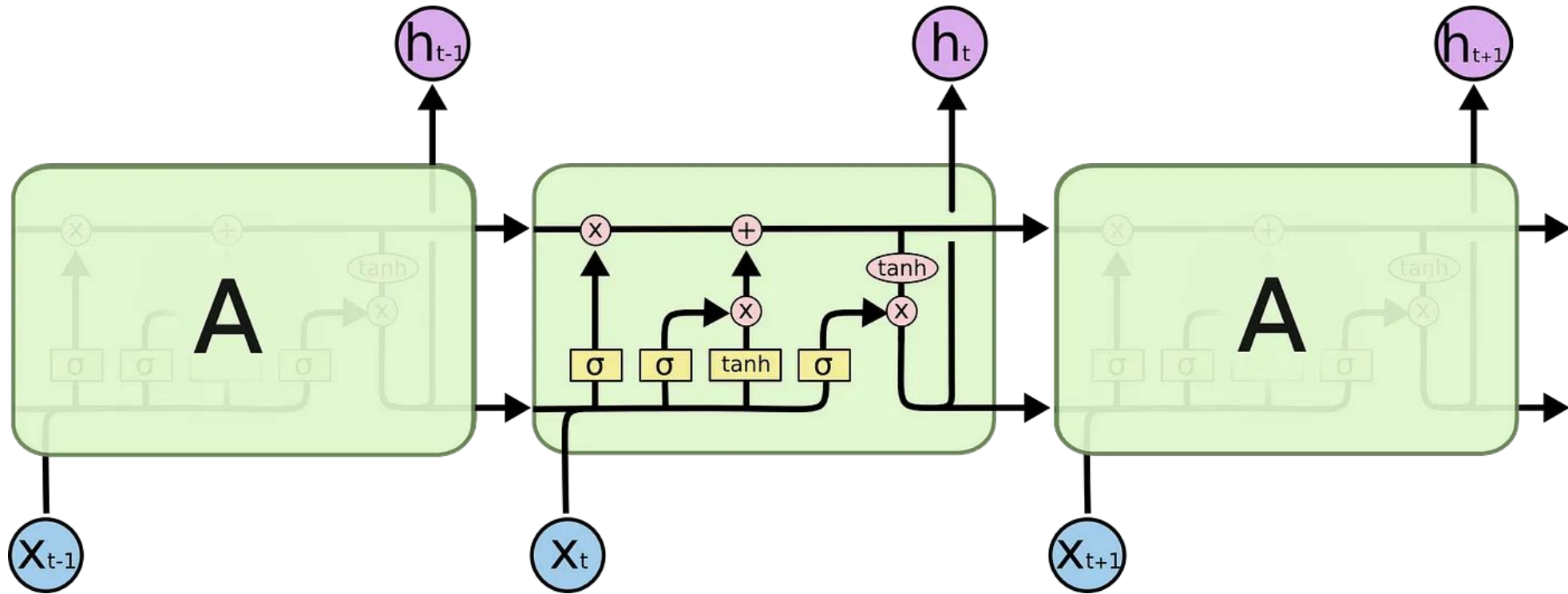
Although all have the same format (output a value), each has its own weights and are used in different ways!

GRU and LSTMs:

Characterization	Gated Recurrent Unit (GRU)	Long Short-Term Memory (LSTM)
$\tilde{c}^{<t>}$	$\tanh(W_c[\Gamma_r \star a^{<t-1>}, x^{<t>}] + b_c)$	$\tanh(W_c[\Gamma_r \star a^{<t-1>}, x^{<t>}] + b_c)$
$c^{<t>}$	$\Gamma_u \star \tilde{c}^{<t>} + (1 - \Gamma_u) \star c^{<t-1>}$	$\Gamma_u \star \tilde{c}^{<t>} + \Gamma_f \star c^{<t-1>}$
$a^{<t>}$	$c^{<t>}$	$\Gamma_o \star c^{<t>}$
Dependencies		

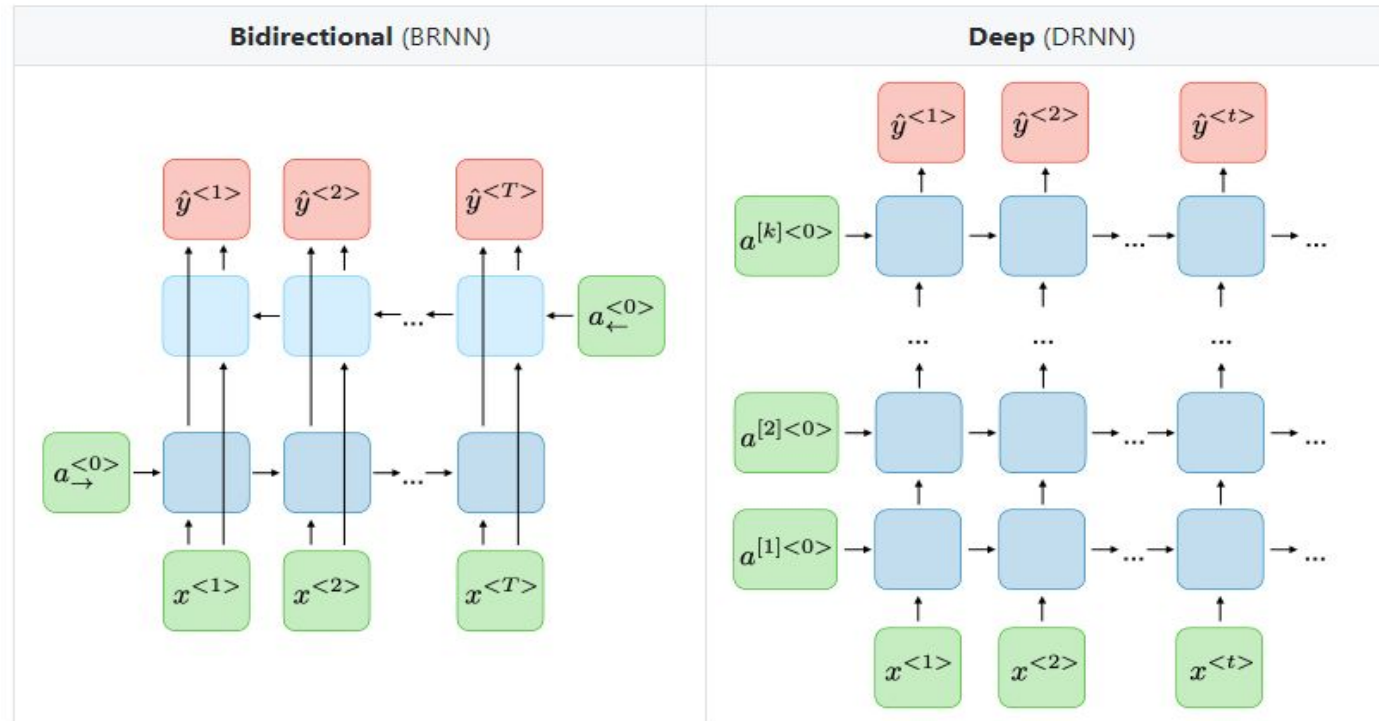
Type of gate	Role	Used in
Update gate Γ_u	How much past should matter now?	GRU, LSTM
Relevance gate Γ_r	Drop previous information?	GRU, LSTM
Forget gate Γ_f	Erase a cell or not?	LSTM
Output gate Γ_o	How much to reveal of a cell?	LSTM

GRU and LSTMs:

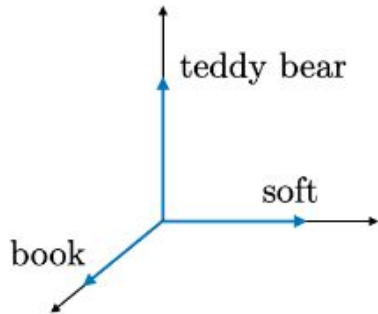
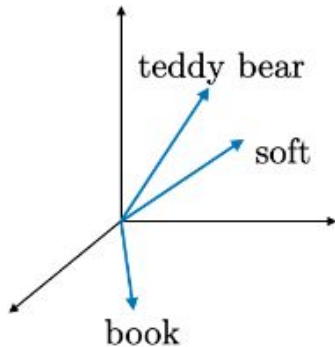


Other RNN:

- What happens if we model a very non-linear problem?
- What if the output should not only depend on the pass (i.e., SP500 values) but we should also use the information from the future (i.e., classify words in a sentence).



Word embeddings - word2vectors:

1-hot representation	Word embedding
 A 3D coordinate system with three axes. Three blue vectors originate from the origin: one pointing along the vertical axis labeled 'teddy bear', one along the horizontal axis labeled 'soft', and one along the diagonal axis labeled 'book'. The vectors are mutually orthogonal.	 A 3D coordinate system with three axes. Three blue vectors originate from the origin: 'teddy bear' points up and to the right, 'soft' points up and to the right but at a shallower angle than 'teddy bear', and 'book' points down and to the left. The vectors are not orthogonal, indicating similarity information.
• Noted o_w • Naive approach, no similarity information	• Noted e_w • Takes into account words similarity

One-hot vectors are very sparse and high-dimensional. To represent words in a more compact way, we map each word w to a **dense vector** $\mathbf{v}_w \in \mathbb{R}^d$, where d is much smaller than the vocabulary size $|V|$. This dense vector is known as the **word embedding**.

To do this, we define an **embedding matrix** $W \in \mathbb{R}^{|V| \times d}$, where each row of the matrix corresponds to the embedding vector of a word. So for each word w , its embedding vector $\mathbf{v}_w$ is the row corresponding to the word in the embedding matrix.

Mathematically, if the one-hot vector of a word w is $\mathbf{w}$, the embedding vector is obtained by the matrix multiplication:

$$\mathbf{v}_w = W^T \mathbf{w}$$

Here, W^T is the transpose of the embedding matrix W , and $\mathbf{v}_w$ is the dense vector representation of the word.

Embedding matrix: learning the embedding matrix can be done using target/context likelihood models.

Demo RNN – Sentiment Analysis:

Demo: Review Sentiment Analysis

We will build a LSTM Recurrent Neural Network that gives 87-88% accuracy on the IMDB Movie Review Sentiment Analysis Dataset.

Built for Google Collaboratory

Train your network more quickly in Google Collaboratory. From the **Runtime** menu select **Change Runtime** Type and choose "GPU"!

Don't forget to select **Runtime** -> **Restart runtime** to put your changes into effect.

[Lab LSTM](#)

Problems with LSTMs:

- Inputs are sequential, no parallelization.
- No explicit modeling of long and short range dependencies.
- Distance between positions is constant.
- Memory problems.

What we want:

- handle different length inputs
- parallel processing of the entire input
- shared parameters (like CNNs)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

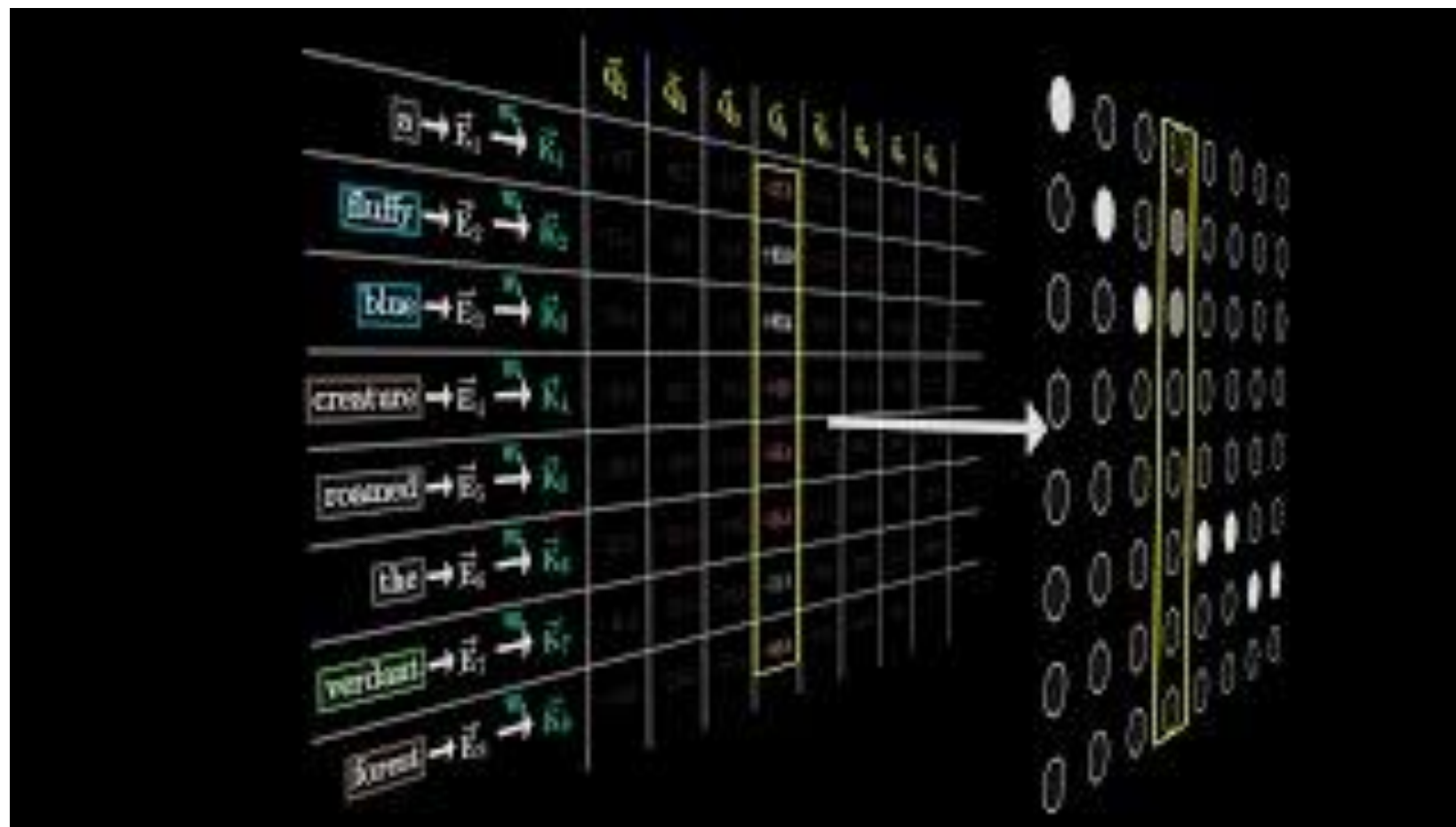
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Why attention:

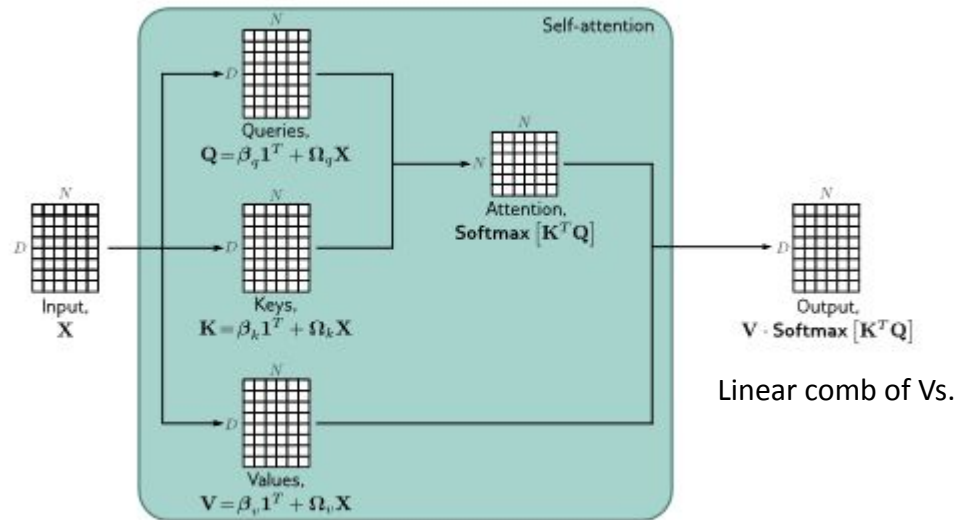
Before attention, sequence models (RNNs and LSTMs) compressed an entire inputs sequence into a single fixed-sized vector before decoding it. Long sequences lost information by the time it reached the output.

Attention fixes this by letting the model look back at every input position every time it produces an output.

Attention mechanism:



Self-attention:



Suppose we have a sequence of n tokens, $\{x_1, x_2, \dots, x_n\}$, and each token is represented as a vector of size d_{model} . The self-attention mechanism proceeds as follows:

1. Input Representation: Query, Key, and Value Vectors

For each token x_i , we create three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**. These vectors are linear transformations of the input:

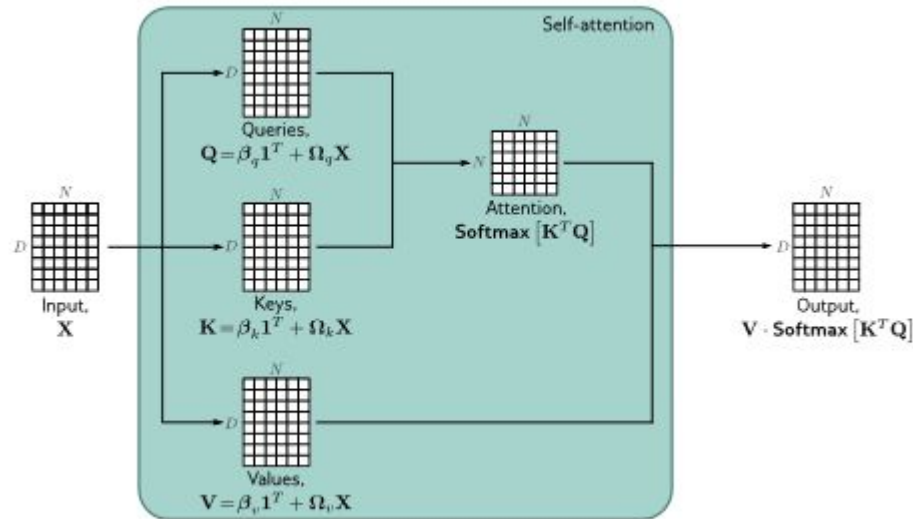
$$Q_i = W_Q x_i, \quad K_i = W_K x_i, \quad V_i = W_V x_i$$

where:

- $W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$,
- $W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$,
- $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$,

are learned weight matrices, and d_k, d_v are the dimensions of the query/key and value vectors respectively (typically $d_k = d_v = d_{\text{model}}$).

Self-attention:



$$\text{scaled_score}(x_i, x_j) = \frac{\mathbf{Q}_i \cdot \mathbf{K}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \frac{\exp\left(\frac{\mathbf{Q}_i \cdot \mathbf{K}_j}{\sqrt{d_k}}\right)}{\sum_{k=1}^n \exp\left(\frac{\mathbf{Q}_i \cdot \mathbf{K}_k}{\sqrt{d_k}}\right)}$$

Here, α_{ij} represents how much focus token x_i should give to token x_j .

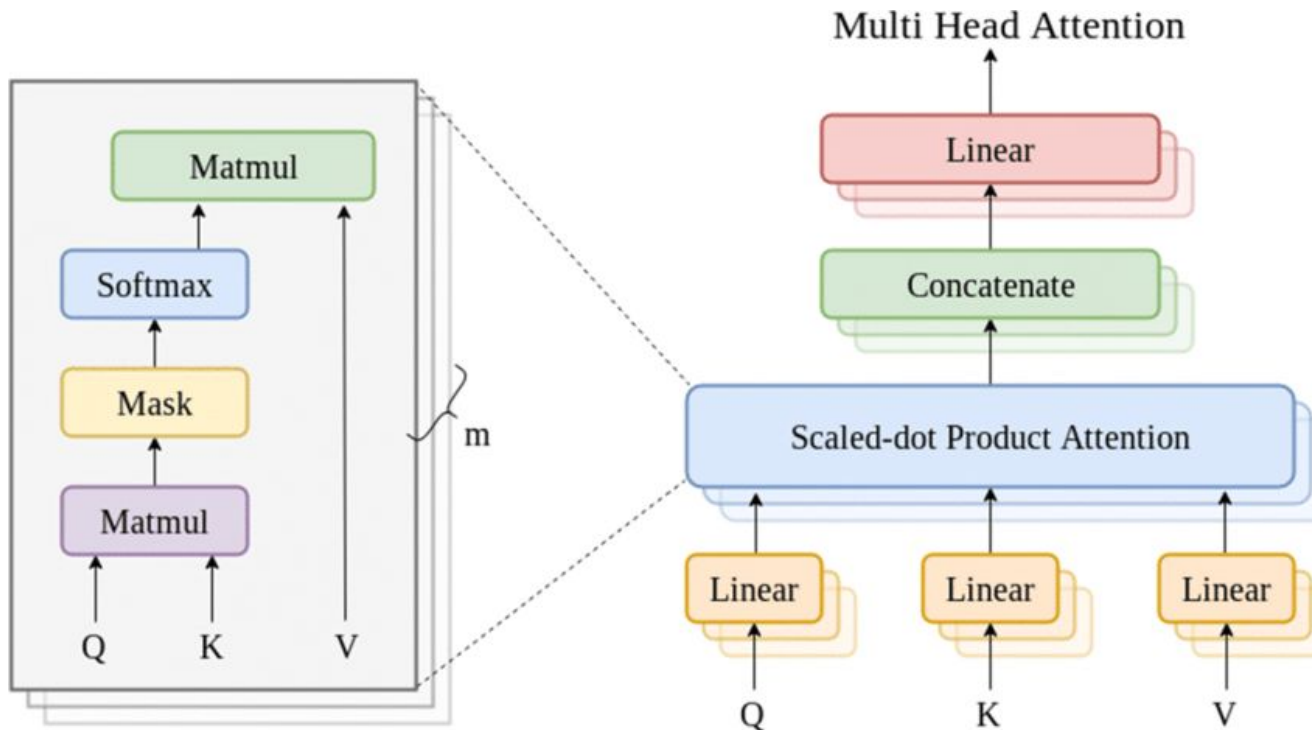
Finally, the output of the self-attention mechanism for token x_i is a weighted sum of the value vectors $\mathbf{V}_j$, where the weights are the attention scores α_{ij} :

$$\text{Attention}(x_i) = \sum_{j=1}^n \alpha_{ij} \mathbf{V}_j$$

This process is done for each token in the sequence, resulting in an attention-weighted representation of each token, where each token is represented as a combination of all the tokens in the sequence, weighted by attention scores.

Self-Attention and multihead:

Train multiple in parallel and let each focus in different things.



1. Multiple Attention Heads

Instead of computing a single set of queries, keys, and values, **multi-head attention** computes multiple sets of queries, keys, and values (one for each head). Suppose there are h heads, then for each head i , we have:

$$\mathbf{Q}_i = W_Q^i x, \quad \mathbf{K}_i = W_K^i x, \quad \mathbf{V}_i = W_V^i x$$

where W_Q^i , W_K^i , and W_V^i are the weight matrices specific to head i .

2. Independent Attention Computation

Each head computes self-attention independently, as described in the self-attention section. For each token, we get an output from each head:

$$\text{Attention}_i(x) = \sum_{j=1}^n \alpha_{ij} \mathbf{V}_j^i$$

3. Concatenation of Heads

The outputs of the h attention heads are concatenated:

$$\text{Concat}(\text{Attention}_1(x), \text{Attention}_2(x), \dots, \text{Attention}_h(x))$$

This results in a vector of size $h \cdot d_v$ for each token, where d_v is the dimensionality of the value vectors in each head.

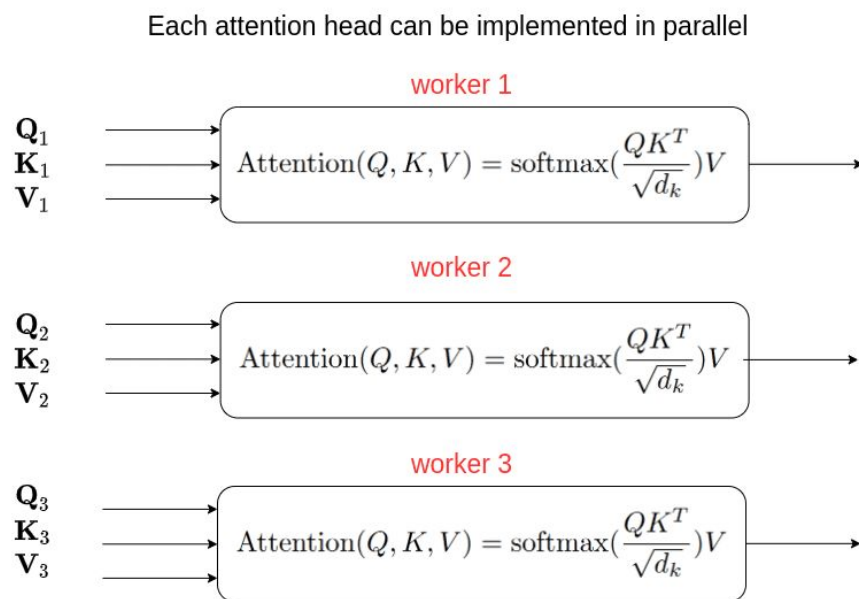
4. Final Linear Transformation

The concatenated output is then passed through a final linear transformation (with a weight matrix W_O) to produce the final output of the multi-head attention layer:

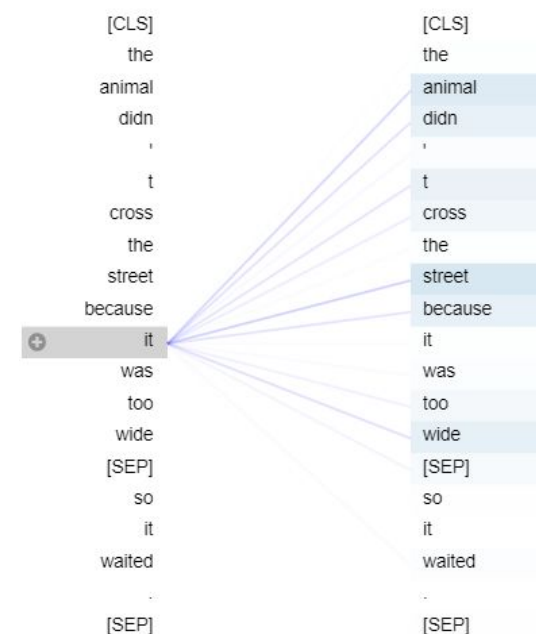
$$\text{MultiHeadAttention}(x) = W_O \cdot \text{Concat}(\text{Attention}_1(x), \dots, \text{Attention}_h(x))$$

Multihead-Attention:

It is called attention because the softmax output looks for relationship between input words, i.e. where to pay attention and then mix the information.



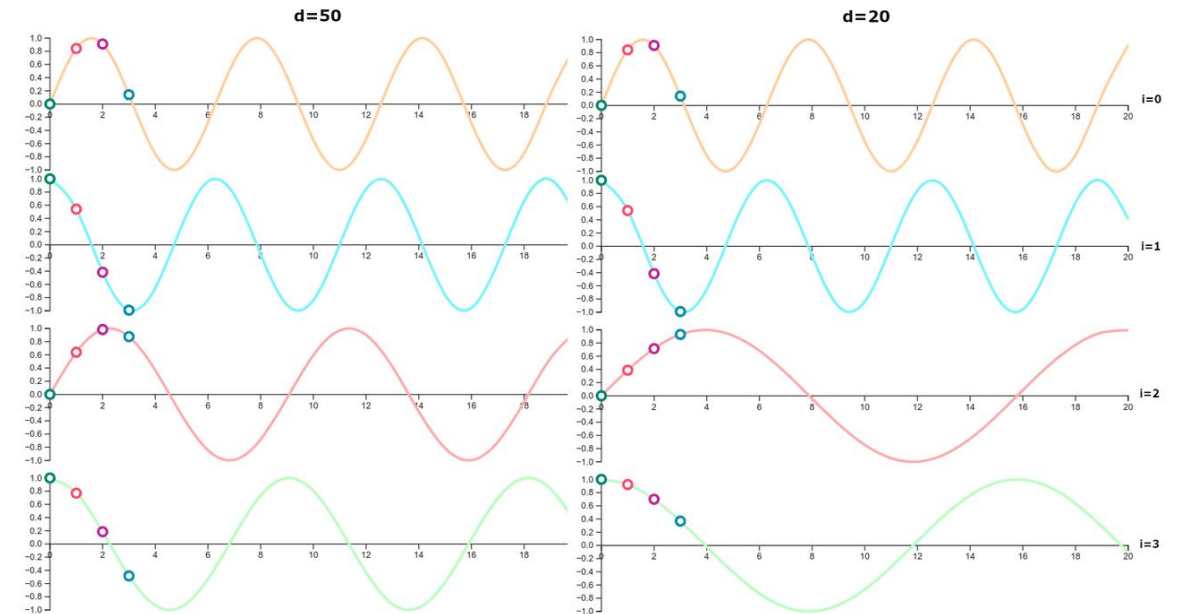
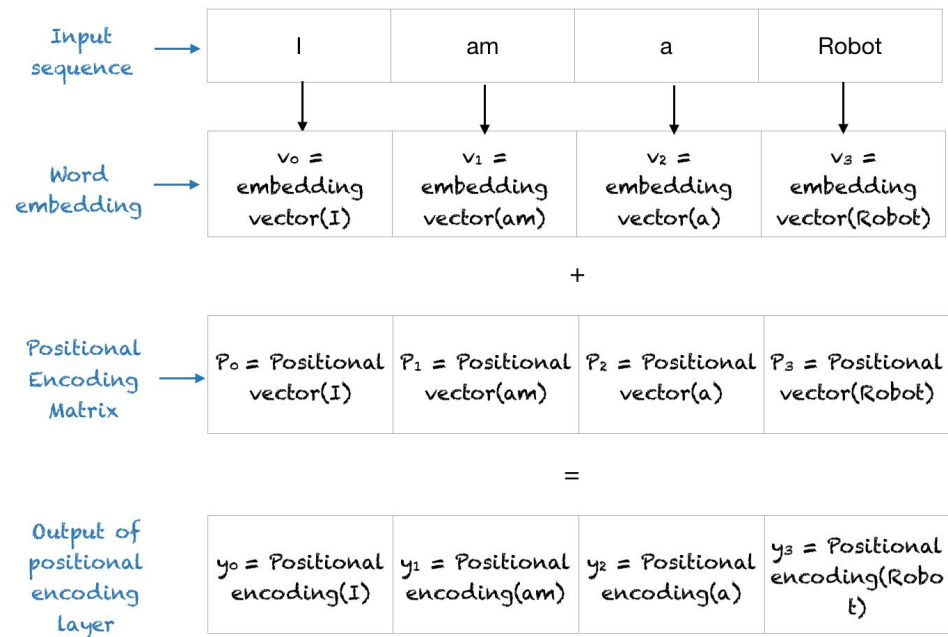
Layer: 0 Head: 1 Attention: All



Does order of words matter?

Positional encodings:

Transformers have no notion of relative position, therefore, we encode this information by adding encodings to the embedded inputs (add a Matrix).

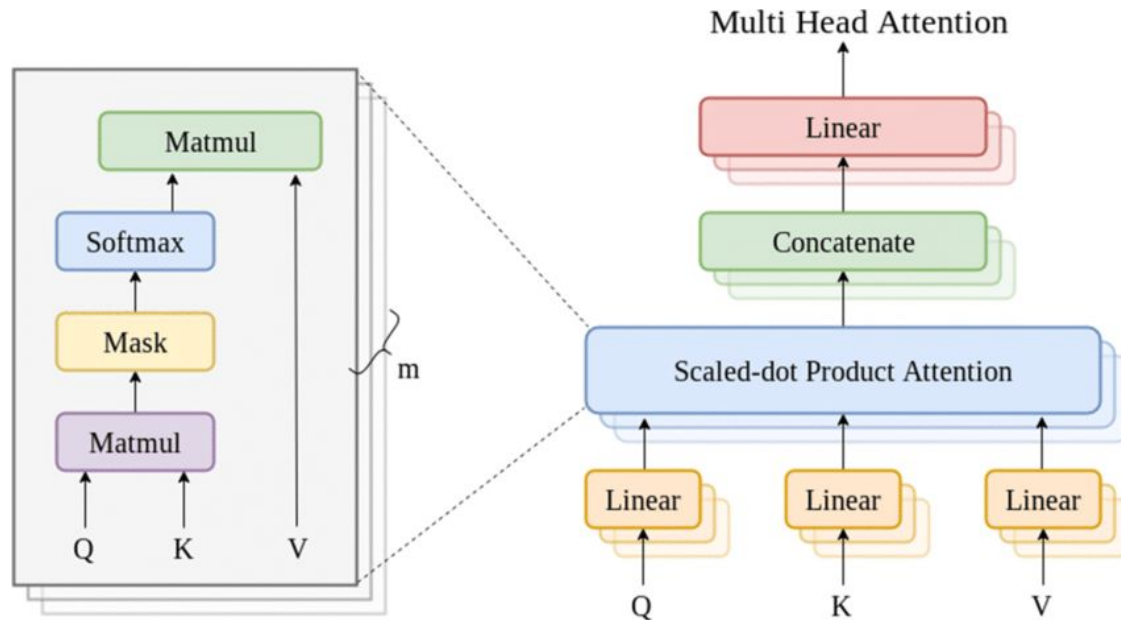


$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

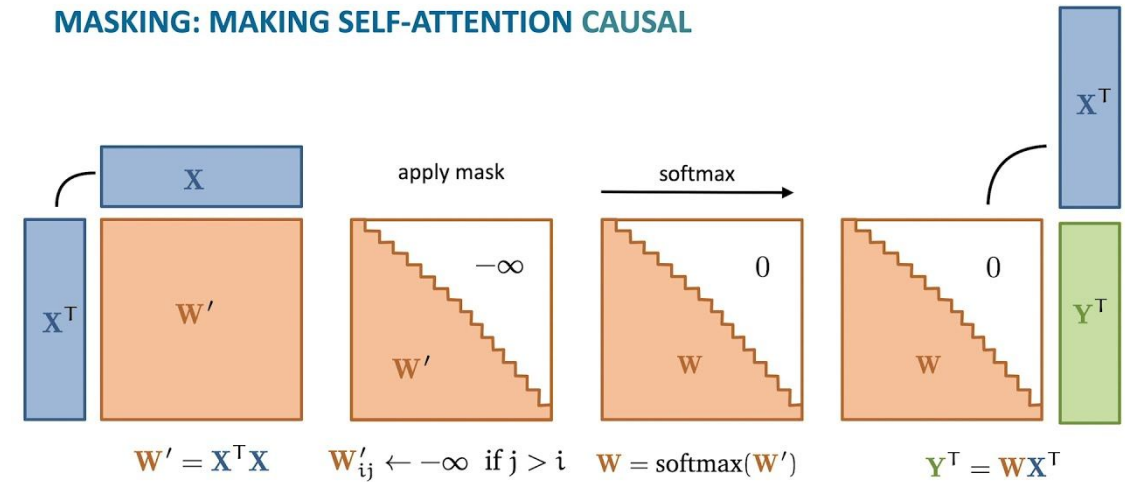
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Self-Attention mask:

Mask is optional, but it is used for applications where it does not make sense for a word to focus on future words because it will not be the case in reality. For example, not used for translation, but used for next word prediction.



MASKING: MAKING SELF-ATTENTION CAUSAL



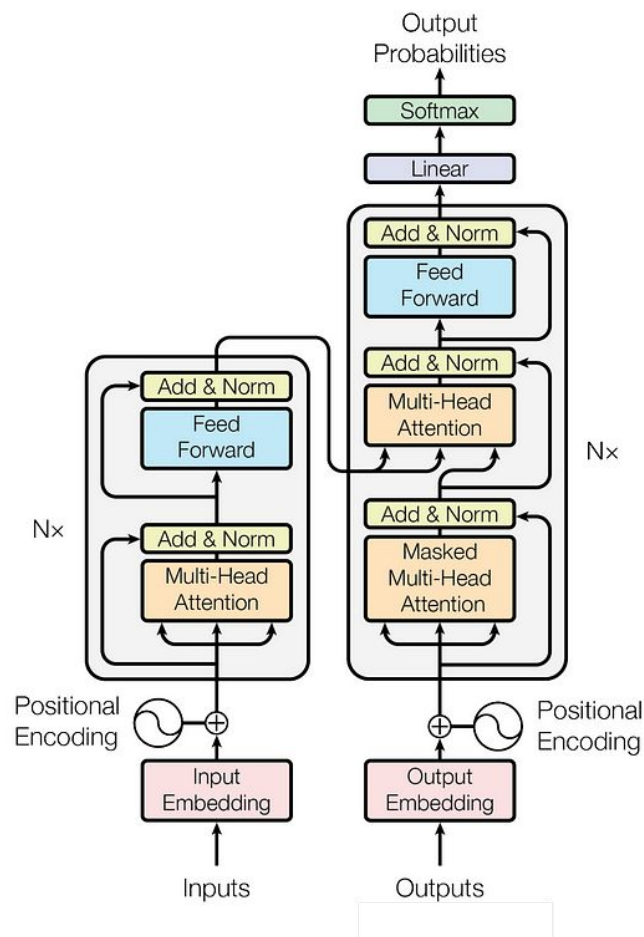
Transformer encoder-decoder:

BERT

Encoder

GPT

Decoder



Demo GPT:

Can you please come **here** ?

History

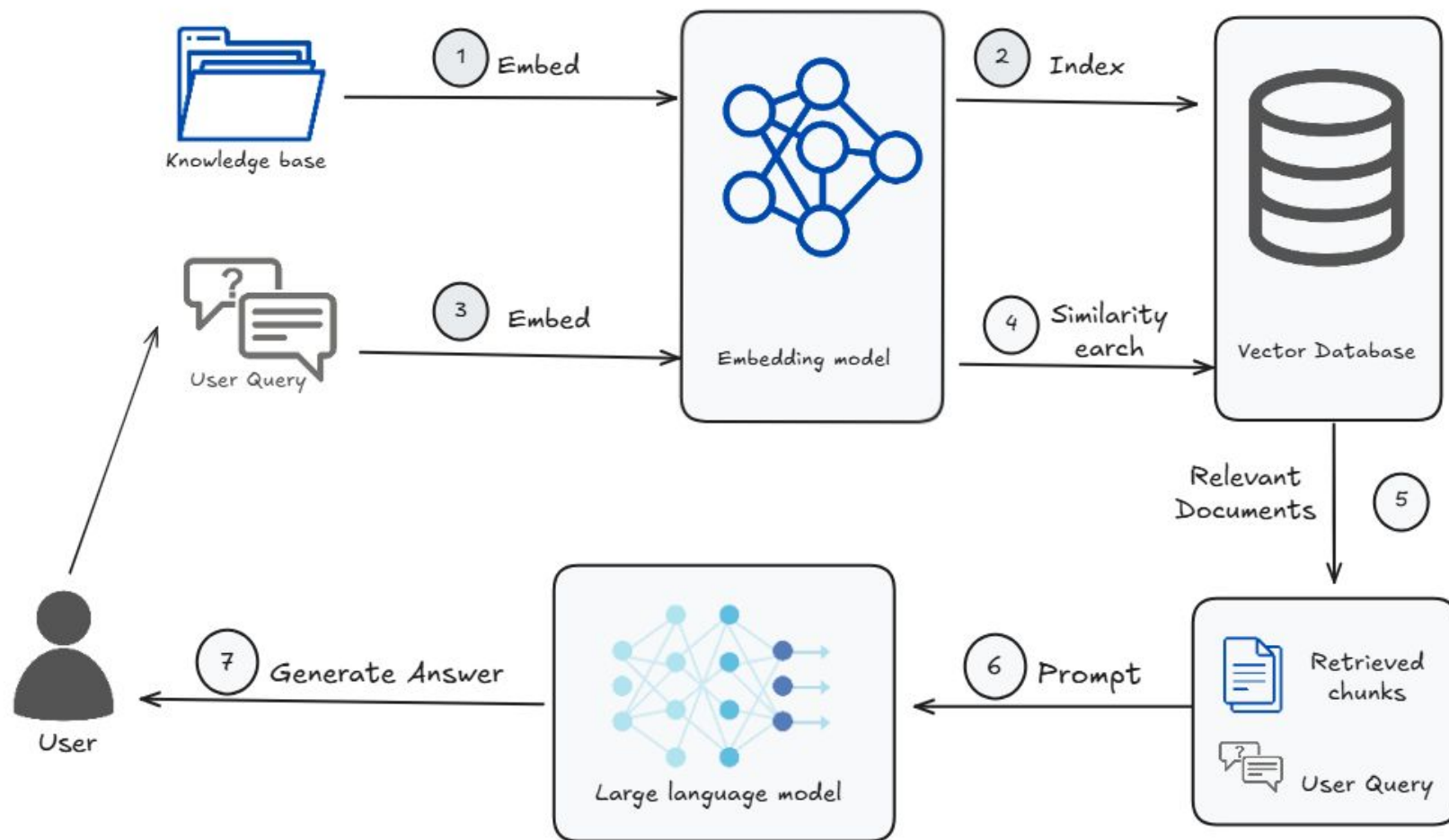
Word being predicted

[Demo GPT](#)

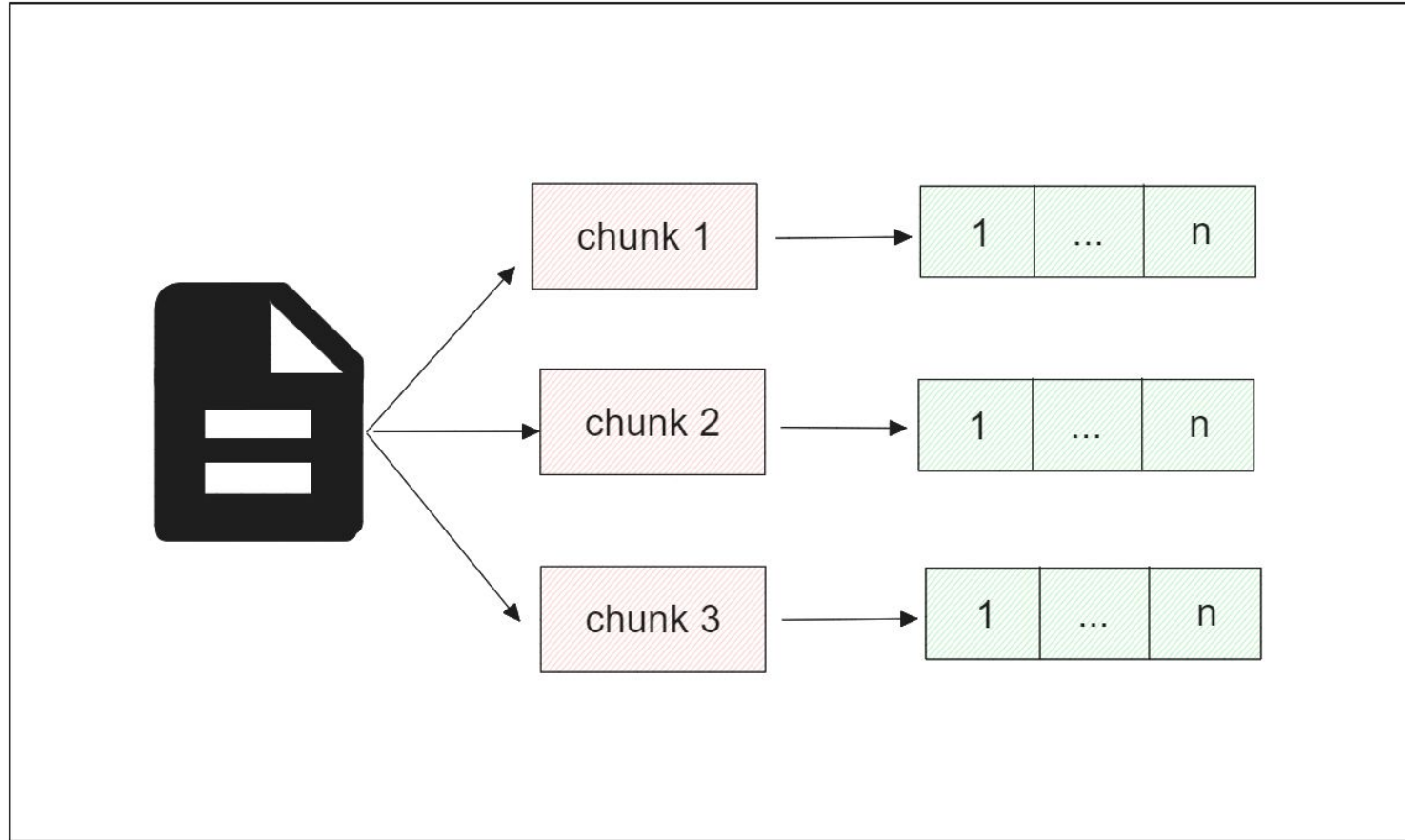
How do companies construct bots with custom data?



Retrieval-Augmented Generation (RAG)

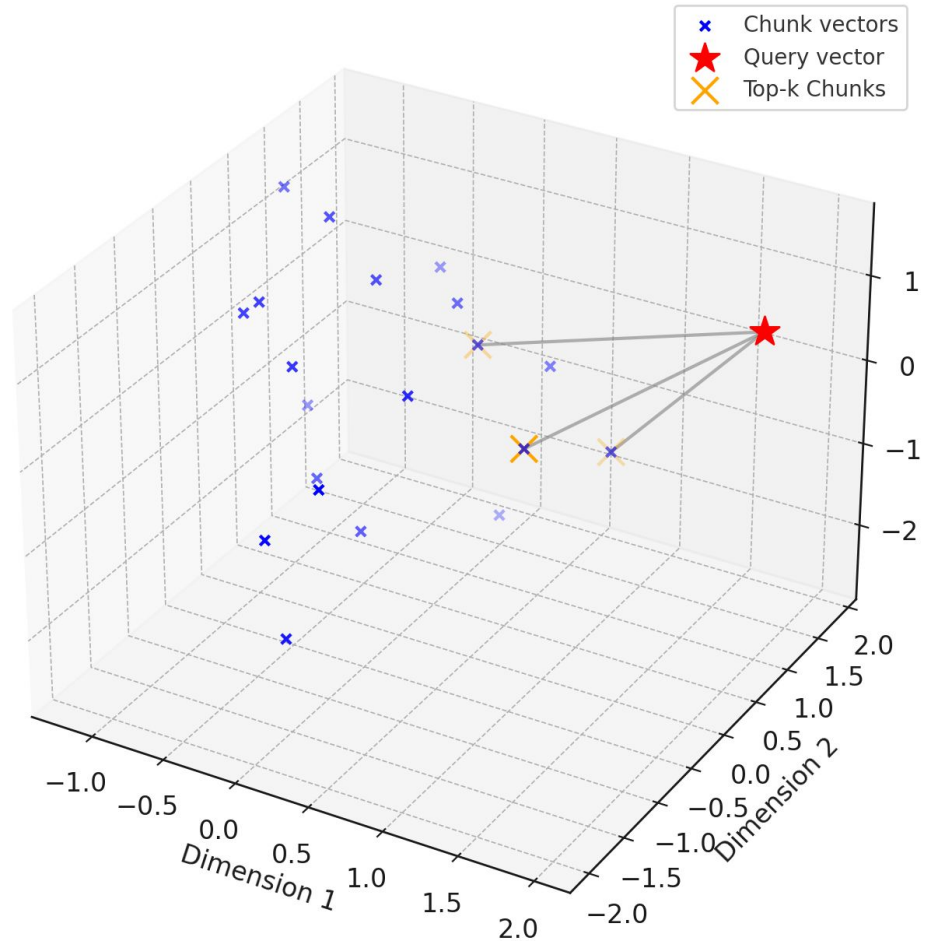


Chunking and embedding generation:



Chunk selection:

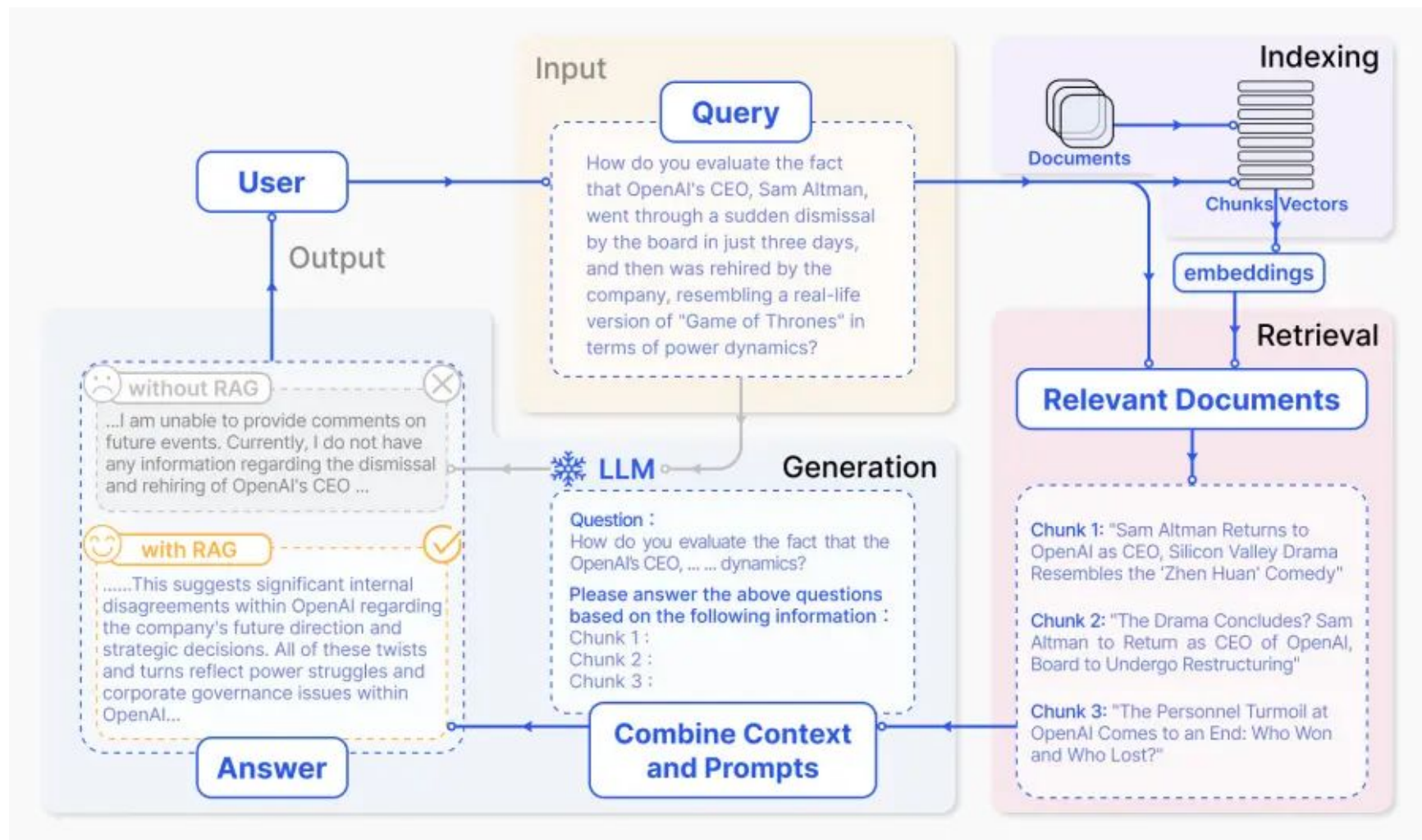
Query and Chunk Embeddings in 3D Vector Space



- dim - 512, 768, 1024..
- Cosine similarity to select top-k

[Lab embeddings](#)

Prompt:



Exercise lightning:

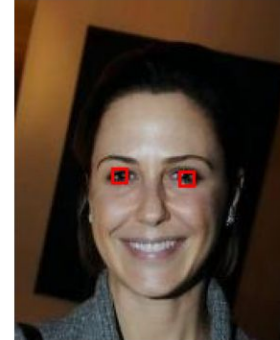
female • age≈39



female • age≈33



female • age≈37



female • age≈35



male • age≈39



male • age≈41



Faces Demo

Demo lightning:

[illegible]

1. *Book title*
 Author, *Year*
 Ed. *Year*
 2. *Book or corporate entity*
 3. *Book title* in *Year*

[illegible]

© 2000 Blackwell Science Ltd
Journal of Internal Medicine 247: 399–406

[illegible]

9. `100V_persec_Lightning@yolo1e`

Exercise lightning:

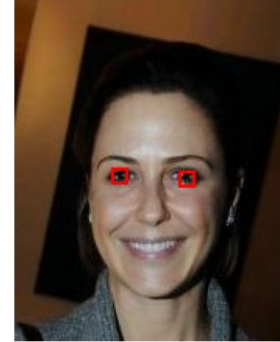
female • age≈39



female • age≈33



female • age≈37



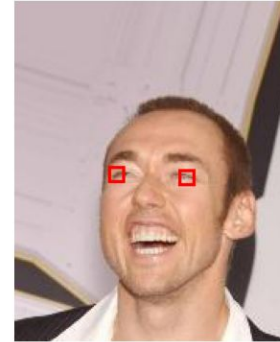
female • age≈35



male • age≈39



male • age≈41



Faces Demo - lightning

Neural Networks

Máster en Innovación Tecnológica (MINT)

imanolgranada@gmail.com